

tomers, find key features of customers, such as salary, education level, job history, and so on, that help in prediction of loan default. The bank would then build a prediction model (such as a decision tree, which we study later in this chapter) using the chosen features. When a customer then applies for a loan, the features of that particular customer are fed into the model which makes a prediction, such as an estimated likelihood of loan default. The prediction is used to make business decisions, such as whether to give a loan to the customer.

Similarly, analysts may look at the past history of sales and use it to predict future sales, to make decisions on what and how much to manufacture or order, or how to target their advertising. For example, a car company may search for customer attributes that help predict who buys different types of cars. It may find that most of its small sports cars are bought by young women whose annual incomes are above \$50,000. The company may then target its marketing to attract more such women to buy its small sports cars and may avoid wasting money trying to attract other categories of people to buy those cars.

Machine-learning techniques are key to finding patterns in data and in making predictions from these patterns. The field of *data mining* combines knowledge-discovery techniques invented by machine-learning researchers with efficient implementation techniques that enable them to be used on extremely large databases. Section 11.4 discusses data mining.

The term **business intelligence (BI)** is used in a broadly similar sense to data analytics. The term **decision support** is also used in a related but narrower sense, with a focus on reporting and aggregation, but not including machine learning/data mining. Decision-support tasks typically use SQL queries to process large amounts of data. Decision-support queries can be contrasted with queries for *online transaction processing*, where each query typically reads only a small amount of data and may perform a few small updates.

11.2 Data Warehousing

Large organizations have a complex internal organization structure, and therefore different data may be present in different locations, or on different operational systems, or under different schemas. For instance, manufacturing-problem data and customer-complaint data may be stored on different database systems. Organizations often purchase data from external sources, such as mailing lists that are used for product promotions, or credit scores of customers that are provided by credit bureaus, to decide on creditworthiness of customers.¹

Corporate decision makers require access to information from multiple such sources. Setting up queries on individual sources is both cumbersome and inefficient.

¹Credit bureaus are companies that gather information about consumers from multiple sources and compute a creditworthiness score for each consumer.

Moreover, the sources of data may store only current data, whereas decision makers may need access to past data as well; for instance, information about how purchase patterns have changed in the past few years could be of great importance. Data warehouses provide a solution to these problems.

A **data warehouse** is a repository (or archive) of information gathered from multiple sources, stored under a unified schema, at a single site. Once gathered, the data are stored for a long time, permitting access to historical data. Thus, data warehouses provide the user a single consolidated interface to data, making decision-support queries easier to write. Moreover, by accessing information for decision support from a data warehouse, the decision maker ensures that online transaction-processing systems are not affected by the decision-support workload.

11.2.1 Components of a Data Warehouse

Figure 11.1 shows the architecture of a typical data warehouse and illustrates the gathering of data, the storage of data, and the querying and data analysis support. Among the issues to be addressed in building a warehouse are the following:

- **When and how to gather data.** In a **source-driven architecture** for gathering data, the data sources transmit new information, either continually (as transaction processing takes place), or periodically (nightly, for example). In a **destination-driven architecture**, the data warehouse periodically sends requests for new data to the sources.

Unless updates at the sources are “synchronously” replicated at the warehouse, the warehouse will never be quite up-to-date with the sources. Synchronous replication can be expensive, so many data warehouses do not use synchronous replication, and they perform queries only on data that are old enough that they have

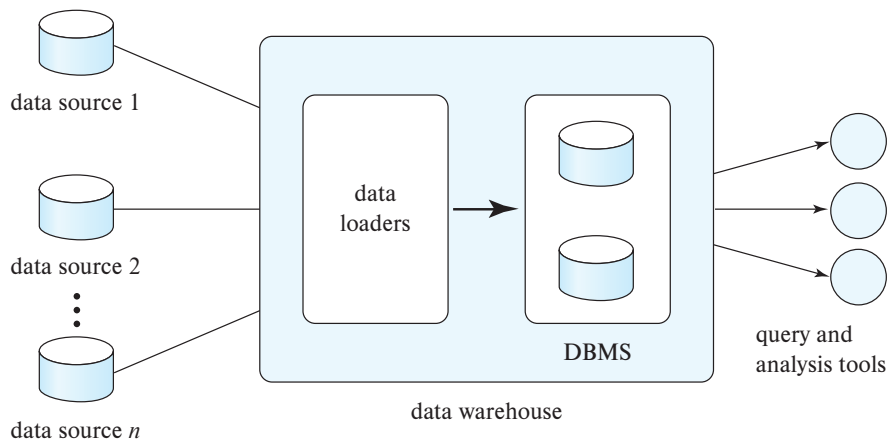


Figure 11.1 Data-warehouse architecture.

been completely replicated. Traditionally, analysts were happy with using yesterday's data, so data warehouses could be loaded with data up to the end of the previous day. However, increasingly organizations want more up-to-date data. The data freshness requirements depend on the application. Data that are within a few hours old may be sufficient for some applications; others that require real-time responses to events may use stream processing infrastructure (described in Section 10.5) instead of depending on a warehouse infrastructure.

- **What schema to use.** Data sources that have been constructed independently are likely to have different schemas. In fact, they may even use different data models. Part of the task of a warehouse is to perform schema integration and to convert data to the integrated schema before they are stored. As a result, the data stored in the warehouse are not just a copy of the data at the sources. Instead, they can be thought of as a materialized view of the data at the sources.
- **Data transformation and cleansing.** The task of correcting and preprocessing data is called **data cleansing**. Data sources often deliver data with numerous minor inconsistencies, which can be corrected. For example, names are often misspelled, and addresses may have street, area, or city names misspelled, or postal codes entered incorrectly. These can be corrected to a reasonable extent by consulting a database of street names and postal codes in each city. The approximate matching of data required for this task is referred to as **fuzzy lookup**.

Address lists collected from multiple sources may have duplicates that need to be eliminated in a **merge-purge operation** (this operation is also referred to as **deduplication**). Records for multiple individuals in a house may be grouped together so only one mailing is sent to each house; this operation is called **householding**.

Data may be **transformed** in ways other than cleansing, such as changing the units of measure, or converting the data to a different schema by joining data from multiple source relations. Data warehouses typically have graphical tools to support data transformation. Such tools allow transformation to be specified as boxes, and edges can be created between boxes to indicate the flow of data. Conditional boxes can route data to an appropriate next step in transformation.

- **How to propagate updates.** Updates on relations at the data sources must be propagated to the data warehouse. If the relations at the data warehouse are exactly the same as those at the data source, the propagation is straightforward. If they are not, the problem of propagating updates is basically the *view-maintenance* problem, which was discussed in Section 4.2.3, and is covered in more detail in Section 16.5.
- **What data to summarize.** The raw data generated by a transaction-processing system may be too large to store online. However, we can answer many queries by maintaining just summary data obtained by aggregation on a relation, rather than maintaining the entire relation. For example, instead of storing data about every sale of clothing, we can store total sales of clothing by item name and category.

The different steps involved in getting data into a data warehouse are called **extract, transform, and load** or **ETL** tasks; extraction refers to getting data from the sources, while load refers to loading the data into the data warehouse. In current generation data warehouses that support user-defined functions or MapReduce frameworks, data may be extracted, loaded into the warehouse, and then transformed. The steps are then referred to as **extract, load, and transform** or **ELT** tasks. The ELT approach permits the use of parallel processing frameworks for data transformation.

11.2.2 Multidimensional Data and Warehouse Schemas

Data warehouses typically have schemas that are designed for data analysis, using tools such as OLAP tools. The relations in a data **warehouse schema** can usually be classified as *fact tables* and *dimension tables*. **Fact tables** record information about individual events, such as sales, and are usually very large. A table recording sales information for a retail store, with one tuple for each item that is sold, is a typical example of a fact table. The attributes in fact table can be classified as either *dimension attributes* or *measure attributes*. The **measure attributes** store quantitative information, which can be aggregated upon; the measure attributes of a *sales* table would include the number of items sold and the price of the items. In contrast, **dimension attributes** are dimensions upon which measure attributes, and summaries of measure attributes, are grouped and viewed. The dimension attributes of a *sales* table would include an item identifier, the date when the item is sold, which location (store) the item was sold from, the customer who bought the item, and so on.

Data that can be modeled using dimension attributes and measure attributes are called **multidimensional data**.

To minimize storage requirements, dimension attributes are usually short identifiers that are foreign keys into other tables called **dimension tables**. For instance, a fact table *sales* would have dimension attributes *item_id*, *store_id*, *customer_id*, and *date*, and measure attributes *number* and *price*. The attribute *store_id* is a foreign key into a dimension table *store*, which has other attributes such as store location (city, state, country). The *item_id* attribute of the *sales* table would be a foreign key into a dimension table *item_info*, which would contain information such as the name of the item, the category to which the item belongs, and other item details such as color and size. The *customer_id* attribute would be a foreign key into a *customer* table containing attributes such as name and address of the customer. We can also view the *date* attribute as a foreign key into a *date_info* table giving the month, quarter, and year of each date.

The resultant schema appears in Figure 11.2. Such a schema, with a fact table, multiple dimension tables, and foreign keys from the fact table to the dimension tables is called a **star schema**. More complex data-warehouse designs may have multiple levels of dimension tables; for instance, the *item_info* table may have an attribute *manufacturer_id* that is a foreign key into another table giving details of the manufacturer. Such schemas are called **snowflake schemas**. Complex data-warehouse designs may also have more than one fact table.

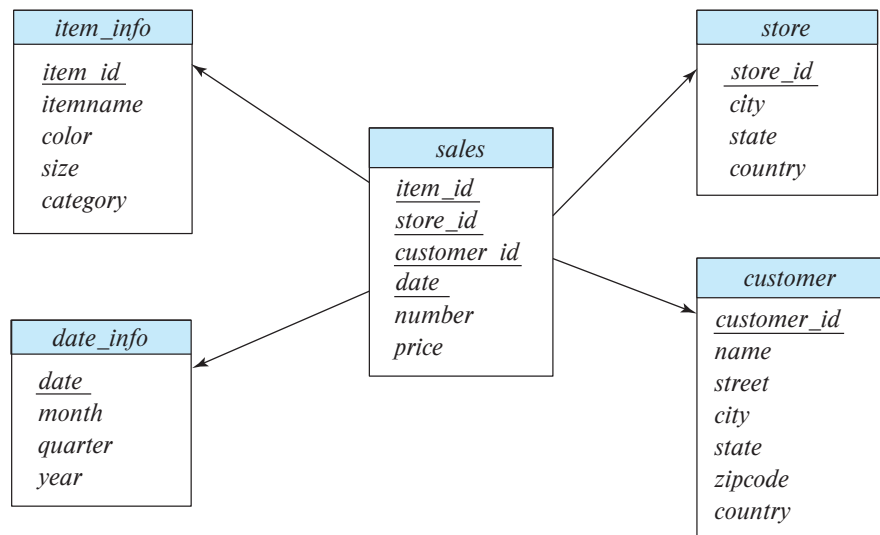


Figure 11.2 Star schema for a data warehouse.

11.2.3 Database Support for Data Warehouses

The requirements of a database system designed for transaction processing are somewhat different from one designed to support a data-warehouse system. One key difference is that a transaction-processing database needs to support many small queries, which may involve updates in addition to reads. In contrast, data warehouses typically need to process far fewer queries, but each query accesses a much larger amount of data.

Most importantly, while new records are inserted into relations in a data warehouse, and old records may be deleted once they are no longer needed, to make space for new data, records are typically never updated once they are added to a relation. Thus, data warehouses do not need to pay any overhead for concurrency control. (As described in Chapter 17 and Chapter 18, if concurrent transactions read and write the same data, the resultant data may become inconsistent. Concurrency control restricts concurrent accesses in a way that ensures there is no erroneous update to the database.) The overhead of concurrency control can be significant in terms of not just time taken for query processing, but also in terms of storage, since databases often store multiple versions of data to avoid conflicts between small update transactions and long read-only transactions. None of these overheads are needed in a data warehouse.

Databases traditionally store all attributes of a tuple together, and tuples are stored sequentially in a file. Such a storage layout is referred to as *row-oriented storage*. In contrast, in **column-oriented storage**, each attribute of a relation is stored in a separate file, with values from successive tuples stored at successive positions in the file. Assuming fixed-size data types, the value of attribute *A* of the *i*th tuple of a relation can be found

by accessing the file corresponding to attribute A and reading the value at offset $(i - 1)$ times the size (in bytes) of values in attribute A .

Column-oriented storage has at least two major benefits over row-oriented storage:

1. When a query needs to access only a few attributes of a relation with a large number of attributes, the remaining attributes need not be fetched from disk into memory. In contrast, in row-oriented storage, not only are irrelevant attributes fetched into memory, but they may also get prefetched into processor cache, wasting cache space and memory bandwidth, if they are stored adjacent to attributes used in the query.
2. Storing values of the same type together increases the effectiveness of compression; compression can greatly reduce both the disk storage cost and the time to retrieve data from disk.

On the other hand, column-oriented storage has the drawback that storing or fetching a single tuple requires multiple I/O operations.

As a result of these trade-offs, column-oriented storage is not widely used for transaction-processing applications. However, column-oriented storage is today widely used for data-warehousing applications, where accesses are rarely to individual tuples but rather require scanning and aggregating multiple tuples. Column-oriented storage is described in more detail in Section 13.6.

Database implementations that are designed purely for data warehouse applications include Teradata, Sybase IQ, and Amazon Redshift. Many traditional databases support efficient execution of data warehousing applications by adding features such as columnar storage; these include Oracle, SAP HANA, Microsoft SQL Server, and IBM DB2.

In the 2010s there has been an explosive growth in Big Data systems that are designed to process queries over data stored in files. Such systems are now a key part of the data warehouse infrastructure. As we saw in Section 10.3, the motivation for such systems was the growth of data generated by online systems in the form of log files, which have a lot of valuable information that can be exploited for decision support. However, these systems can handle any kind of data, including relational data. Apache Hadoop is one such system, and the Hive system allows SQL queries to be executed on top of the Hadoop system.

A number of companies provide software to optimize Hive query processing, including Cloudera and Hortonworks. Apache Spark is another popular Big Data system that supports SQL queries on data stored in files. Compressed file structures containing records with columns, such as Orc and Parquet, are increasingly used to store such log records, simplifying integration with SQL. Such file formats are discussed in more detail in Section 13.6.

11.2.4 Data Lakes

While data warehouses pay a lot of attention to ensuring a common data schema to ease the job of querying the data, there are situations where organizations want to store data without paying the cost of creating a common schema and transforming data to the common schema. The term **data lake** is used to refer to a repository where data can be stored in multiple formats, including structured records and unstructured file formats. Unlike data warehouses, data lakes do not require up-front effort to preprocess data, but they do require more effort when creating queries. Since data may be stored in many different formats, querying tools also need to be quite flexible. Apache Hadoop and Apache Spark are popular tools for querying such data, since they support querying of both unstructured and structured data.

11.3 Online Analytical Processing

Data analysis often involves looking for patterns that arise when data values are grouped in “interesting” ways. As a simple example, summing credit hours for each department is a way to discover which departments have high teaching responsibilities. In a retail business, we might group sales by product, the date or month of the sale, the color or size of the product, or the profile (such as age group and gender) of the customer who bought the product.

11.3.1 Aggregation on Multidimensional Data

Consider an application where a shop wants to find out what kinds of clothes are popular. Let us suppose that clothes are characterized by their *item_name*, *color*, and *size*, and that we have a relation *sales* with the schema.

sales (*item_name*, *color*, *clothes_size*, *quantity*)

Suppose that *item_name* can take on the values (skirt, dress, shirt, pants), *color* can take on the values (dark, pastel, white), *clothes_size* can take on values (small, medium, large), and *quantity* is an integer value representing the total number of items of a given {*item_name*, *color*, *clothes_size*}. An instance of the *sales* relation is shown in Figure 11.3.

Statistical analysis often requires grouping on multiple attributes. The attribute *quantity* of the *sales* relation is a measure attribute, since it measures the number of units sold, while *item_name*, *color*, and *clothes_size* are dimension attributes. (A more realistic version of the *sales* relation would have additional dimensions, such as time and sales location, and additional measures such as monetary value of the sale.)

To analyze the multidimensional data, a manager may want to see data laid out as shown in the table in Figure 11.4. The table shows total quantities for different combinations of *item_name* and *color*. The value of *clothes_size* is specified to be **all**, indicating that the displayed values are a summary across all values of *clothes_size* (i.e., we want to group the “small,” “medium,” and “large” items into one single group.

<i>item_name</i>	<i>color</i>	<i>clothes_size</i>	<i>quantity</i>
dress	dark	small	2
dress	dark	medium	6
dress	dark	large	12
dress	pastel	small	4
dress	pastel	medium	3
dress	pastel	large	3
dress	white	small	2
dress	white	medium	3
dress	white	large	0
pants	dark	small	14
pants	dark	medium	6
pants	dark	large	0
pants	pastel	small	1
pants	pastel	medium	0
pants	pastel	large	1
pants	white	small	3
pants	white	medium	0
pants	white	large	2
shirt	dark	small	2
shirt	dark	medium	6
shirt	dark	large	6
shirt	pastel	small	4
shirt	pastel	medium	1
shirt	pastel	large	2
shirt	white	small	17
shirt	white	medium	1
shirt	white	large	10
skirt	dark	small	2
skirt	dark	medium	5
skirt	dark	large	1
skirt	pastel	small	11
skirt	pastel	medium	9
skirt	pastel	large	15
skirt	white	small	2
skirt	white	medium	5
skirt	white	large	3

Figure 11.3 An example of *sales* relation.

The table in Figure 11.4 is an example of a **cross-tabulation** (or **cross-tab**, for short), also referred to as a **pivot-table**. In general, a cross-tab is a table derived from a relation

<i>clothes_size</i> all		<i>color</i>			
<i>item_name</i>		dark	pastel	white	total
	skirt	8	35	10	53
	dress	20	10	5	35
	shirt	14	7	28	49
	pants	20	2	5	27
	total	62	54	48	164

Figure 11.4 Cross-tabulation of *sales* by *item_name* and *color*.

(say R), where values for one attribute (say A) form the column headers and values for another attribute (say B) form the row header. For example, in Figure 11.4, the attribute *color* corresponds to A (with values “dark,” “pastel,” and “white”), and the attribute *item_name* corresponds to B (with values “skirt,” “dress,” “shirt,” and “pants”).

Each cell in the pivot-table can be identified by (a_i, b_j) , where a_i is a value for A and b_j a value for B . The values of the various cells in the pivot-table are derived from the relation R as follows: If there is at most one tuple in R with any (a_i, b_j) value, the value in the cell is derived from that single tuple (if any); for instance, it could be the value of one or more other attributes of the tuple. If there can be multiple tuples with an (a_i, b_j) value, the value in the cell must be derived by aggregation on the tuples with that value. In our example, the aggregation used is the sum of the values for attribute *quantity*, across all values for *clothes_size*, as indicated by “*clothes_size*: **all**” above the cross-tab in Figure 11.4. Thus, the value for cell (skirt, pastel) is 35, since there are three tuples in the *sales* table that meet that criteria, with values 11, 9, and 15.

In our example, the cross-tab also has an extra column and an extra row storing the totals of the cells in the row/column. Most cross-tabs have such summary rows and columns.

The generalization of a cross-tab, which is two-dimensional, to n dimensions can be visualized as an n -dimensional cube, called the **data cube**. Figure 11.5 shows a data cube on the *sales* relation. The data cube has three dimensions, *item_name*, *color*, and *clothes_size*, and the measure attribute is *quantity*. Each cell is identified by values for these three dimensions. Each cell in the data cube contains a value, just as in a cross-tab. In Figure 11.5, the value contained in a cell is shown on one of the faces of the cell; other faces of the cell are shown blank if they are visible. All cells contain values, even if they are not visible. The value for a dimension may be **all**, in which case the cell contains a summary over all values of that dimension, as in the case of cross-tabs.

The number of different ways in which the tuples can be grouped for aggregation can be large. In the example of Figure 11.5, there are 3 colors, 4 items, and 3 sizes resulting in a cube size of $3 \times 4 \times 3 = 36$. Including the summary values, we obtain a

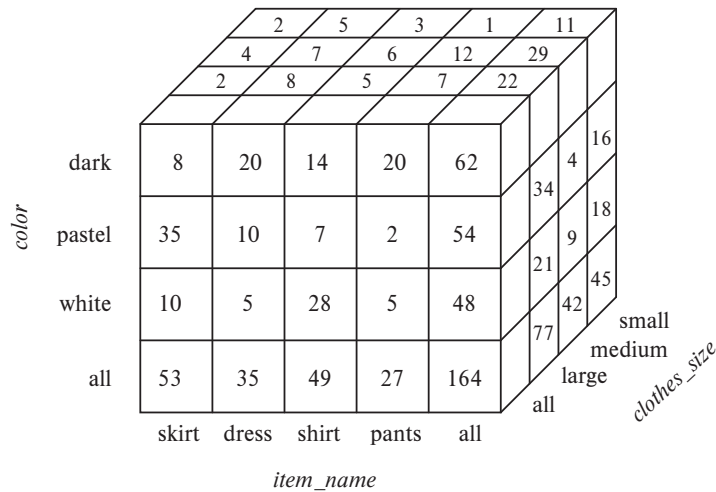


Figure 11.5 Three-dimensional data cube.

$4 \times 5 \times 4$ cube, whose size is 80. In fact, for a table with n dimensions, aggregation can be performed with grouping on each of the 2^n subsets of the n dimensions.²

An **online analytic processing** (OLAP) system allows a data analyst to look at different cross-tabs on the same data by *interactively* selecting the attributes in the cross-tab. Each cross-tab is a two-dimensional view on a multidimensional data cube. For instance, the analyst may select a cross-tab on *item_name* and *clothes_size* or a cross-tab on *color* and *clothes_size*. The operation of changing the dimensions used in a cross-tab is called **pivoting**.

OLAP systems allow an analyst to see a cross-tab on *item_name* and *color* for a fixed value of *clothes_size*, for example, large, instead of the sum across all sizes. Such an operation is referred to as **slicing**, since it can be thought of as viewing a slice of the data cube. The operation is sometimes called **dicing**, particularly when values for multiple dimensions are fixed.

When a cross-tab is used to view a multidimensional cube, the values of dimension attributes that are not part of the cross-tab are shown above the cross-tab. The value of such an attribute can be **all**, as shown in Figure 11.4, indicating that data in the cross-tab are a summary over all values for the attribute. Slicing/dicing simply consists of selecting specific values for these attributes, which are then displayed on top of the cross-tab.

OLAP systems permit users to view data at any desired level of granularity. The operation of moving from finer-granularity data to a coarser granularity (by means of aggregation) is called a **rollup**. In our example, starting from the data cube on the

²Grouping on the set of all n dimensions is useful only if the table may have duplicates.

sales table, we got our example cross-tab by rolling up on the attribute *clothes_size*. The opposite operation—that of moving from coarser-granularity data to finer-granularity data—is called a **drill down**. Finer-granularity data cannot be generated from coarse-granularity data; they must be generated either from the original data or from even finer-granularity summary data.

Analysts may wish to view a dimension at different levels of detail. For instance, consider an attribute of type **datetime** that contains a date and a time of day. Using time precise to a second (or less) may not be meaningful: An analyst who is interested in rough time of day may look at only the hour value. An analyst who is interested in sales by day of the week may map the date to a day of the week and look only at that. Another analyst may be interested in aggregates over a month, or a quarter, or for an entire year.

The different levels of detail for an attribute can be organized into a **hierarchy**. Figure 11.6a shows a hierarchy on the **datetime** attribute. As another example, Figure 11.6b shows a hierarchy on location, with the city being at the bottom of the hierarchy, state above it, country at the next level, and region being the top level. In our earlier example, clothes can be grouped by category (for instance, menswear or womenswear); *category* would then lie above *item_name* in our hierarchy on clothes. At the level of actual values, skirts and dresses would fall under the womenswear category and pants and shirts under the menswear category.

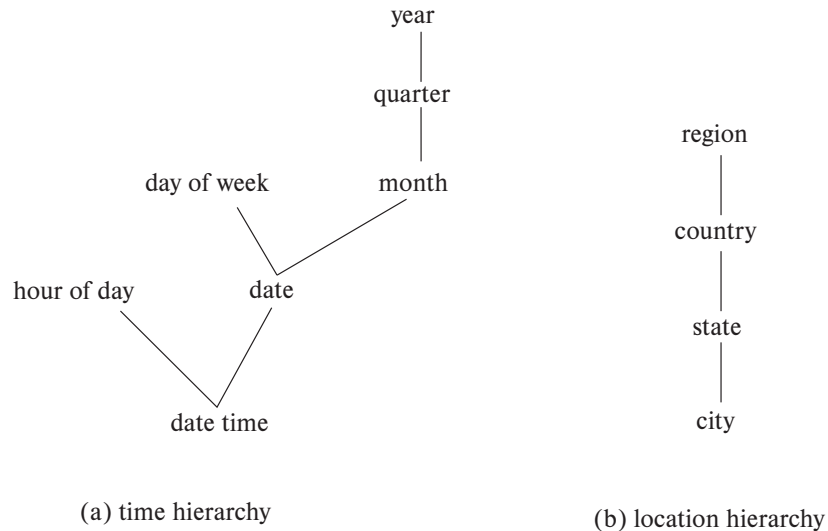


Figure 11.6 Hierarchies on dimensions.

clothes_size: **all**

		<i>color</i>				
<i>category</i>	<i>item_name</i>	dark	pastel	white	total	
womenswear	skirt	8	8	10	53	
	dress	20	20	5	35	
	subtotal	28	28	15		88
menswear	pants	14	14	28	49	
	shirt	20	20	5	27	
	subtotal	34	34	33		76
total		62	62	48		164

Figure 11.7 Cross-tabulation of *sales* with hierarchy on *item_name*.

An analyst may be interested in viewing sales of clothes divided as menswear and womenswear, and not interested in individual values. After viewing the aggregates at the level of womenswear and menswear, an analyst may *drill down the hierarchy* to look at individual values. An analyst looking at the detailed level may *drill up the hierarchy* and look at coarser-level aggregates. Both levels can be displayed on the same cross-tab, as in Figure 11.7.

11.3.2 Relational Representation of Cross-Tabs

A cross-tab is different from relational tables usually stored in databases, since the number of columns in the cross-tab depends on the actual data. A change in the data values may result in adding more columns, which is not desirable for data storage. However, a cross-tab view is desirable for display to users. It is straightforward to represent a cross-tab without summary values in a relational form with a fixed number of columns. A cross-tab with summary rows/columns can be represented by introducing a special value **all** to represent subtotals, as in Figure 11.8. The SQL standard actually uses the *null* value in place of **all**, but to avoid confusion with regular null values, we shall continue to use **all**.

Consider the tuples (skirt, **all**, **all**, 53) and (dress, **all**, **all**, 35). We have obtained these tuples by eliminating individual tuples with different values for *color* and *clothes_size*, and by replacing the value of *quantity* with an aggregate—namely, the sum of the quantities. The value **all** can be thought of as representing the set of all values for an attribute. Tuples with the value **all** for the *color* and *clothes_size* dimensions can be obtained by an aggregation on the *sales* relation with a **group by** on the column *item_name*. Similarly, a **group by** on *color*, *clothes_size* can be used to get the tuples with the value **all** for *item_name*, and a **group by** with no attributes (which can simply be omitted in SQL) can be used to get the tuple with value **all** for *item_name*, *color*, and *clothes_size*.

<i>item_name</i>	<i>color</i>	<i>clothes_size</i>	<i>quantity</i>
skirt	dark	all	8
skirt	pastel	all	35
skirt	white	all	10
skirt	all	all	53
dress	dark	all	20
dress	pastel	all	10
dress	white	all	5
dress	all	all	35
shirt	dark	all	14
shirt	pastel	all	7
shirt	white	all	28
shirt	all	all	49
pants	dark	all	20
pants	pastel	all	2
pants	white	all	5
pants	all	all	27
all	dark	all	62
all	pastel	all	54
all	white	all	48
all	all	all	164

Figure 11.8 Relational representation of the data in Figure 11.4.

Hierarchies can also be represented by relations. For example, the fact that skirts and dresses fall under the womenswear category and the pants and shirts under the menswear category can be represented by a relation *itemcategory* (*item_name*, *category*). This relation can be joined with the *sales* relation to get a relation that includes the category for each item. Aggregation on this joined relation allows us to get a cross-tab with hierarchy. As another example, a hierarchy on city can be represented by a single relation *city_hierarchy* (*ID*, *city*, *state*, *country*, *region*), or by multiple relations, each mapping values in one level of the hierarchy to values at the next level. We assume here that cities have unique identifiers, stored in the attribute *ID*, to avoid confusing between two cities with the same name, for example, the Springfield in Missouri and the Springfield in Illinois.

11.3.3 OLAP in SQL

Analysts using OLAP systems need answers to multiple aggregates to be generated interactively, without having to wait for multiple minutes or hours. This led initially to the development of specialized systems for OLAP (see Note 11.1 on page 535). Many database systems now implement OLAP along with SQL constructs to express OLAP queries. As we saw in Section 5.5.3, several SQL implementations, such as Microsoft

SQL Server and Oracle, support a **pivot** clause that allows creation of cross-tabs. Given the *sales* relation from Figure 11.3, the query:

```
select *
from sales
pivot (
    sum(quantity)
    for color in ('dark','pastel','white')
)
order by item_name;
```

returns the cross-tab shown in Figure 11.9. Note that the **for** clause within the **pivot** clause specifies the *color* values that appear as attribute names in the pivot result. The attribute *color* itself does not appear in the result, although all other attributes are retained, except that the values for the newly created attributes are specified to come from the attribute *quantity*. In case more than one tuple contributes values to a given cell, the aggregate operation within the **pivot** clause specifies how the values should be combined. In the above example, the *quantity* values are summed up.

Note that the pivot clause by itself does not compute the subtotals we saw in the pivot table from Figure 11.4. However, we can first generate the relational representation shown in Figure 11.8, using a cube operation, as outlined shortly, and then apply the pivot clause on that representation to get an equivalent result. In this case, the value **all** must also be listed in the **for** clause, and the **order by** clause needs to be modified to order **all** at the end.

The data in a data cube cannot be generated by a single SQL query if we use only the basic **group by** constructs, since aggregates are computed for several different groupings

<i>item_name</i>	<i>clothes_size</i>	<i>dark</i>	<i>pastel</i>	<i>white</i>
dress	small	2	4	2
dress	medium	6	3	3
dress	large	12	3	0
pants	small	14	1	3
pants	medium	6	0	0
pants	large	0	1	2
shirt	small	2	4	17
shirt	medium	6	1	1
shirt	large	6	2	10
skirt	small	2	11	2
skirt	medium	5	9	5
skirt	large	1	15	3

Figure 11.9 Result of SQL pivot operation on the *sales* relation of Figure 11.3.

Note 11.1 OLAP IMPLEMENTATION

The earliest OLAP systems used multidimensional arrays in memory to store data cubes and are referred to as **multidimensional OLAP (MOLAP)** systems. Later, OLAP facilities were integrated into relational systems, with data stored in a relational database. Such systems are referred to as **relational OLAP (ROLAP)** systems. Hybrid systems, which store some summaries in memory and store the base data and other summaries in a relational database, are called **hybrid OLAP (HOLAP)** systems.

Many OLAP systems are implemented as client-server systems. The server contains the relational database as well as any MOLAP data cubes. Client systems obtain views of the data by communicating with the server.

A naïve way of computing the entire data cube (all groupings) on a relation is to use any standard algorithm for computing aggregate operations, one grouping at a time. The naïve algorithm would require a large number of scans of the relation. A simple optimization is to compute an aggregation on, say, *(item_name, color)* from an aggregation *(item_name, color, clothes_size)*, instead of from the original relation. The amount of data read drops significantly by computing an aggregate from another aggregate, instead of from the original relation. Further improvements are possible; for instance, multiple groupings can be computed on a single scan of the data.

Early OLAP implementations precomputed and stored entire data cubes, that is, groupings on all subsets of the dimension attributes. Precomputation allows OLAP queries to be answered within a few seconds, even on datasets that may contain millions of tuples adding up to gigabytes of data. However, there are 2^n groupings with n dimension attributes; hierarchies on attributes increase the number further. As a result, the entire data cube is often larger than the original relation that formed the data cube and in many cases it is not feasible to store the entire data cube.

Instead of precomputing and storing all possible groupings, it makes sense to precompute and store some of the groupings, and to compute others on demand. Instead of computing queries from the original relation, which may take a very long time, we can compute them from other precomputed queries. For instance, suppose that a query requires grouping by *(item_name, color)*, and this has not been precomputed. The query result can be computed from summaries by *(item_name, color, clothes_size)*, if that has been precomputed. See the bibliographical notes for references on how to select a good set of groupings for precomputation, given limits on the storage available for precomputed results.

of the dimension attributes. Using only the basic **group by** construct, we would have to write many separate SQL queries and combine them using a union operation. SQL supports special syntax to allow multiple group by operations to be specified concisely.

As we saw in Section 5.5.4, SQL supports generalizations of the **group by** construct to perform the **cube** and **rollup** operations. The **cube** and **rollup** constructs in the **group by** clause allow multiple **group by** queries to be run in a single query with the result returned as a single relation in a style similar to that of the relation of Figure 11.8.

Consider again our retail shop example and the relation:

sales (*item_name*, *color*, *clothes_size*, *quantity*)

If we want to generate the entire data cube using individual group by queries, we have to write a separate query for each of the following eight sets of group by attributes:

{ (*item_name*, *color*, *clothes_size*), (*item_name*, *color*), (*item_name*, *clothes_size*),
(*color*, *clothes_size*), (*item_name*), (*color*), (*clothes_size*), () }

where () denotes an empty **group by** list.

As we saw in Section 5.5.4, the **cube** construct allows us to accomplish this in one query:

```
select item_name, color, clothes_size, sum(quantity)
from sales
group by cube(item_name, color, clothes_size);
```

The preceding query produces a relation whose schema is:

(*item_name*, *color*, *clothes_size*, sum(*quantity*))

So that the result of this query is indeed a relation, tuples in the result contain *null* as the value of those attributes not present in a particular grouping. For example, tuples produced by grouping on *clothes_size* have a schema (*clothes_size*, sum(*quantity*)). They are converted to tuples on (*item_name*, *color*, *clothes_size*, sum(*quantity*)) by inserting *null* for *item_name* and *color*.

Data cube relations are often very large. The cube query above, with 3 possible colors, 4 possible item names, and 3 sizes, has 80 tuples. The relation of Figure 11.8 is generated by doing a **group by cube** on *item_name* and *color*, with an extra column specified in the **select** clause showing **all** for *clothes_size*.

To generate that relation in SQL, we substitute **all** for *null* using the **grouping** function, as we saw earlier in Section 5.5.4. The **grouping** function distinguishes those nulls generated by OLAP operations from “normal” nulls actually stored in the database or arising from an outer join. Recall that the **grouping** function returns 1 if its argument

is a null value generated by a **cube** or **rollup** and 0 otherwise. We may then operate on the result of a call to **grouping** using **case** expressions to replace OLAP-generated nulls with **all**. Then the relation in Figure 11.8, with occurrences of *null* replaced by **all**, can be computed by the query:

```
select case when grouping(item_name) = 1 then 'all'
         else item_name end as item_name,
       case when grouping(color) = 1 then 'all'
         else color end as color,
       'all' as clothes_size, sum(quantity) as quantity
from sales
group by cube(item_name, color);
```

The **rollup** construct is the same as the **cube** construct except that **rollup** generates fewer **group by** queries. We saw that **group by cube** (*item_name, color, clothes_size*) generated all eight ways of forming a **group by** query using some (or all or none) of the attributes. In:

```
select item_name, color, clothes_size, sum(quantity)
from sales
group by rollup(item_name, color, clothes_size);
```

the clause **group by rollup**(*item_name, color, clothes_size*) generates only four groupings:

$$\{ (item_name, color, clothes_size), (item_name, color), (item_name), () \}$$

Notice that the order of the attributes in the **rollup** makes a difference; the last attribute (*clothes_size*, in our example) appears in only one grouping, the penultimate (second last) attribute in two groupings, and so on, with the first attribute appearing in all groups but one (the empty grouping).

Why might we want the specific groupings that are used in **rollup**? These groups are of frequent practical interest for hierarchies (as in Figure 11.6, for example). For the location hierarchy (*Region, Country, State, City*), we may want to group by *Region* to get sales by region. Then we may want to “drill down” to the level of countries within each region, which means we would group by *Region, Country*. Drilling down further, we may wish to group by *Region, Country, State* and then by *Region, Country, State, City*. The **rollup** construct allows us to specify this sequence of drilling down for further detail.

As we saw earlier in Section 5.5.4, multiple **rollups** and **cubes** can be used in a single **group by** clause. For instance, the following query:

```
select item_name, color, clothes_size, sum(quantity)
from sales
group by rollup(item_name), rollup(color, clothes_size);
```

generates the groupings:

```
{ (item_name, color, clothes_size), (item_name, color), (item_name),
  (color, clothes_size), (color), () }
```

To understand why, observe that **rollup**(*item_name*) generates two groupings, {(*item_name*), ()}, and **rollup**(*color*, *clothes_size*) generates three groupings, {(*color*, *clothes_size*), (*color*), ()}. The Cartesian product of the two gives us the six groupings shown.

Neither the **rollup** nor the **cube** clause gives complete control on the groupings that are generated. For instance, we cannot use them to specify that we want only groupings {(*color*, *clothes_size*), (*clothes_size*, *item_name*)}. Such restricted groupings can be generated by using the **grouping sets** construct, in which one can specify the specific list of groupings to be used. To obtain only groupings {(*color*, *clothes_size*), (*clothes_size*, *item_name*)}, we would write:

```
select item_name, color, clothes_size, sum(quantity)
from sales
group by grouping sets ((color, clothes_size), (clothes_size, item_name));
```

Specialized languages have been developed for querying multidimensional OLAP schemas, which allow some common tasks to be expressed more easily than with SQL. These include the MDX and DAX query languages developed by Microsoft.

11.3.4 Reporting and Visualization Tools

Report generators are tools to generate human-readable summary reports from a database. They integrate querying the database with the creation of formatted text and summary charts (such as bar or pie charts). For example, a report may show the total sales in each of the past 2 months for each sales region.

The application developer can specify report formats by using the formatting facilities of the report generator. Variables can be used to store parameters such as the month and the year and to define fields in the report. Tables, graphs, bar charts, or other graphics can be defined via queries on the database. The query definitions can make use of the parameter values stored in the variables.

Once we have defined a report structure on a report-generator facility, we can store it and can execute it at any time to generate a report. Report-generator systems provide a variety of facilities for structuring tabular output, such as defining table and column headers, displaying subtotals for each group in a table, automatically splitting long tables into multiple pages, and displaying subtotals at the end of each page.

Figure 11.10 is an example of a formatted report. The data in the report are generated by aggregation on information about orders.

Report-generation tools are available from a variety of vendors, such as SAP Crystal Reports and Microsoft (SQL Server Reporting Services). Several application suites, such as Microsoft Office, provide a way of embedding formatted query results from

Acme Supply Company, Inc.
Quarterly Sales Report

Period: Jan. 1 to March 31, 2009

Region	Category	Sales	Subtotal
North	Computer Hardware	1,000,000	1,500,000
	Computer Software	500,000	
	All categories		
South	Computer Hardware	200,000	600,000
	Computer Software	400,000	
	All categories		
Total Sales			2,100,000

Figure 11.10 A formatted report.

a database directly into a document. Chart-generation facilities provided by Crystal Reports or by spreadsheets such as Excel can be used to access data from databases and to generate tabular depictions of data or graphical depictions using charts or graphs. Such charts can be embedded within text documents. The charts are created initially from data generated by executing queries against the database; the queries can be re-executed and the charts regenerated when required, to generate a current version of the overall report.

Techniques for **data visualization**, that is, graphical representation of data, that go beyond the basic chart types, are very important for data analysis. Data-visualization systems help users to examine large volumes of data and to detect patterns visually. Visual displays of data—such as maps, charts, and other graphical representations—allow data to be presented compactly to users. A single graphical screen can encode as much information as a far larger number of text screens.

For example, if the user wants to find out whether the occurrence of a disease is correlated to the locations of the patients, the locations of patients can be encoded in a special color—say, red—on a map. The user can then quickly discover locations where problems are occurring. The user may then form hypotheses about why problems are occurring in those locations and may verify the hypotheses quantitatively against the database.

As another example, information about values can be encoded as a color and can be displayed with as little as one pixel of screen area. To detect associations between pairs of items, we can use a two-dimensional pixel matrix, with each row and each column representing an item. The percentage of transactions that buy both items can be encoded by the color intensity of the pixel. Items with high association will show up as bright pixels in the screen—easy to detect against the darker background.

13.6 Column-Oriented Storage

Databases traditionally store all attributes of a tuple together in a record, and tuples are stored in a file as we have just seen. Such a storage layout is referred to as a *row-oriented storage*.

In contrast, in **column-oriented storage**, also called a **columnar storage**, each attribute of a relation is stored separately, with values of the attribute from successive tuples stored at successive positions in the file. Figure 13.14 shows how the *instructor* relation would be stored in column-oriented storage, with each attribute stored separately.

In the simplest form of column-oriented storage, each attribute is stored in a separate file. Further, each file is *compressed*, to reduce its size. (We discuss more complex schemes that store columns consecutively in a single file later in this section.)

If a query needs to access the entire contents of the i th row of a table, the values at the i th position in each of the columns are retrieved and used to reconstruct the row. Column-oriented storage thus has the drawback that fetching multiple attributes of a single tuple requires multiple I/O operations. Thus, it is not suitable for queries that fetch multiple attributes from a few rows of a relation.

However, column-oriented storage is well suited for data analysis queries, which process many rows of a relation, but often only access some of the attributes. The reasons are as follows:

- **Reduced I/O.** When a query needs to access only a few attributes of a relation with a large number of attributes, the remaining attributes need not be fetched from disk into memory. In contrast, in row-oriented storage, irrelevant attributes are fetched into memory from disk. The reduction in I/O can lead to significant reduction in query execution cost.

10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Figure 13.14 Columnar representation of the *instructor* relation.

- **Improved CPU cache performance.** When the query processor fetches the contents of a particular attribute, with modern CPU architectures multiple consecutive bytes, called a cache line, are fetched from memory to CPU cache. If these bytes are accessed later, access is much faster if they are in cache than if they have to be fetched from main memory. However, if these adjacent bytes contain values for attributes that are not needed by the query, fetching them into cache wastes memory bandwidth and uses up cache space that could have been used for other data. Column-oriented storage does not suffer from this problem, since adjacent bytes are from the same column, and data analysis queries usually access all these values consecutively.
- **Improved compression.** Storing values of the same type together significantly increases the effectiveness of compression, when compared to compressing data stored in row format; in the latter case, adjacent attributes are of different types, reducing the efficiency of compression. Compression significantly reduces the time taken to retrieve data from disk, which is often the highest-cost component for many queries. If the compressed files are stored in memory, the in-memory storage space is also reduced correspondingly, which is particularly important since main memory is significantly more expensive than disk storage.
- **Vector processing.** Many modern CPU architectures support **vector processing**, which allows a CPU operation to be applied in parallel on a number of elements of an array. Storing data columnwise allows vector processing of operations such as comparing an attribute with a constant, which is important for applying selection conditions on a relation. Vector processing can also be used to compute an aggregate of multiple values in parallel, instead of aggregating the values one at a time.

As a result of these benefits, column-oriented storage is increasingly used in data-warehousing applications, where queries are primarily data analysis queries. It should be noted that indexing and query processing techniques need to be carefully designed to get the performance benefits of column-oriented storage. We outline indexing and query processing techniques based on bitmap representations, which are well suited to column-oriented storage, in Section 14.9; further details are provided in Section 24.3.

Databases that use column-oriented storage are referred to as **column stores**, while databases that use row-oriented storage are referred to as **row stores**.

It should be noted that column-oriented storage does have several drawbacks, which make them unsuitable for transaction processing.

- **Cost of tuple reconstruction.** As we saw earlier, reconstructing a tuple from the individual columns can be expensive, negating the benefits of columnar representation if many columns need to be reconstructed. While tuple reconstruction is common in transaction-processing applications, data analysis applications usually

output only a few columns out of many that are stored in “fact tables” in data warehouses.

- **Cost of tuple deletion and update.** Deleting or updating a single tuple in a compressed representation would require rewriting the entire sequence of tuples that are compressed as one unit. Since updates and deletes are common in transaction-processing applications, column-oriented storage would result in a high cost for these operations if a large number of tuples were compressed as one unit.

In contrast, data-warehousing systems typically do not support updates to tuples, and instead support only insert of new tuples and bulk deletes of a large number of old tuples at a time. Inserts are done at the end of the relation representation, that is, new tuples are appended to the relation. Since small deletes and updates do not occur in a data warehouse, large sequences of attribute values can be stored and compressed together as one unit, allowing for better compression than with small sequences.

- **Cost of decompression.** Fetching data from a compressed representation requires *decompression*, which in the simplest compressed representations requires reading all the data from the beginning of a file. Transaction processing queries usually only need to fetch a few records; sequential access is expensive in such a scenario, since many irrelevant records may have to be decompressed to access a few relevant records.

Since data analysis queries tend to access many consecutive records, the time spent on decompression is typically not wasted. However, even data analysis queries do not need to access records that fail selection conditions, and attributes of such records should be skipped to reduce disk I/O.

To allow skipping of attribute values from such records, compressed representations for column stores allow decompression to start at any of a number of points in the file, skipping earlier parts of the file. This could be done by starting compression afresh after every 10,000 values (for example). By keeping track of where in the file the data start for each group of 10,000 values, it is possible to access the i th value by going to the start of the group $\lfloor i/10000 \rfloor$ and starting decompression from there.

ORC and Parquet are columnar file representations used in many big-data processing applications. In ORC, a row-oriented representation is converted to column-oriented representation as follows: A sequence of tuples occupying several hundred megabytes is broken up into a columnar representation called a **stripe**. An ORC file contains several such stripes, with each stripe occupying around 250 megabytes.

Figure 13.15 illustrates some details of the ORC file format. Each stripe has index data followed by row data. The row data area stores a compressed representation of the sequence of value for the first column, followed by the compressed representation of the second column, and so on. The index data region of a stripe stores for each attribute the starting point within the stripe for each group of (say) 10,000 values of

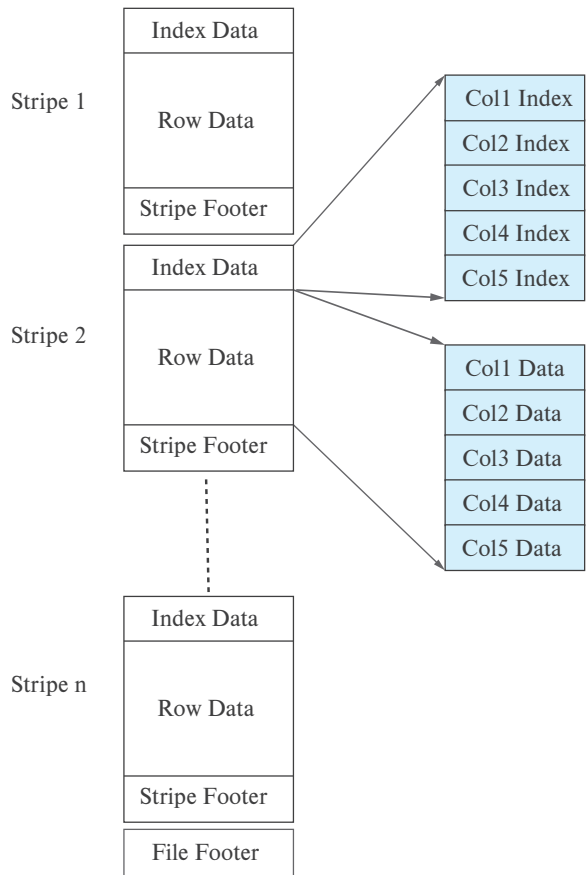


Figure 13.15 Columnar data representation in the ORC file format.

that attribute.⁷ The index is useful for quick access to a desired tuple or sequence of tuples; the index also allows queries containing selections to skip groups of tuples if the query determines that no tuple in those groups satisfies the selections. ORC files store several other pieces of information in the stripe footer and file footer, which we skip here.

Some column-store systems allow groups of columns that are often accessed together to be stored together, instead of breaking up each column into a different file. Such systems thus allow a spectrum of choices that range from pure column-oriented storage, where every column is stored separately, to pure row-oriented storage, where all columns are stored together. The choice of which attributes to store together depends on the query workload.

⁷ORC files have some other information that we ignore here.

up to date. When hash indices are used on r and s , the resultant algorithm is called the **double-pipelined hash-join** technique.

The double-pipelined join algorithm in Figure 15.13 assumes that both inputs fit in memory. In case the two inputs are larger than memory, it is still possible to use the double-pipelined join technique as usual until available memory is full. When available memory becomes full, r and s tuples that have arrived up to that point can be treated as being in partition r_0 and s_0 , respectively. Tuples for r and s that arrive subsequently are assigned to partitions r_1 and s_1 , respectively, which are written to disk, and are not added to the in-memory index. However, tuples assigned to r_1 and s_1 are used to probe s_0 and r_0 , respectively, before they are written to disk. Thus, the join of r_1 with s_0 , and s_1 with r_0 , is also carried out in a pipelined fashion. After r and s have been fully processed, the join of r_1 tuples with s_1 tuples must be carried out to complete the join; any of the join techniques we have seen earlier can be used to join r_1 with s_1 .

15.7.3 Pipelines for Continuous-Stream Data

Pipelining is also applicable in situations where data are entered into the database in a continuous manner, as is the case, for example, for inputs from sensors that are continuously monitoring environmental data. Such data are called *data streams*, as we saw earlier in Section 10.5. Queries may be written over stream data in order to respond to data as they arrive. Such queries are called *continuous queries*.

The operations in a continuous query should be implemented using pipelined algorithms, so that results from the pipeline can be output without blocking. Producer-driven pipelines (which we discussed earlier in Section 15.7.2.1) are the best suited for continuous query evaluation.

Many such queries perform aggregation with windowing; tumbling windows which divide time into fixed size intervals, such as 1 minute, or 1 hour, are commonly used. Grouping and aggregation is performed separately on each window, as tuples are received; assuming memory size is large enough, an in-memory hash index is used to perform aggregation.

The result of aggregation on a window can be output once the system knows that no further tuples in that window will be received in future. If tuples are guaranteed to arrive sorted by timestamp, the arrival of a tuple of a following window indicates no more tuples will be received for an earlier window. If tuples may arrive out of order, streams must carry punctuations that indicate that all future tuples will have a timestamp greater than some specified value. The arrival of a punctuation allows the output of aggregates of windows whose end-timestamp is less than or equal to the timestamp specified by the punctuation.

15.8 Query Processing in Memory

The query processing algorithms that we have described so far focus on minimizing I/O cost. In this section, we discuss extensions to the query processing techniques that

help minimize memory access costs by using cache-conscious query processing algorithms and query compilation. We then discuss query processing with column-oriented storage. The algorithms we describe in this section give significant benefits for memory resident data; they are also very useful with disk-resident data, since they can speed up processing once data has been brought into the in-memory buffer.

15.8.1 Cache-Conscious Algorithms

When data is resident in memory, access is much faster than if data were resident on magnetic disks, or even SSDs. However, it must be kept in mind that data already in CPU cache can be accessed as much as 100 times faster than data in memory. Modern CPUs have several levels of cache. Commonly used CPUs today have an L1 cache of size around 64 kilobytes, with a latency of about 1 nanosecond, an L2 cache of size around 256 kilobytes, with a latency of around 5 nanoseconds, and an L3 cache of having a size of around 10 megabytes, with a latency of 10 to 15 nanoseconds. In contrast, reading data in memory results in a latency of around 50 to 100 nanoseconds. For simplicity in the rest of this section we ignore the difference between the L1, L2 and L3 cache levels, and assume that there is only a single cache level.

As we saw in Section 14.4.7, the speed difference between cache memory and main memory, and the fact that data are transferred between main memory and cache in units of a *cache-line* (typically about 64 bytes), results in a situation where the relationship between cache and main memory is not dissimilar to the relationship between main memory and disk (although with smaller speed differences). But there is a difference: while the contents of the main memory buffers disk-based data are controlled by the database system, CPU cache is controlled by the algorithms built into the computer hardware. Thus, the database system cannot directly control what is kept in cache.

However, query processing algorithms can be designed in a way that makes the best use of cache, to optimize performance. Here are some ways this can be done:

- To sort a relation that is in-memory, we use the external merge-sort algorithm, with the run size chosen such that the run fits into the cache; assuming we focus on the L3 cache, each run should be a few megabytes in size. We then use an in-memory sorting algorithm on each run; since the run fits in cache, cache misses are likely to be minimal when the run is sorted. The sorted runs (all of which are in memory) are then merged. Merging is cache efficient, since access to the runs is sequential: when a particular word is accessed from memory, the cache line that is fetched will contain the words that would be accessed next from that run.

To sort a relation larger than memory, we can use external sort-merge with much larger run sizes, but use the in-memory merge-sort technique we just described to perform the in-memory sort of the large runs.

- Hash-join requires probing of an index on the build relation. If the build relation fits in memory, an index could be built on the whole relation; however, cache hits during probe can be maximized by partitioning the relations into smaller pieces

such that each partition of the build-relation along with the index fits in the cache. Each partition is processed separately, with a build and a probe phase; since the build partition and its index fit in cache, cache misses are minimized during the build as well as the probe phase.

For relations larger than memory, the first stage of hash-join should partition the two relations such that for each partition, the partitions of the two relations together fit in memory. The technique just described can then be used to perform the hash join on each of these partitions, after fetching the contents into memory.

- Attributes in a tuple can be arranged such that attributes that tend to be accessed together are laid out consecutively. For example, if a relation is often used for aggregation, those attributes used as group by attributes, and those that are aggregated upon, can be stored consecutively. As a result, if there is a cache miss on one attribute, the cache line that is fetched would contain attributes that are likely to be used immediately.

Cache-aware algorithms are of increasing importance in modern database systems, since memory sizes are often large enough that much of the data is memory-resident.

In cases where the requisite data item is not in cache, there is a processing **stall** while the data item is retrieved from memory and loaded into cache. In order to continue to make use of the core that made the request resulting in the stall, the operating system maintains multiple threads of execution on which a core may work. Parallel query processing algorithms, which we study in Chapter 22 can use multiple threads running on a single CPU core; if one thread is stalled, another can start execution so the CPU core is utilized better.

15.8.2 Query Compilation

With data resident in memory, CPU cost becomes the bottleneck, and minimizing CPU cost can give significant benefits. Traditional databases query processors act as interpreters that execute a query plan. However, there is a significant overhead due to interpretation: for example, to access an attribute of a record, the query execution engine may repeatedly look up the relation meta-data to find the offset of the attribute within the record, since the same code must work for all relations. There is also significant overhead due to function calls that are performed for each record processed by an operation.

To avoid overhead due to interpretation, modern main-memory databases compile query plans into machine code or intermediate level byte-code. For example, the compiler can compute the offset of an attribute at compile time, and generate code where the offset is a constant. The compiler can also combine the code for multiple functions in a way that minimizes function calls. With these, and other related optimizations, compiled code has been found to execute faster, by up to a factor of 10, than interpreted code.

Concurrency control is another issue in parallel and distributed databases. Since a transaction may access data items at several nodes, transaction managers at several nodes may need to coordinate to implement concurrency control. If locking is used, locking can be performed locally at the nodes containing accessed data items, but there is also a possibility of deadlock involving transactions originating at multiple nodes. Therefore deadlock detection needs to be carried out across multiple nodes. Failures are more common in distributed systems since not only may computers fail, but communication links may also fail. Replication of data items, which is the key to the continued functioning of distributed databases when failures occur, further complicates concurrency control. We describe concurrency-control techniques for distributed databases in Section 23.3 (which describes techniques based on locking) and Section 23.3.4 (which describes techniques based on timestamps).

The standard transaction models, based on multiple actions carried out by a single program unit, are often inappropriate for carrying out tasks that cross the boundaries of databases that cannot or will not cooperate to implement protocols such as 2PC. Alternative approaches, based on *persistent messaging* for communication, are generally used for such tasks; persistent messaging is discussed in Section 23.2.3.

When the tasks to be carried out are complex, involving multiple databases and/or multiple interactions with humans, coordination of the tasks and ensuring transaction properties for the tasks become more complicated. *Workflow management systems* are systems designed to help with carrying out such tasks.

20.7 Cloud-Based Services

Traditionally, enterprises purchased and ran servers that execute the database as well as the applications. There is a high cost to maintaining servers, including setting up server room infrastructure dealing with all kinds of failures such as air conditioning and power failures, not to mention failures of CPUs, disks, and other components of the servers. Further, if there is a sudden increase in demand, it is very difficult to add infrastructure to service the demand, and if demand falls, the infrastructure may lie idle.

In contrast, in the **cloud computing** model, applications of an enterprise are executed on an infrastructure that is managed by another company, typically at a data center that hosts a large number of machines used by many different enterprises/users. The service provider may provide not just hardware, but also support platforms such as databases, and application software.

A variety of vendors offer cloud services; these include major vendors such as Amazon, Microsoft, IBM, and Google, and a number of smaller vendors. One of the pioneers of cloud services, Amazon, originally built a large computing infrastructure purely for its internal use; then, seeing a business opportunity, it offered computing infrastructure as a service to other users. Cloud services became very popular within just a few years.

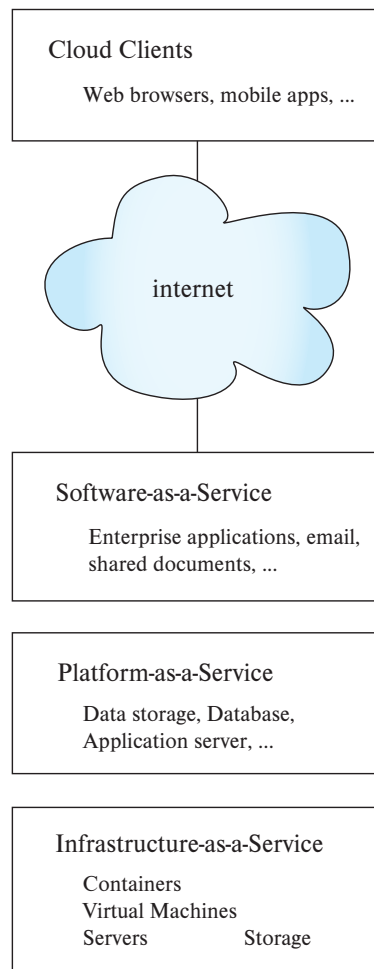


Figure 20.10 Cloud service models.

20.7.1 Cloud Service Models

There are several ways in which cloud computing can be utilized, which are summarized in Figure 20.10. These include infrastructure-as-a-service, platform-as-a-service, and software-as-a-service models.

- In the **infrastructure-as-a-service** model, an enterprise rents computing facilities; for example, an enterprise may rent one or more physical machines, along with disk storage space.

More frequently, cloud computing providers provide an abstraction of a **virtual machine (VM)**, which appears to the user to be a real machine. These machines are not “real” machines, but rather are simulated by software that allows a single

real computer to simulate several independent computers. *Containers* are a lower cost alternative to VMs and are described later in this section. Multiple VMs can run on a single server machine, and multiple containers can run on a single VM or server.

By running a very large data center with many machines, cloud-service providers can exploit economies of scale and deliver computing power at much lower cost than an enterprise can do using its own infrastructure.

Another major advantage of cloud computing is that the cloud-service provider usually has a large number of machines, with spare capacity, and thus an enterprise can rent more (virtual) machines as needed to meet demand and release them at times of light load. The ability to expand or contract capacity at short notice is often referred to as **elasticity**.

The above benefits of on-demand elastic provisioning of server systems have led to the widespread adoption of infrastructure-as-service platforms, especially by companies that anticipate rapid growth in their computing usage. However, due to the potential security risks of storing data outside the enterprise, the use of cloud computing is still limited in high-security enterprise needs, such as banking.

In the infrastructure-as-service model, the client enterprise runs its own software, including database systems, on virtual machines provided by the cloud-service provider; the client has to install the database system and deal with maintenance issues such as backup and restore.

- In the **platform-as-a-service** model, the service provider not only provides computing infrastructure, but it also deploys and manages platforms, such as data storage, databases, and application servers, that are used by application software. The client has to install and maintain application software, such as *enterprise resource planning (ERP)* systems, which run on such platform-provided services as application servers, database services, or data storage services.
 - **Cloud-based data storage** platforms provide a service that applications can use to store and retrieve data. The service provider takes care of provisioning sufficient amount of storage and computing power to support the load on the data storage platform. Such storage systems could support files, which are typically large, ranging in size from a few megabytes to thousands of megabytes, supporting millions of such files. Or such storage systems could support data items, which are typically small, ranging from hundreds of bytes to a few megabytes, but supporting billions of such data items. Such distributed file systems and data storage systems are discussed in Section 21.6 and Section 21.7. Database applications using cloud-based storage may run on the same cloud (i.e., the same set of machines), or on another cloud.

One of the main attractions of cloud-based storage is that it can be used by paying a fee without worrying about purchasing, maintaining, and managing the computer systems on which such a service runs. Further, if there is an

increase in demand, the number of servers on which the service runs can be increased by paying a larger fee, without having to actually purchase and deploy more servers. The service provider would of course have to deploy extra servers, but they benefit from economies of scale; the cost of deployment, and especially the time to deployment, are greatly reduced compared to what they would be if the end-users did it on their own.

The fees for cloud-based data storage are typically based on the amount of data stored, and amount of data input to, and the amount of data output from, the data storage system.

- **Database-as-a-service** platforms provide a database that can be accessed and queried by clients. Unlike storage services, database-as-a-service platforms provide database functionality such as querying using SQL or other query languages, which data storage systems do not provide. Early offerings of database-as-a-service only supported databases that run on a single node, although the node itself can have a substantial number of processors, memory, and storage. More recently, parallel database systems are being offered as a service on the cloud.
- In the **software-as-a-service** model, the service provider provides the application software as a service. The client does not need to deal with issues such as software installation or upgrades; these tasks are left to the service provider. The client can directly use interfaces provided by the software-as-a-service provider, such as web interfaces, or mobile app interfaces that provide a front end, with the application software acting as the back end.

The concept of virtual machines was developed in the 1960s to allow an expensive mainframe computer to be shared concurrently by users running different operating systems. Although computers are now much cheaper, there is still a cost associated with supplying electrical power to the computers and maintaining them; virtual machines allow this cost to be shared by multiple concurrent users. Virtual machines also ease the task of moving services to new machines: a virtual machine can be shut down on one physical server and restarted on another physical server with very little delay or downtime. This feature is particularly important for quick recovery in the event of hardware failure or upgrade.

Although multiple virtual machines can run on a single real machine, each VM has a high overhead, since it runs an entire operating system internally. When a single organization wishes to run a number of services, if it creates a separate VM for each service, the overhead can be very high. If multiple applications are run on one machine (or VM), two problems often arise: (1) applications conflict on network ports by each trying to listen to the same network port, and (2) applications require different versions of shared libraries, causing conflicts.

Containers solve both these problems; applications run in a container, which has its own IP address and its own set of shared libraries. Each application can consist of

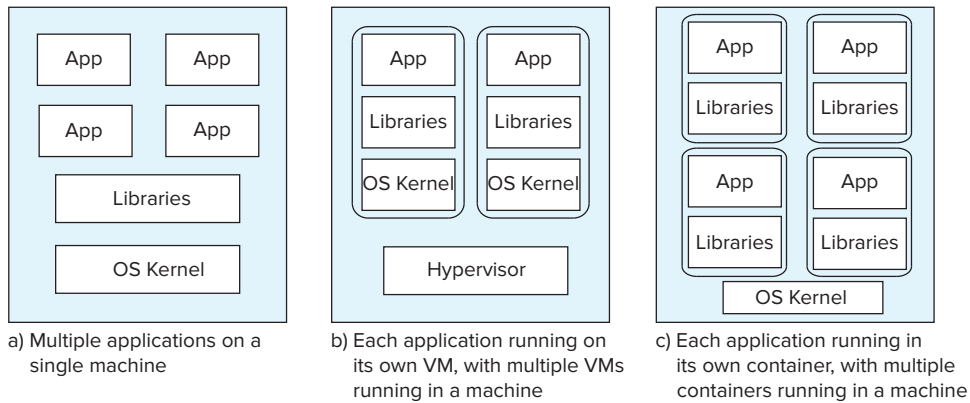


Figure 20.11 Application deployment alternatives.

multiple processes, all running within the same container. The cost of using containers to run applications is much less than the alternative of running each application in its own VM, since many containers can share the same operating system kernel. Each container appears to have its own file system, but the files are all stored in a common underlying file system across all containers. Processes within a container can interact with each other through the file system as well as interprocess communication, but they can interact with processes from other containers only via network connections.

Figure 20.11 depicts the different deployment alternatives for a set of applications. Figure 20.11a shows the alternative of multiple applications running in a single machine, sharing libraries and operating-system kernel. Figure 20.11b shows the alternative of running each application in its own VM, with multiple VMs running on a single machine. The different VMs running on a single real machine are managed by a software layer called the *hypervisor*. Figure 20.11c shows the alternative of using containers, with each container having its own libraries, and multiple containers running on a single machine. Since containers have lower overheads, a single machine can support more containers than VMs.

Containers provide low-cost support for elasticity, since more containers can be deployed very quickly on existing virtual machines, instead of starting up fresh virtual machines.

Many applications today are built as a collection of multiple **services**, each of which runs as a separate process, offering a network API; that is, the functions provided by the service are invoked by creating a network connection to the process and sending a service request over the network connection. Such an application architecture, which builds an application as a collection of small services, is called a **microservices architecture**. Containers fit the microservices architecture very well, since they provide a very low overhead mechanism to execute processes supporting each service.

partitions, each of which is called a reduce task. A scheduler assigns reduce tasks to workers.

This step is identical to the repartitioning done for parallelizing the relational aggregation operation, with records partitioned into a number of virtual nodes based on their group-by key.

To process the records in a particular **reduce** task, the records are sorted (or grouped) by the reduce key, so that all records with the same reduce-key value are brought together, and then the `reduce()` is executed on each group of reduce-key values.

The reduce tasks are executed in parallel by the workers. When a worker completes a reduce task, another task is assigned to it, until all reduce tasks have been completed. A reduce task may have multiple different reduce key values, but a particular call to the `reduce()` function is for a single reduce key; thus, the `reduce()` function is called for each key in the reduce task.

Tasks correspond to virtual nodes in the virtual-node partitioning scheme. There are far more tasks than there are nodes, and tasks are divided among the nodes. As discussed in Section 22.3.3, virtual-node partitioning reduces skew. Also note that as discussed in Section 22.4.1, skew can be reduced by partial aggregation, which corresponds to *combiners* in the MapReduce framework.

Further, MapReduce implementations typically also carry out dynamic detection and handling of skew, as discussed in Section 22.4.1.

Most MapReduce implementations include techniques to ensure that processing can be continued even if some nodes fail during query execution. Details are discussed further in Section 22.5.4.

22.5 Parallel Evaluation of Query Plans

As discussed in Section 22.1, there are two types of parallelism: intraoperation and interoperation. Until now in this chapter, we have focused on intraoperation parallelism. In this section, we consider execution plans for queries containing multiple operations.

We first consider how to exploit interoperator parallelism. We then consider a model of parallel query execution which breaks parallel query processing into two types of steps: partitioning of data using the *exchange operator*, and execution of operations on purely local data, without any data exchange. This model is surprisingly powerful and is widely used in parallel database implementations.

22.5.1 Interoperation Parallelism

There are two forms of interoperation parallelism: pipelined parallelism and independent parallelism. We first describe these forms of parallelism, assuming each operator runs on a single node without intraoperation parallelism.

We then describe a model for parallel execution based on the exchange operator, in Section 22.5.2. Finally, in Section 22.5.3, we describe how a complete plan can be executed, combining all the forms of parallelism.

22.5.1.1 Pipelined Parallelism

Recall from Section 15.7.2 that in pipelining, the output tuples of one operation, A , are consumed by a second operation, B , even before the first operation has produced the entire set of tuples in its output. The major advantage of pipelined execution in a sequential evaluation is that we can carry out a sequence of such operations without writing any of the intermediate results to disk.

Parallel systems use pipelining primarily for the same reason that sequential systems do. However, pipelines are a source of parallelism as well, since it is possible to run operations A and B simultaneously on different nodes (or different cores of a single node), so that B consumes tuples in parallel with A producing them. This form of parallelism is called **pipelined parallelism**.

Pipelined parallelism is useful with a small number of nodes, but it does not scale up well. First, pipeline chains generally do not attain sufficient length to provide a high degree of parallelism. Second, it is not possible to pipeline relational operators that do not produce output until all inputs have been accessed, such as the set-difference operation. Third, only marginal speedup is obtained for the frequent cases in which one operator's execution cost is much higher than are those of the others.

All things considered, when the degree of parallelism is high, pipelining is a less important source of parallelism than partitioning. The real reason for using pipelining with parallel query processing is the same reason that pipelining is used with sequential query processing: namely, that pipelined executions can avoid writing intermediate results to disk.

Pipelining in centralized databases was discussed in Section 15.7.2; as mentioned there, pipelining can be done using a demand-driven, or pull, model of computation, or using a producer-driven, or push, model of computation. The pull model is widely used in centralized database systems.

However, the push model is greatly preferred in parallel database systems, since, unlike the pull model, the push model allows both the producer and consumer to execute in parallel.

Unlike the pull model, the push model requires a buffer that can hold multiple tuples, between the producer and consumer; without such a buffer, the producer would stall as soon as it generates one tuple. Figure 22.5 shows a producer and consumer with a buffer in-between. If the producer and consumer are on the same node, as shown in Figure 22.5a, the buffer can be in shared memory. However, if the producer and consumer are in different nodes, as shown in Figure 22.5b, there will be two buffers: one at the producer node to collect tuples as they are produced, and another at the consumer node to collect them as they are sent across the network.

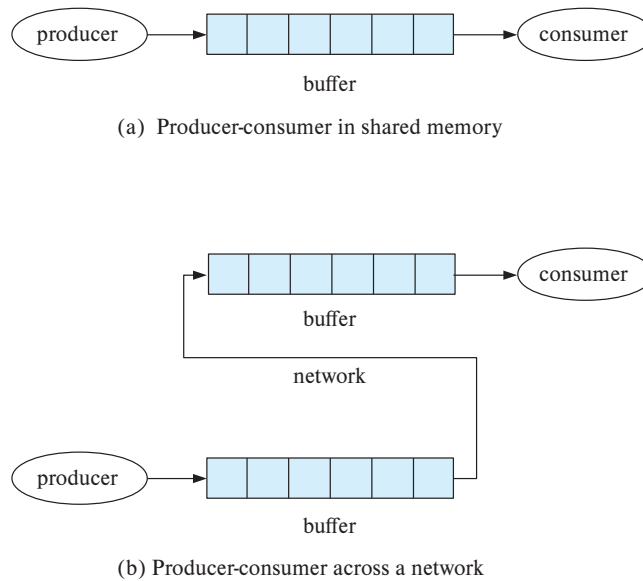


Figure 22.5 Producer and consumer with buffer.

When sending tuples across a network, it makes sense to collect multiple tuples and send them as a single *batch*, rather than send tuples one at a time, since there is usually a very significant overhead per message. Batching greatly reduces this overhead.

If the producer and consumer are on the same node and can communicate via a shared memory buffer, mutual exclusion needs to be ensured when inserting tuples into, or fetching tuples from, the buffer. Mutual exclusion protocols have some overhead, which can be reduced by inserting/retrieving a batch of tuples at a time, instead of one tuple at a time.

Note that with the pull model, either the producer or the consumer, but not both, can be executing at a given time; while this avoids the contention on the shared buffer that arises with the use of the push model, it also prevents the producer and consumer from running concurrently.

22.5.1.2 Independent Parallelism

Operations in a query expression that do not depend on one another can be executed in parallel. This form of parallelism is called **independent parallelism**.

Consider the join $r_1 \bowtie r_2 \bowtie r_3 \bowtie r_4$. One possible plan is to compute intermediate result $t_1 \leftarrow r_1 \bowtie r_2$ in parallel with intermediate result $t_2 \leftarrow r_3 \bowtie r_4$. Neither of these computations depends on each other, and hence they can be parallelized by independent parallelism. In other words, the execution of these two joins can be scheduled in parallel.

When these two computations complete, we can compute:

$$t_1 \bowtie t_2$$

Note that computation of the above join depends on the results of the first two joins, hence it cannot be done using independent parallelism.

Like pipelined parallelism, independent parallelism does not provide a high degree of parallelism and is less useful in a highly parallel system, although it is useful with a lower degree of parallelism.

22.5.2 The Exchange Operator Model

The Volcano parallel database popularized a model of parallelization called the *exchange-operator* model. The exchange operation repartitions data in a specified way; data interchange between nodes is done only by the exchange operator. All other operations work on local data, just as they would in a centralized database system; the data may be available locally either because it is already present, or because of the execution of a preceding exchange operator.

The **exchange operator** has two components: a scheme for *partitioning* outgoing data, applied at each source node, and a scheme for *merging* incoming data, applied at each destination node. The operator is shown pictorially in Figure 22.6, with the partitioning scheme denoted as “Partition,” and the merging scheme denoted as “Merge.”

The exchange operator can *partition* data in one of several ways:

1. By hash partitioning on a specified set of attributes.
2. By range partitioning on a specified set of attributes.
3. By replicating the input data at all nodes, referred to as **broadcasting**.
4. By sending all data to a single node.

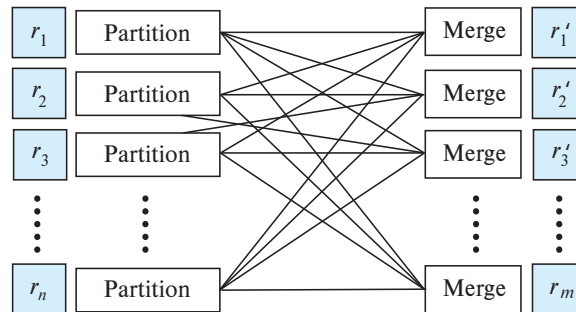


Figure 22.6 The exchange operator used for repartitioning.

Broadcasting data to all nodes is required for operations such as the asymmetric fragment-and-replicate join. Sending all data to a single node is usually done as a final step of parallel query processing, to get partitioned results together at a single site.

Note also that the input to the exchange operator can be at a single site (referred to as **unpartitioned**), or already partitioned across multiple sites. Repartitioning of already partitioned data results in each destination node receiving data from multiple source nodes, as shown in Figure 22.6.

Each destination node *merges* the data items received from the source nodes. This merge step can store data in the order received (which may be nondeterministic, since it depends on the speeds of the machines and unpredictable network delays); such merging is called **random merge**.

On the other hand, if the input data from each source is sorted, the merge step can exploit the sort order by performing an **ordered merge**. Suppose, for example, nodes $N_1, \dots, N_m$ first sort a relation locally, and then repartition the sorted relation using range partitioning. Each node performs an ordered merge operation on the tuples that it receives, to generate a sorted output locally.

Thus, the exchange operator performs the partitioning of data at the source nodes, as well the merging of data at the destination nodes.

All the parallel operator implementations we have seen so far can be modeled as a sequence of exchange operations, and local operators, at each node, that are completely unaware of parallelism.

- Range partitioning sort: can be implemented by an exchange operation that performs range partitioning, with random merge at the destination nodes, followed by a local sort operation at each destination node.
- Parallel external sort-merge: can be implemented by local sorting at the source nodes, followed by an exchange operation that performs range partitioning, along with ordered merging.
- Partitioned join: can be implemented by an exchange operation that performs the desired partitioning, followed by local join at each node.
- Asymmetric fragment-and-replicate join: can be implemented by an exchange operation that performs broadcast “partitioning” of the smaller relation, followed by a local join at each node.
- Symmetric fragment-and-replicate join: can be implemented by an exchange operation that partitions, and partially broadcasts each partition, followed by a local join at each node.
- Aggregation: can be implemented by an exchange operation that performs hash-partitioning on the grouping attributes, followed by a local aggregation operation at each node. The partial-aggregation optimization simply requires an extra local aggregation operation at each node, before the exchange operation.

Other relational operations can be implemented similarly, by a sequence of local operations running in parallel at each node, interspersed with exchange operations.

As noted earlier, parallel execution where data are partitioned, and operations are executed locally at each node, is referred to as *data parallelism*. The use of the exchange operator model to implement data parallel execution has the major benefit of allowing existing database query engines to be used at each of the local nodes, without any significant code changes. As a result, the exchange-operator model of parallel execution is widely used in parallel database systems.

There are, however, some operator implementations that can benefit from being aware of the parallel nature of the system they are running on. For example, an indexed nested-loops join where the inner relation is indexed on a parallel data-store would require remote access for each index lookup; the index lookup operation is thus aware of the parallel nature of the underlying system. Similarly, in a shared-memory system it may make sense to have a hash table or index in shared-memory, which is accessed by multiple processors (this approach is discussed briefly in Section 22.6); the operations running on each processor are then aware of the parallel nature of the system.

As we discussed in Section 22.5.1.1, while the demand-driven (or pull) iterator model for pipelined execution of operators is widely used in centralized database engines, the push model is preferred for parallel execution of operators in a pipeline.

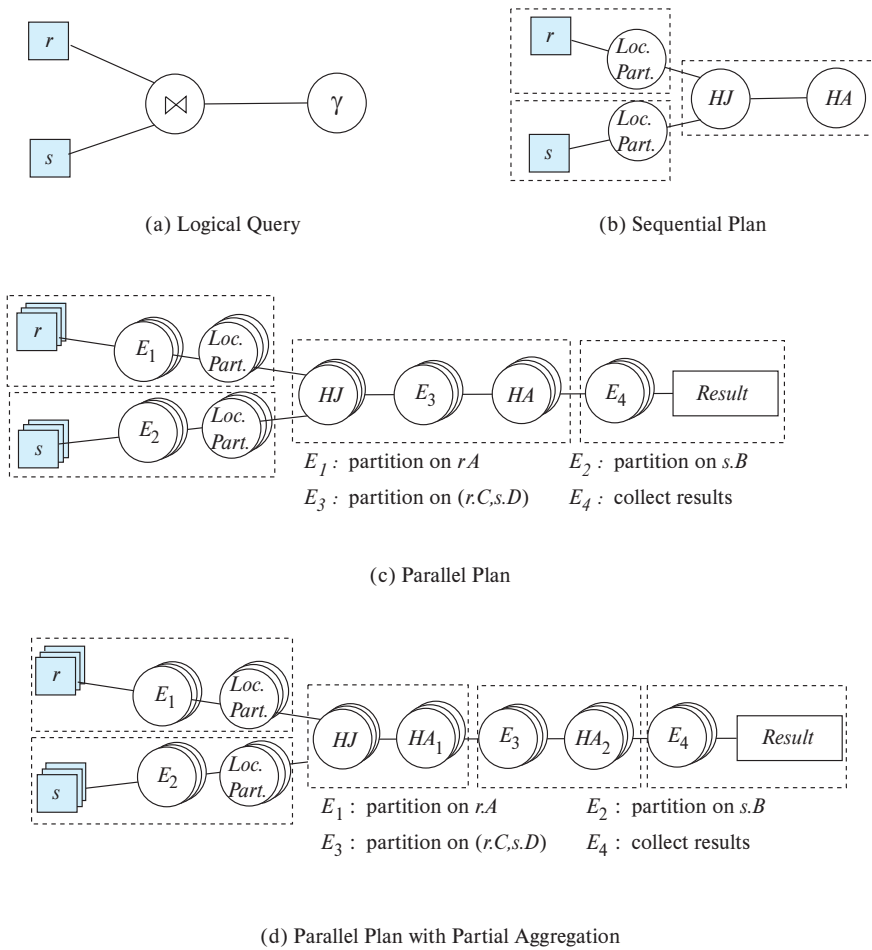
The exchange operator can be used to implement the push model between nodes in a parallel system, while allowing existing implementations of local operators to run using the pull model. To do so, at each source node of an exchange operator, the operator can pull multiple tuples from its input and create a batch of tuples destined for each destination node. The input may be computed by a local operation, whose implementation can use the demand-driven iterator model.

The exchange operator then sends batches of tuples to the destination nodes, where they are merged and kept in a buffer. The local operations can then consume the tuples in a demand-driven manner.

22.5.3 Putting It All Together

Figure 22.7 shows a query, along with a sequential and two alternative parallel query execution plans. The query, shown in Figure 22.7a, computes a join of two relations, r and s , and then computes an aggregate on the join result. Assume for concreteness that the query is $_{r.C,s.D}\gamma_{\text{sum}(s.E)}(r \bowtie_{r.A=s.B} s)$.

The sequential plan, shown in Figure 22.7b, uses a hash join (denoted as “HJ” in the figure), which executes in three separate stages. The first stage partitions the first input (r) locally on $r.A$; the second stage partitions the second input (s) locally on $s.B$; and the third stage computes the join of each of the corresponding partitions of r and s . The aggregate is computed using in-memory hash-aggregation, denoted by the operator HA; we assume that the number of groups is small enough that the hash table fits in memory.

**Figure 22.7** Parallel query execution plans.

The dashed boxes in the figure show which steps run in a pipelined fashion. In the sequential plan, the read of the relation r is pipelined to the first partitioning stage of the sequential hash join; similarly, the read of relations s is pipelined to the second partitioning stage of the hash join. The third stage of the hash join pipelines its output tuples to the hash aggregation operator.

The parallel query evaluation plan, shown in Figure 22.7c, starts with r and s already partitioned, but not on the required join attributes.³ The plan, therefore, uses the exchange operation E_1 to repartition r using attribute $r.A$; similarly, exchange operator E_2 repartitions s using $s.B$. Each node then uses hash join locally to compute the join

³Note the multiple boxes indicating a relation is stored in multiple nodes; similarly, multiple circles indicate that an operation is executed in parallel on multiple nodes.

of its partition of r and s . Note that these partitions are not assumed to fit in memory, so they must be further partitioned by the first two stages of hash join; this local partitioning step is denoted as “Loc. Part.” in the figure. The dashed boxes indicate that the output of the exchange operator can be pipelined to the local partitioning step. As in the sequential plan, there are two pipelined stages, one each for r and s . Note that exchange of tuples across nodes is done only by the exchange operator, and all other edges denote tuple flows within each node.

Subsequently, the hash join algorithm is executed in parallel at all participating nodes, and its output pipelined to the exchange operator E_3 . This exchange operator repartitions its input on the pair of attributes $(r.C, s.D)$, which are the grouping attributes of the subsequent aggregation. At the receiving end of the exchange operator, tuples are pipelined to the hash aggregation operator. Note that the above steps all run together as a single pipelined stage, even though there is an exchange operator as part of the stage. Note that the local operators computing hash join and hash aggregate need not be aware of the parallel execution.

The results of the aggregates are then collected together at a central location by the final exchange operator E_4 , to create the final result relation.

Figure 22.7d shows an alternative plan that performs partial aggregation on the results of the hash join, before partitioning the results. The partial aggregation is computed locally at each node by the operator HA_1 . Since no tuple is output by the partial aggregation operator until all its input is consumed, the pipelined stage contains only the local hash join and hash aggregation operators. The subsequent exchange operator E_3 which partitions its input on $(r.C, s.D)$ is part of a subsequent pipelined stage along with the hash aggregation operation HA_2 which computes the final aggregate values. As before, the exchange operator E_4 collects the results at a centralized location.

The above example shows how pipelined execution can be performed across nodes, as well as within nodes, and further how it can be done along with intra-operator parallel execution. The example also shows that some pipelined stages depend on the output of earlier pipelined stages; therefore their execution can start only after the previous step finishes. On the other hand, the initial exchange and partitioning of r and s occur in pipelined stages that are independent of each other; such independent stages can be scheduled concurrently, that is, at the same time, if desired.

To execute a parallel plan such as the one in our example, the different pipelined stages have to be scheduled for execution, in an order that ensures inter-stage dependencies are met. When executing a particular stage, the system must decide how many nodes an operation should be executed on. These decisions are usually made as part of the scheduling phase, before query execution starts.

22.5.4 Fault Tolerance in Query Plans

Parallel processing of queries across a moderate number of nodes, for example, hundreds of nodes, can be done without worrying about fault tolerance. If a fault occurs, the query is rerun, after removing any failed nodes from the system (replication of data

at the storage layer ensures that data continues to be available even in the event of a failure). However, this simple solution does not work well when operating at the scale of thousands or tens of thousands of nodes: if a query runs for several hours, there is a significant chance that there will be a failure while the query is being executed. If the query is restarted, there is a significant chance of another failure while it is executing, which is obviously an undesirable situation.

To deal with this problem, the query processing system should ideally just be able to redo the actions of a failed node, without redoing the rest of the computation.

Implementations of MapReduce that are designed to work at a massively parallel scale can be made fault tolerant as follows:

1. Each map operation executed at each node writes its output to local files.
2. The next operation, which is a reduce operation, executes at each node; the operation execution at a node reads data from the files stored at multiple nodes, collects the data, and starts processing the data only after it has got all its required data.
3. The reduce operation writes its output to a distributed file system (or distributed storage system) that replicates data, so that the data would be available even in the event of a failure.

Let us now examine the reason why things are done as above. First, if a particular map node fails, the work done at that node can be redone at a backup node; the work done at other map nodes is not affected. Work is not carried out by reduce nodes until all the required data has been fetched; the failure of a map node just means the reduce nodes fetch data from the backup map nodes. There is certainly a delay while the backup node does its work, but there is no need to repeat the entire computation.

Further, once a reduce node has finished its work, its output goes to replicated storage to ensure it is not lost even if a data storage node fails. This means that if a reduce node fails before it completes its work, it will have to be reexecuted at a backup node; other reduce nodes are not affected. Once a reduce node has finished its work, there is no need to reexecute it.

Note that it is possible to store the output of a map node in a replicated storage system. However, this increases the execution cost significantly, and hence map output is stored in local storage, even at the risk of having to reexecute the work done by a map node in case it fails before all the reduce nodes have fetched the data that they require from that map node.

It is also worth noting that sometimes nodes do not completely fail, but run very slowly; such nodes are called *straggler* nodes. Even a single straggler node can delay all the nodes in the next step (if there is a following step), or delay task completion (if it is in the last step). Straggler nodes can be dealt with by treating them similar to failed nodes, and reexecuting their tasks on other nodes (the original task on the straggler node can also be allowed to continue, in case it finishes first). Such reexecution to deal with

stragglers has been found to significantly improve time to completion of MapReduce tasks.

While the above scheme for fault tolerance is quite effective, there is an overhead that must be noted: a `reduce` stage cannot perform any work until the previous map stage has finished;⁴ and if multiple map and reduce steps are executed, the next map stage cannot perform any work until the preceding reduce stage has finished. In particular, this means that pipelining of data between stages cannot be supported; data are always materialized before it is sent to the next stage. Materialization carries a significant overhead, which can slow down computation.

Apache Spark uses an abstraction called *Resilient Distributed Datasets* (RDDs) to implement fault tolerance. As we have seen in Section 10.4.2, RDDs can be viewed as collections, and Spark supports algebraic operations that take RDDs as input, and generate RDDs as output. Spark keeps track of the operations used to create an RDD. In case of failures that result in loss of an RDD, the operations used to create the RDD can be reexecuted to regenerate the RDD. However, this may be time-consuming, so Spark also supports replication to reduce the chance of data loss, as well as storing of local copies of data when a shuffle (exchange) step is executed, to allow reexecution to be restricted to computation that was performed on failed nodes.

There has been a good deal of research on how to allow pipelining of data, while not requiring query execution to restart from the beginning in case of a single failure. Such schemes typically require nodes to track what data they have received from each source node. In the event of a source node failure, the work of the source node is redone on a backup node, which can result in some tuples that were received earlier being received again. Tracking the data received earlier is important to ensure duplicate tuples are detected and eliminated by the receiving node. The above ideas can also be used to implement fault tolerance for other algebraic operations, such as joins. In particular, if we use the exchange operator with data parallelism, fault tolerance can be implemented as an extension of the exchange operator.

References to more information on fault tolerant pipelining based on extensions of the exchange operator, as well as on fault tolerance schemes used in MapReduce and in Apache Spark, may be found in the Further Reading section at the end of the chapter.

22.6 Query Processing on Shared-Memory Architectures

Parallel algorithms designed for shared-nothing architectures can be used in shared-memory architectures. Each processor can be treated as having its own partition of memory, and we can ignore the fact that the processors have a common shared-

⁴Once a map node finishes its tasks, redistribution of results from that node to the reduce nodes can start even if other map nodes are still active; but the actual computation at the reduce node cannot start until all map tasks have completed and all map results redistributed.

memory. However, execution can be optimized significantly by exploiting the fast access to shared-memory from any of the processors.

Before we study optimizations that exploit shared-memory, we note that while many large-scale systems can execute on a single shared-memory system, the largest-scale systems today are typically implemented using a hierarchical architecture, with a shared-nothing architecture at the outer level, but with each node having a shared-memory architecture locally, as discussed in Section 20.4.8. The techniques we have studied so far for storing, indexing, and querying data in shared-nothing architectures are used to divide up storage, indexing, and query processing tasks among the different nodes in the system. Each node is a shared-memory parallel system, which uses parallel query processing techniques to execute the query processing tasks assigned to it. The optimizations we describe in this section can thus be used locally, at each node.

Parallel processing in a shared memory system is typically done by using threads, rather than separate processes. A **thread** is an execution stream that shares its entire memory⁵ with other threads. Multiple threads can be started up, and the operating system schedules threads on available processors.

We list below some optimizations that can be applied when parallel algorithms that we saw earlier are executed in a shared memory system.

1. If we use asymmetric fragment-and-replicate join, the smaller relation need not be replicated to each processor. Instead, only one copy needs to be stored in shared memory, which can be accessed by all the processors. This optimization is particularly useful if there are a large number of processors in the shared-memory system.
2. Skew is a significant problem in parallel systems, and it becomes worse as the number of processors grows. Handing off work from an overloaded node to an underloaded node is expensive in a shared-nothing system since it involves network traffic. In contrast, in a shared memory system, data assigned to a processor can be easily accessed from another processor.

To address skew in a shared-memory system, a good option is to use virtual-node partitioning, which allows work to be redistributed in order to balance load. Such redistribution could be done when a processor is found to be overloaded. Alternatively, whenever a processor finds that it has finished processing all the virtual nodes assigned to it, it can find other processors that still have virtual nodes left to be processed, and take over some of those tasks; as mentioned in Section 22.3.3, this approach is called *work stealing*. Note that such an approach to avoiding skew would be much more expensive in a shared-nothing environment since a significant amount of data movement would be involved, unlike in the shared-memory case.

⁵Technically, in operating-system terminology, its address space.

3. Hash join can be executed in two distinct ways.

- a. The first option is to partition both relations to each processor and then compute the joins of the partitions, in a manner similar to shared-nothing hash join. Each partition must be small enough that the hash index on a build-relation partition fits in the part of shared memory allocated to each processor.
- b. The second option is to partition the relations into fewer pieces, such that the hash index on a build-relation partition fits into common shared memory, rather than a fraction of the shared memory. The construction of the in-memory index, as well as probing of the index, must now be done in parallel by all the processors.

Parallelizing the probe phase is relatively easy, since each processor can work on some partition of the probe relation. In fact it makes sense to use the virtual node approach and partition the probe relation into many small pieces (sometimes called “morsels”), and have processors process a morsel at a time. When a processor is done with a morsel, it finds an unprocessed morsel and works on it, until there are no morsels left to be processed.

Parallelizing the construction of the shared hash index is more complicated, since multiple processors may attempt to update the same part of the hash index. Using locks is an option, but there are overheads due to locking. Techniques based on lock-free data structures can be used to construct the hash index in parallel.

References to more details on how to parallelize join implementations in shared memory may be found in the Further Reading section at the end of the chapter.

Algorithms designed for shared-memory systems must take into account the fact that in today’s processors, memory is divided into multiple memory banks, with each bank directly linked to some processor. The cost of accessing memory from a given processor is less if the memory is directly linked to the processor, and is more if it is linked to a different processor. Such memory systems are said to have a *Non-Uniform Memory Access* or *NUMA* architecture.

To get the best performance, algorithms must be *NUMA-aware*; that is, they must be designed to ensure that data accessed by a thread running on a particular processor is, as far as possible, stored in memory local to that processor. Operating systems support this task in two ways:

1. Each thread is scheduled, as far as possible, on the same processor core, every time it is executed.
2. When a thread requests memory from the operating system memory manager, the operating system allocates memory that is local to that processor core.

Note that the techniques for making the best use of shared memory are complementary to techniques that make the best use of processor caches, including cache-

conscious index structures (which we saw in Section 14.4.7) and cache-conscious algorithms for processing relational operators.

But in addition, since each processor core has its own cache, it is possible for a cache to have an old value that was subsequently updated on another processor core. Thus, query processing algorithms that update shared data structures should be careful to ensure that there are no bugs due to the use of outdated values, and due to race conditions on updating the same memory location from two processor cores. Locking and fence instructions to ensure cache consistency (Section 20.4.5) are used in combination to implement updates to shared data structures.

The form of parallelism we have studied so far allows each processor to execute its own code independently of other processors. However, some parallel systems support a different form of parallelism, called **Single Instruction Multiple Data (SIMD)**. With SIMD parallelism, the same instruction is executed on each of multiple data items, which are typically elements of an array. SIMD architectures became widely used in graphics processing units (GPUs), which were initially used for speeding up processing of computer graphics tasks. However, more recently, GPU chips have been used for parallelizing a variety of other tasks, one of which is parallel processing of relational operations using the SIMD support provided by GPUs. Intel's Xeon Phi coprocessor supports not only multiple cores in a single chip, but also several SIMD instructions that can operate in parallel on multiple words. There has been a good deal of research on how to process relational operations in parallel on such SIMD architectures; references to more information on this topic may be found in the bibliographic notes for this chapter, available online.

22.7 Query Optimization for Parallel Execution

Query optimizers for parallel query evaluation are more complicated than query optimizers for sequential query evaluation. First, the space of plan alternatives can be much larger than that for sequential plans. In particular, in a parallel setting, we need to take into account the different possible ways of partitioning the inputs and intermediate results, since different partitioning schemes can lead to different query execution costs, which is not an issue for a sequential plan.

Second, the cost models are more complicated, since the cost of partitioning must be taken into account, and issues such as skew and resource contention must be addressed.

22.7.1 Parallel Query Plan Space

As we have seen in Section 15.1, a sequential query plan can be expressed as an algebraic expression tree, where each node is a physical operator, such as a sort operator, a hash join operator, a merge-join operator, etc. Such a plan may be further annotated with instructions on what operations are to be pipelined and what intermediate results are to be materialized, as we saw in Section 15.7.

In addition to the above, a parallel query plan must specify

- How to parallelize each operation, including deciding what algorithm to use, and how to partition the inputs and intermediate results. Exchange operators can be used to partition inputs as well as intermediate results and final results.
- How the plan is to be **scheduled**; specifically:
 - How many nodes to use for each operation.
 - What operations to pipeline within the same node, or across different nodes.
 - What operations to execute sequentially, one after the other.
 - What operations to execute independently in parallel.

As an example of the partitioning decision, a join $r \bowtie_{r.A=s.A \wedge r.B=s.B} s$ can be parallelized by partitioning r and s on the attributes $r.A$ and $s.A$ alone, or on the attributes $r.B$ and $s.B$ alone, or on $(r.A, r.B)$ and $(s.A, s.B)$. The last option is likely to be the best if we consider only this join, since it minimizes the chances of skew which can arise if the cardinalities of $r.A$, $r.B$, $s.A$ or $s.B$ are low.

But consider now the query $_{r.A} \gamma_{\text{sum}(s.C)}(r \bowtie_{r.A=s.A \wedge r.B=s.B} s)$. If we perform the join by partitioning on $(r.A, r.B)$ (and $(s.A, s.B)$), we would then need to repartition the join result by $r.A$ to compute the aggregate. On the other hand, if we performed the join by partitioning on r and s on $r.A$ and $s.A$ respectively, both the join and the aggregate can be computed without any repartitioning, which could reduce the cost. This option is particularly likely to be a good option if $r.A$ and $s.A$ have high cardinality and few duplicates, since the chance of skew is less in this case.

Thus, the optimizer has to consider a larger space of alternatives, taking partitioning into account. References with more details about how to implement query optimization for parallel query processing systems, taking partitioning alternatives into account, may be found in the Further Reading section at the end of the chapter.

22.7.2 Cost of Parallel Query Evaluation

The cost of a sequential query plan is typically estimated based on the total resource consumption of the plan, adding up the CPU and I/O costs of the operators in a query plan. The *resource consumption* cost model can also be used in a parallel system, additionally taking into account the network cost, and adding it along with the other costs. As discussed in Section 15.2, even in a sequential system, the resource consumption cost model does not guarantee minimization of the execution time of an individual query. There are other cost models that are better at modeling the time to complete a query; however, the resource consumption cost model has the benefit of reducing the cost of query optimization, and is thus widely used. We return to the issue of other cost models later in the section.

We now study how the cost of a parallel query plan can be estimated based on the resource consumption model. If a query plan is data parallel, then each operation, other than the exchange operation, runs locally; the cost of such operations can be estimated using techniques we have seen earlier in Chapter 15, if we assume that the input relations are uniformly partitioned across n nodes, with each node receiving $1/n$ th of the overall input.

The cost of the exchange operation can be estimated based on the network topology, the amount of data transferred, and the network bandwidth; as before it is assumed that each node is equally loaded during the exchange operation. The cost of a query plan under the *resource-consumption* model can then be found by adding up the costs of the individual operations.

However, in a parallel system, two plans with the same resource consumption may have significantly different time to completion. A **response-time cost model** is a cost model that attempts to better estimate the time to completion of a query. If a particular operation is able to perform I/O operations in parallel with CPU execution, the response time would be better modeled as $\max(\text{CPU cost}, \text{I/O cost})$, rather than the resource consumption cost model of $(\text{CPU cost} + \text{I/O cost})$. Similarly, if two operations o_1 and o_2 are in a pipeline on a single node, and their CPU and I/O costs are c_1, io_1 and c_2, io_2 respectively, then their response time cost would be $\max(c_1 + c_2, io_1 + io_2)$. Similarly, if operations o_1 and o_2 are executed sequentially, then their response time cost would be $\max(c_1, io_1) + \max(c_2, io_2)$.

When executing operations in parallel across multiple nodes, the response time cost model would have to take into account:

- **Start-up costs** for initiating an operation at multiple nodes.
- **Skew** in the distribution of work among the nodes, with some nodes getting a larger number of tuples than others, and thus taking longer to complete. It is the time to completion of the slowest node that determines the time to completion of the operation.

Thus, any skew in the distribution of the work across nodes greatly affects performance.

Estimating the time to completion of the slowest node due to skew is not an easy task since it is highly data dependent. However, statistics such as number of distinct values of partitioning attributes, histograms on the distribution of values of partitioning attributes, and counts of most frequent values can be used to estimate the potential for skew. Partitioning algorithms that can detect and minimize the effect of skew, such as those discussed in Section 21.3, are important to minimize skew.

22.7.3 Choosing a Parallel Query Plan

The number of parallel evaluation plans from which to choose is much larger than the number of sequential evaluation plans. Optimization of parallel queries by considering all alternatives is therefore much more expensive than optimization of sequential

queries. Hence, we usually adopt heuristic approaches to reduce the number of parallel execution plans that we have to consider. We describe two popular approaches here.

1. A simple approach is to choose the most efficient sequential evaluation plan, and then to choose the optimal way to parallelize the operations in that evaluation plan. When choosing a sequential plan, the optimizer may use a basic sequential cost model; or it may use a simple cost model that takes some aspects of parallel execution into account but does not consider issues such as partitioning or scheduling. This option allows an existing sequential query optimizer to be used with minimal changes for the first step.

Next, the optimizer decides how to create an optimal parallel plan corresponding to the chosen sequential plan. At this point, choices of what partitioning techniques to use and how to schedule operators can be made in a cost-based manner.

The chosen sequential plan may not be optimal in the parallel context, since the exact cost formulae for parallel execution were not used when choosing it; nevertheless, the approach works reasonably well in many cases.

2. A more principled approach is to find the best plan, assuming that each operation is executed in parallel across all the nodes (operations with very small inputs may be executed on fewer nodes). Scheduling of independent operations in parallel on different nodes is not considered at this stage.

Partitioning of inputs and intermediate results is taken into consideration when estimating the cost of a query plan. Existing techniques for query optimization have been extended by considering partitioning as a physical property, in addition to physical properties such as sort orders that are already taken into account when choosing a sequential query plan. Just as sort operators are added to a query plan to get a desired sort order, exchange operators are added to get the desired partitioning property. The cost model used in practice is typically a resource consumption model, which we saw earlier. Although response-time cost models offer better estimates of query execution time, the cost of query optimization is higher when using a response-time cost model compared to the cost of optimization when using a resource-consumption cost model. References providing more information on the response-time cost model may be found in the Further Reading section at the end of the chapter.

Yet another dimension of optimization is the design of physical-storage organization to speed up queries. For example, a relation can be stored partitioned on any of several different attributes, and it may even be replicated and replicas can be stored partitioned on different attributes. For example, a relation $r(A, B, C)$ could be stored partitioned on A , and a replica could be partitioned on B . The query optimizer chooses the replica that is best suited for the query. The optimal physical organization differs

for different queries. The database administrator must choose a physical organization that appears to be good for the expected mix of database queries.

22.7.4 Colocation of Data

Even with parallel data storage and parallel processing of operations, the execution time of some queries can be too slow for the needs of some applications. In particular, queries that access small amounts of data stored at multiple nodes may run quite slowly when executed in parallel, as compared to the execution of the same query on a single node, if all the data were available at that node. There are many applications that need such queries to return answers with very low latency.

An important technique to speed up such queries is to colocate data that a query accesses in a single node. For example, suppose an application needs to access student information, along with information about courses taken by the student. Then, the *student* relation may be partitioned on the *ID*, and the *takes* relation also partitioned in exactly the same manner on *ID*. Tuples in the *course* relation, which is a small relation, may be replicated to all nodes. With such a partitioning, any query involving these three relations that retrieves data for a single *ID* can be answered locally at a single node. The query processing engine just detects which node is responsible for that *ID* and sends the query to that node, where it is executed locally.

Colocation of tuples from different relations is supported by many data storage systems. If the data storage system does not natively support colocation, an alternative is to create an object containing related tuples that share a key (e.g., *student* and *takes* records corresponding to a particular *ID*), and store it in the data storage system with the associated key (*ID*, in our example).

The colocation technique, however, does not work directly if different queries need a relation partitioned in different ways. For example, if the goal is to find all students who have taken a particular course section, the *takes* relation needs to be partitioned on the section information (*course_id*, *year*, *semester*, *sec_id*) instead of being partitioned on *ID*.

A simple technique to handle this situation is to allow multiple copies of a relation, partitioned on different attributes. These copies can be modeled as indices on the relation, partitioned on different attributes; when tuples in the relation are updated, so are the copies, to keep them consistent. In our example, one copy of *takes* can be partitioned on *ID* to be colocated with the *student* partitions, while another copy is partitioned on (*course_id*, *year*, *semester*, *sec_id*) to be colocated with the *section* relation.

Colocation helps optimize queries that compute joins between two relations; it can extend to three or more relations if either the remaining relations are replicated, or if all relations share a common set of join attributes. In the latter case, all tuples that would join together can be colocated by partitioning on the common join attributes. In either case, the join can be computed locally at a single node, if the query only wants the results for a specific value of the join attribute, as we saw earlier. However, not all join

queries are amenable to evaluation at a single node by using colocation. Materialized views, which we discuss next, offer a more general alternative.

22.7.5 Parallel Maintenance of Materialized Views

Materialized views, which we saw in Section 4.2.3 for speeding up queries in centralized databases, can also be used to speed up queries in parallel databases. Materialized views need to be maintained when the database is updated, as we saw in Section 16.5. Materialized views in a parallel database can have a very large amount of data, and must, therefore, be stored partitioned across multiple nodes.

As in the centralized case, materialized views in a parallel database speed up query answering at the cost of the overhead of view maintenance at the time of processing updates.

We consider first a very simple case of materialized views. It is often useful to store an extra copy of a relation, partitioned on different attributes, to speed up answering of some queries. Such a repartitioning can be considered a very simple case of a materialized view; view maintenance for such a view is straightforward, just requiring updates to be sent to the appropriate partition.

Indices can be considered a form of materialized views. Recall from Section 21.5 how parallel indices are maintained. Consider the case of maintenance of a parallel secondary index on an attribute B of a relation $r(A, B, C)$, with primary key A . The secondary index would be sorted on attribute B and would include unique key A ; assume it also includes attribute C . Suppose now that attribute B of a tuple $(a1, b1, c1)$ is updated from $b1$ to $b2$. This update results in two updates to the secondary index: delete the index entry with key $(b1, a1, c1)$, and add an entry $(b2, a1, c1)$. Since the secondary index is itself partitioned, these two updates need to be sent to the appropriate partition, based on the unique key attributes (B, A) .

In some cases, materialized view maintenance can be done by partitioning followed by local view maintenance. Consider a view that groups *takes* tuples by $(course_id, year, semester, sec_id)$, and then counts the number of students who have taken that course section. Such a materialized view would be stored partitioned on the grouping attributes $(course_id, year, semester, sec_id)$. It can be computed by maintaining a copy of the *takes* relation partitioned on $(course_id, year, semester, sec_id)$, and materializing the aggregates locally at each node. When there is an update, say an insert or delete to the *takes* relation, that update must be propagated to the appropriate node based on the partitioning chosen above. The materialized aggregate can be maintained locally at each node, as updates are received for the set of local tuples of the *takes* relation.

For more complex views, materialized view maintenance cannot be done by a single step of partitioning and local view maintenance. We consider a more general approach next.

First, consider an operator o , whose result is materialized, and an update (insert or delete) to one of the input relations of o that requires maintenance of the materialized result of o . Suppose the execution of operator o is parallelized using the exchange op-

erator model (Section 22.5.2) where the inputs are partitioned, and then operators are executed at each node on data made available locally by (re)partitioning. To support materialized view maintenance of the result of o , we materialize the output of o at each node and additionally materialize (store) the input partitions sent to the node when the materialized view result is initially computed.

Now, when there is an update (insert or delete) to an input to o , we send the update to the appropriate node using the same partition function used during the initial computation of o . Consider a node that has received such an update. Now, the maintenance of the locally materialized result at the node can be done using standard (nonparallel) view maintenance techniques using only locally available data.

Note that as we saw in Section 22.5.2, a variety of operations can be parallelized using the exchange operator model, and hence the preceding scheme provides a parallel view maintenance technique for all such operators.

Next, consider a query with multiple operators. Such a query can be parallelized using the exchange operator model. The exchange operators repartition data between nodes, and each node computing (possibly multiple) operations using data made available locally by the exchange operators, as we saw in Section 22.5.2.

We can materialize the inputs and results at each node. Now, when there is a change to an input at a node, we use standard view maintenance techniques locally to find the change to the materialized result, say v , at that node. If that result v is the final output of the query, we are done. Otherwise, there must be an exchange operator above it; we use the exchange operator to route the updates (inserts or delete) to v to the next operator. That operator in turn computes the change to its result, and propagates it further if required, until we get to the root of the original materialized view.

The issue of consistency of materialized views in the face of concurrent updates to the underlying relations is addressed in Section 23.6.3.

22.8 Parallel Processing of Streaming Data

We saw several applications of streaming data in Section 10.5. Many of the streaming data applications that we saw in that section, such as network monitoring or stock market applications, have very high rates of tuple arrival. Incoming tuples cannot be processed by a single computer, and parallel processing is essential for such systems. Streaming data systems apply a variety of operations on incoming data. We now see how some of these operations can be executed in parallel.

Parallelism is essential at all stages of query processing, starting with the entry of tuples from the sources. Thus, a parallel stream processing system needs to support a large number of entry points for data.

For example, a system that is monitoring queries posed on a search engine such as Google or Bing search has to keep up with a very high rate of queries. Search engines have a large number of machines across which user queries are distributed and executed. Each of these machines becomes a source for the query stream. The stream

processing system must have multiple entry points for the data, which receive data from the original sources and route them within the stream processing system.

Processing of data must be done by routing tuples from producers to consumers. We discuss routing of tuples in Section 22.8.1. Parallel processing of stream operations is discussed in Section 22.8.2, while fault tolerance is discussed in Section 22.8.3.

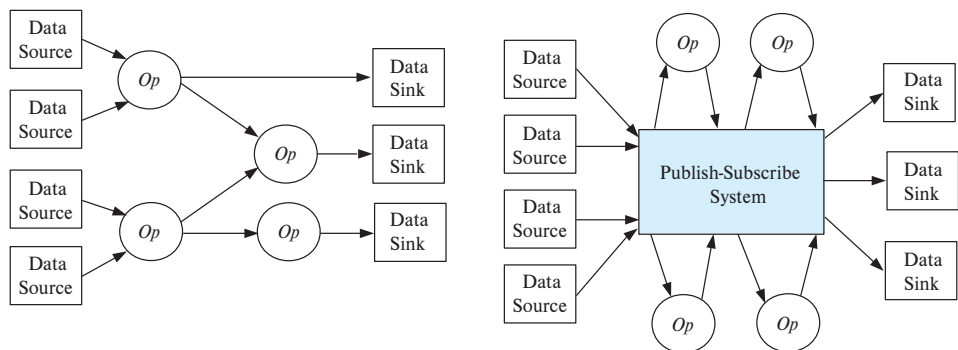
It is also worth noting that many applications that perform real-time analytics on streaming data also need to store the data and analyze it in other ways subsequently. Thus, many systems duplicate incoming data streams, sending one copy to a storage system for subsequent analysis and sending the other copy to a streaming data system; such an architecture is called the **lambda** architecture: the Greek symbol λ is used to pictorially denote that incoming data are forked into two copies, sent to two different systems.

While the lambda architecture allows streaming systems to be built quickly, it also results in duplication of effort: programmers need to write code to store and query the data in the format/language supported by a database, as well as to query the data in the language supported by a streaming data system. More recently, there have been efforts to perform stream processing as well as query processing on stored data within the same system to avoid this duplication.

22.8.1 Routing of Tuples

Since processing of data typically involves multiple operators, routing of data to operators is an important task. We first consider the logical structure of such routing, and address the physical structure, which takes parallel processing into account, later.

The *logical routing* of tuples is done by creating a directed acyclic graph (DAG) with operators as nodes. Edges between nodes define the flow of tuples. Each tuple output by an operator is sent along all the out-edges of the operator, to the consuming operators. Each operator receives tuples from all its in-edges. Figure 22.8a depicts the logical



(a) DAG representation of streaming data flow

(b) Publish-subscribe representation of streaming data flow

Figure 22.8 Routing of streams using DAG and publish-subscribe representations.

routing of stream tuples through a DAG structure. Operation nodes are denoted as “Op” nodes in the figure. The entry points to the stream processing system are the data source nodes of the DAG; these nodes consume tuples from the stream sources and inject them into the stream processing system. The exit points of the stream processing system are data sink nodes; tuples exiting the system through a data sink may be stored in a data store or file system or may be output in some other manner.

One way of implementing a stream processing system is by specifying the graph as part of the system configuration, which is read when the system starts processing tuples and is then used to route tuples. The Apache Storm stream processing system is an example of a system that uses a configuration file to define the graph, which is called a *topology* in the Storm system. (Data source nodes are called *spouts* in the Storm system, while operator nodes are called *bolts*, and edges connect these nodes.)

An alternative way of creating such a routing graph is by using **publish-subscribe** systems. A publish-subscribe system allows publication of documents or other forms of data, with an associated topic. Subscribers correspondingly subscribe to specified topics. Whenever a document is published to a particular topic, a copy of the document is sent to all subscribers who have subscribed to that topic. Publish-subscribe systems are also referred to as **pub-sub** systems for short.

When a publish-subscribe system is used for routing tuples in a stream processing system, tuples are considered documents, and each tuple is tagged with a topic. The entry points to the system conceptually “publish” tuples, each with an associated topic. Operators subscribe to one or more topics; the system routes all tuples with a specific topic to all subscribers of that topic. Operators can also publish their outputs back to the publish-subscribe system, with an associated topic.

A major benefit of the publish-subscribe approach is that operators can be added to the system, or removed from it, with relative ease. Figure 22.8b depicts the routing of tuples using a publish-subscribe representation. Each data source is assigned a unique topic name; the output of each operator is also assigned a unique topic name. Each operator subscribes to the topics of its inputs and publishes to the topics corresponding to its output. Data sources publish to their associated topic, while data sinks subscribe to the topics of the operators whose output goes to the sink.

The Apache Kafka system uses the publish-subscribe model to manage routing of tuples in streams. In the Kafka system, tuples published for a topic are retained for a specified period of time (called the retention period), even if there is currently no subscriber for the topic. Subscribers usually process tuples at the earliest possible time, but in case processing is delayed or temporarily stopped due to failures, the tuples are still available for processing until the retention time expires.

Many streaming data systems, such as Google’s Millwheel, and the Muppet stream processing system, use the term **stream** in place of the term *topic*. In such systems, streams are assigned names; operators can publish tuples to a stream, or subscribe to a stream, based on the name.

We now consider the *physical routing* of tuples. Regardless of the model used above, each logical operator must have multiple physical instances running in parallel on dif-

ferent nodes. Incoming tuples for a logical operator must be routed to the appropriate physical instance(s) of the operator. A partitioning function is used to determine which tuple goes to which instance of the operator.

In the context of a publish-subscribe system, each topic can be thought of as a separate logical operator that accepts tuples and passes them on to all subscribers of the topic. Since there may be a very large number of tuples for a given topic, they must be processed in parallel across multiple nodes in a parallel publish-subscribe system. In the Kafka system, for example, a topic is divided into multiple partitions, called a **topic-partition**; each tuple for that topic is sent to only one of the topic-partitions. Kafka allows a partition key to be attached to each tuple, and ensures that tuples with the same key are delivered to the same partition.

To allow processing by consumers, Kafka allows consumer operators register with a specified “consumer group.” The consumer group corresponds to a logical operator, while the individual consumers correspond to physical instances of the logical operator that run in parallel. Each tuple of a topic is sent to only one consumer in the consumer group. More precisely, all tuples in a particular topic-partition are sent to a single consumer in a consumer group; however, tuples from multiple partitions may be sent to the same consumer, leading to a many-to-one relationship from partitions to consumers.

Kafka is used in many streaming data processing implementations for routing tuples. Kafka Streams provides a client library supporting algebraic operations on streams, which can be used to build streaming applications on top of the Kafka publish-subscribe system.

22.8.2 Parallel Processing of Stream Operations

For standard relational operations, the techniques that we have seen for parallel evaluation of the operations can be used with streaming data.

Some of these, such as selection and projection operations, can be done in parallel on different tuples. Others, such as grouping, have to bring all tuples of a group together to one machine.⁶ When grouping is done with aggregation, optimizations such as pre-aggregation can be used to reduce the data transferred, but information about the tuples in a group must still be delivered to a single machine.

Windowing is an important operation in streaming data systems. Recall from Section 10.5.2.1 that incoming data are divided into windows, typically based on timestamps (windows can also be defined based on the number of tuples). Windowing is often combined with grouping/aggregation, with aggregates computed on groups of tuples within a window. The use of windows ensures that once the system can determine that new tuples will no longer belong to a particular window, aggregates for that window can be output. For example, suppose a window is based on time, say with each 5 minutes defining a window; once the system determines that future tuples will have a

⁶When grouping is combined with windowing (Section 10.5.2.1), a group contains all tuples in a window that have the same values for their grouping attributes.

timestamp larger than the end of a particular window, aggregates on that window can be output. Unlike grouping, windows can overlap each other.

When windowing and grouping are used together to compute aggregates, if there are overlapping windows, it is best to partition on just the grouping attributes. Otherwise, tuples which belong to multiple windows would have to be sent to multiple windows, an overhead that is avoided by partitioning on only the grouping attributes.

Many streaming systems allow users to create their own operators. It is important to be able to parallelize user-defined operators by allowing multiple instances of the operator to run concurrently. Such systems typically require each tuple to have an associated key, and all tuples with a particular key are sent to a particular instance of the operator. Tuples with different keys can be sent to different operator instances, allowing parallel processing.

Stream operations often need to store state. For example, a windowing operator may need to retain all tuples that it has seen in a particular window, as long as the window is active. Or, it may need to store aggregates computed at some resolution (say per minute) to later compute coarser resolution aggregates (say per hour). There are many other reasons for operators to store state. User-defined operators often define state internal to the operator (local variables), which needs to be stored.

Such state may be stored locally, at each node where a copy of the operator is executed. Alternatively, it may be stored centrally in a parallel data-storage system. The store-locally alternative has a lower cost but a higher risk of losing state information on failure, as compared to storing state in a parallel data-storage system. This aspect is discussed further in Section 22.8.3.

22.8.3 Fault Tolerance with Streaming Data

Fault tolerance when querying stored data can be achieved by reexecuting the query, or parts of the query, as we have seen in Section 22.5.4. However, such an approach to fault tolerance does not work well in a streaming setting for multiple reasons. First, many streaming data applications are latency sensitive, and delays in delivering results due to restarts are not desirable. Second, streaming systems provide a continuous stream of outputs. In the event of a failure, reexecuting the entire system or parts of it could potentially lead to duplicate copies of output tuples, which is not acceptable for many applications.

Thus, streaming data systems need to provide guarantees about delivery of output tuples, which can be one of: at-least once, at-most once, and exactly-once. The **at-least-once** semantics guarantees that each tuple is output at least once, but allows duplicate delivery during recovery from failures. The **at-most-once** semantics guarantees that each tuple is delivered at most once, without duplicates, but some tuples may be lost in the event of a failure. The **exactly-once** semantics guarantees that each tuple will be delivered exactly once, regardless of failures. This is the model that most applications require, although some applications may not care about duplicates and may accept at-least-once semantics.

To ensure such semantics, streaming systems must track what tuples have been processed at each operator and what tuples have been output. Duplicates can be detected by comparing against tuples output earlier and removed. (This can be done only if the system guarantees the absence of duplicates during normal processing, since otherwise the semantics of the streaming query may be affected by removal of genuine duplicates.)

One way to implement fault tolerance is to support it in the subsystem that routes tuples between operators. For example, in Kafka, tuples are published to topics, and each topic-partition can be stored in two or more nodes so that even if one of the nodes fails, the other one is available. Further, the tuples are stored on disk in each of the nodes so that they are not lost on power failure or system restart. Thus, the streaming data system can use this underlying fault tolerance and high availability mechanism to implement fault-tolerance and high availability at a higher level of the system.

In such a system, if an operator was executing on a failed node, it can be restarted on another node. The system must also (at least periodically) record up to what point each input stream had been consumed by the operator. The operator must be restarted and each input stream replayed from a point such that the operator can correctly output all tuples that it had not already output before the failure. This is relatively easy for operators without any state; operators without any state need to do extra work to restore the state that existed before failure. For example, a window operator needs to start from a point in the stream corresponding to the start of a window and replay tuples from that point.

If the window is very large, restarting from a very old point in the stream would be very inefficient. Instead, the operator may checkpoint its state periodically, along with points in the input stream up to which processing has been done. In the event of a failure, the latest checkpoint may be restored, and only input stream tuples that were processed since the last checkpoint need to be replayed.

The same approach can be for other operators that have state information; the state can be checkpointed periodically, and replay starts from the last checkpoint. The checkpointed state may be stored locally; however, this means that until the node recovers, stream processing cannot proceed. As an alternative, the state may be stored in a distributed file system or in a parallel data-storage system. Such systems replicate data to ensure high availability even in the event of failures. Thus, if a node has failed, its functions can be restarted on a different node, starting with the last checkpoint, and replaying the stream contents.

If the underlying system does not implement fault tolerance, operators can implement their own fault-tolerance mechanisms to avoid tuple loss. For example, each operator may store all tuples that it has output; a tuple can be discarded only after the operator knows that no consumer will need the tuple, even in the event of a failure.

Further, streaming systems must often guarantee low latency, even in the event of failures. To do so, some streaming systems have replicas of each operator, running concurrently. If one replica fails, the output can be fetched from the other replica. The system must make sure that duplicate tuples from the replicas are not output to

consumers. In such systems, one copy of an operator is treated as a primary copy, and the other copy as a hot-spare replica (recall that we discussed hot-spares in Section 19.7).

What we have described above is a high-level view of how streaming data systems implement fault tolerance. References to more information on how to implement fault tolerance in streaming systems may be found in the Further Reading section at the end of the chapter.

22.9 Distributed Query Processing

The need for distributed query processing originally arose when organizations needed to execute queries across multiple databases that were often geographically distributed. However, today the same need arises because organizations have data stored in multiple different databases and data storage systems, and they need to execute queries that access multiples of these databases and data storage systems.

22.9.1 Data Integration from Multiple Data Sources

Different parts of an enterprise may use different databases, either because of a legacy of how they were automated, or because of mergers of companies. Migrating an entire organization to a common system may be an expensive and time-consuming operation. An alternative is to keep data in individual databases, but to provide users with a logical view of integrated data. The local database systems may employ different logical models, different data-definition and data-manipulation languages, and may differ in their concurrency-control and transaction-management mechanisms. Some of the sources of data may not be full-fledged database systems but may instead be data storage systems, or even just files in a file system. Yet another possibility is that the data source may be on the cloud and accessed as a web service. Queries may need access to data stored across multiple databases and data sources.

Manipulation of information located in multiple databases and other data sources requires an additional software layer on top of existing database systems. This layer creates the illusion of logical database integration without requiring physical database integration and is sometimes called a **federated database system**.

Database integration can be done in several different ways:

- The **federated database** approach creates a common schema, called a **global schema**, for data from all the databases/data sources; each database has its own **local schema**. The task of creating a unified global schema from multiple local schemas is referred to as **schema integration**.

Users can issue queries against the global schema. A query on a global schema must be translated into queries on the local schemas at each of the sites where the query has to be executed. The query results have to be translated back into the global schema and combined to get the final result.

In general, updates to the common schema also need to be mapped to updates to the individual databases; systems that support a common schema and queries, but not updates, against the schema are sometimes referred to as **mediator** systems.

- The **data virtualization** approach allows applications to access data from multiple databases/data sources, but it does not try to enforce a common schema. Users have to be aware of the different schemas used in different databases, but they do not need to worry about which data are stored on which database system, or about how to combine information from multiple databases.
- The **external data** approach allows database administrators to provide schema information about data that are stored in other databases, along with other information, such as connection and authorization information needed to access the data. Data stored in external sources that can be accessed from a database are referred to as **external data**. **Foreign tables** are views defined in a database whose actual data are stored in an external data source. Such tables can be read as well as updated, depending on what operations the external data source supports. Updates on foreign tables, if supported, must be translated into updates on the external data source.

Unlike the earlier-mentioned approaches, the goal here is not to create a full-fledged distributed database, but merely to facilitate access to data from other data sources. The SQL Management of External Data (SQL MED) component of the SQL standard defines standards for accessing external data sources from a database.

If a data source is a database that supports SQL, its data can be easily accessed using ODBC or JDBC connections. Data in parallel data storage systems that do not support SQL, such as HBase, can be accessed using the API methods that they provide.

A **wrapper** provides a view of data stored at a data source, in a desired schema. For example, if the system has a global schema, and the local database schema is different from the global schema, a wrapper can provide a view of the data in the global schema. Wrappers can even be used to provide a relational view of nonrelational data sources, such as web services, flat files (e.g., web logs), and directory systems.

Wrappers can also translate queries on the global schema into queries on the local schema, and translate results back into the global schema. Wrappers may be provided by individual sites, they may be written as part of the federated database system. Many relational databases today support wrappers that provide a relational view of data stored in file systems; such wrappers are specific to the type of data stored in the files.

If the goal of data integration is solely to run decision support queries, *data warehouses*, which we saw in Section 11.2, are a widely used alternative to database integration. Data warehouses import data from multiple sources into a single (possibly parallel) system, with a centralized schema. Data are typically imported periodically,

for example, once in a day or once in a few hours, although continuous import is also increasingly used. Raw data imported from the data sources are typically processed and cleaned before being stored in the data warehouse.

However, with the scale at which data are generated by some applications, creating and maintaining such a warehouse can be expensive. Furthermore, queries cannot access the most recent data, since there is a delay between updates on the source database and import of the updates to the data warehouse. On the other hand, query processing can be done more efficiently in a data warehouse; further, queries at a data warehouse do not affect the performance of other queries and transactions at the data source. Whether to use a data warehouse architecture, or to directly access data from the data sources in response to individual queries, is a decision that each enterprise has to make based on its needs.

The term **data lake** is used to refer to an architecture where data are stored in multiple data storage systems and in different formats, including in file systems, but can be queried from a single system. Data lakes are viewed as an alternative to data warehouses, since they do not require up-front effort to preprocess data, although they do require more effort when creating queries.

22.9.2 Schema and Data Integration

The first task in providing a unified view of data lies in creating a unified conceptual schema, a task that is referred to as **schema integration**. Each local system provides its own conceptual schema. The database system must integrate these separate schemas into one common schema. Schema integration is a complicated task, mainly because of the semantic heterogeneity. The same attribute names may appear in different local databases but with different meanings.

Schema integration requires the creation of a **global schema**, which provides a unified view of data in different databases. Schema integration also requires a way to define how data are mapped from the local schema representation at each database, to the global schema. This step can be done by defining views at each site which, transform data from the local schema to the global schema. Data in the global schema is then treated as the union of the global views at the individual site. This approach is called the **global-as-view (GAV)** approach.

Consider an example with two sites which store student information in two different ways:

- Site s_1 which uses the relation $student1(ID, name, dept_name)$, and the relation $studentCreds(ID, tot_cred)$.
- Site s_2 which uses the relation $student2(ID, name, tot_cred)$, and the relation $studentDept(ID, dept_name)$.

Let the global schema chosen be $student(ID, name, dept_name, tot_cred)$.

Then, the global schema view at site *s1* would be defined as the view:

```
create view student_s1(ID, name, dept_name, tot_cred) as
select ID, name, dept_name, tot_cred
from student1, studentCreds
where student1.ID= studentCreds.ID;
```

While the global schema view at site *s2* would be defined as the view:

```
create view student_s2(ID, name, dept_name, tot_cred) as
select ID, name, dept_name, tot_cred
from student2, studentDept
where student2.ID= studentDept.ID;
```

Finally, the global schema *student* would be defined as the union of *student_s1* and *student_s2*.

Note that with the above view definition, a query on the global schema relation *student* can be easily translated into queries on the local schema relations at the sites *s1* and *s2*. It is harder to translate updates on the global schema into updates on the local schema, since there may not be a unique way to do so as discussed in Section 4.2.

There are more complex mapping schemes that are designed to deal with duplication of information across sites and to allow translation of updates on the global schema into updates on the local schema. The **local-as-view (LAV)** approach, which defines local data in each site as a view on a conceptual unified global relation, is one such approach.

Consider for example a situation where the *student* relation is partitioned between two sites based on the *dept_name* attribute, with all students in the “Comp. Sci.” department at site *s3* and all students in other departments in site *s4*. This can be specified using the local-as-view approach by defining the relations *student_s3* and *student_s4*, which are actually stored at the sites *s3* and *s4*, as equivalent to views defined on the global relation *student*.

```
create view student_s3 as
select *
from student
where student.dept_name = 'Comp. Sci.';
```

and

```
create view student_s4 as
select *
from student
where student.dept_name != 'Comp. Sci.';
```

With this extra information, the query optimizer can figure out that a query that seeks to retrieve students in the Comp. Sci. department need only be executed at site s_3 and need not be executed at site s_4 . More information on schema integration may be found in the bibliographic notes for this chapter, available online.

The second task in providing a unified view of data from multiple sources lies in dealing with differences in data types and values. For example, the data types used in one system may not be supported by other systems, and translation between types may not be simple. Even for identical data types, problems may arise from the physical representation of data: One system may use 8-bit ASCII, while another may use 16-bit Unicode; floating-point representations may differ; integers may be represented in *big-endian* or *little-endian* form. At the semantic level, an integer value for length may be inches in one system and millimeters in another; when integrating the data, a single representation must be used, and values converted to the chosen representation. Mapping between types can be done as part of the view definitions that translate data between the local schemas and the global schema.

A deeper problem is that the same conceptual entity may have different names in different systems. For example, a system based in the United States may refer to the city “Cologne,” whereas one in Germany refers to it as “Köln.” One approach to deal with this problem is to have a globally unique naming system, and map values to the unique names as part of the view definitions used for schema mappings. For example, the International Standards Organization has a unique code for country names, and for states/provinces within the countries. The GeoNames database (www.geonames.org) provides unique names for several million locations such as cities, geographical features, roads, buildings, and so forth.

When such standard naming systems are not available, some systems allow the specification of name equivalences; for example, such a system could allow a user to say that “Cologne” is the same as “Köln”. This approach is used in the [Linked Data](#) project, which supports the integration of a very large number of databases that use the RDF representation of data (the RDF representation is described in Section 8.1.4). However, querying is more complicated in such a scenario.

In our description of the view definitions above, we assumed that data are stored in local databases, and the view definitions are used to provide a global view of the data, without actually materializing the data in the global schema. However, such views can also be used to materialize the data in the global schema, which can then be stored in a data warehouse. In the latter case, updates on underlying data must be propagated to the data warehouse.

22.9.3 Query Processing Across Multiple Data Sources

A naive way to execute a query that accesses data from multiple data sources is to fetch all required data to one database, which then executes the query. But suppose, for example, that the query has a selection condition that is satisfied by only one or a few records out of a large relation. If the data source allows the selection to be performed at

the data source, it makes no sense to retrieve the entire relation; instead, the selection operation should be performed at the data source, while other operations, if any, may be performed at the database where the query was issued.

In general, different data sources may support different query capabilities. For example, if the source is a data storage system, it may support selections on key attributes only. Web data sources may restrict which fields selections are allowed on and may additionally require that selections be present on certain fields. On the other hand, if the source is a database that supports SQL, operations such as join or aggregation could be performed at the source and only the result brought over to the database that issues the query. In general, queries may have to be broken up and performed partly at the data source and partly at the site issuing the query.

The cost of processing a query that accesses multiple data sources depends on the local execution costs, as well as on the data transfer cost. If the network is a low bandwidth wide-area network, particular attention must be paid to minimizing data transfer.

In this section, we study issues in distributed query processing and optimization.

22.9.3.1 Join Locations and Join Ordering

Consider the following relational-algebra expression:

$$r_1 \bowtie r_2 \bowtie r_3$$

Assume that r_1 is stored at site S_1 , r_2 at S_2 , and r_3 at S_3 . Let S_I denote the site at which the query was issued. The system needs to produce the result at site S_I . Among the possible strategies for processing this query are these:

- Ship copies of all three relations to site S_I . Using the techniques of Chapter 16, choose a strategy for processing the entire query locally at site S_I .
- Ship a copy of the r_1 relation to site S_2 , and compute $temp_1 = r_1 \bowtie r_2$ at S_2 . Ship $temp_1$ from S_2 to S_3 , and compute $temp_2 = temp_1 \bowtie r_3$ at S_3 . Ship the result $temp_2$ to S_I .
- Devise strategies similar to the previous one, with the roles of S_1, S_2, S_3 exchanged.

There are several other possible strategies.

No one strategy is always the best one. Among the factors that must be considered are the volume of data being shipped, the cost of transmitting a block of data between a pair of sites, and the relative speed of processing at each site. Consider the first strategy. Suppose indices present at S_2 and S_3 are useful for computing the join. If we ship all three relations to S_I , we would need to either re-create these indices at S_I or use a different, possibly more expensive, join strategy. Re-creation of indices entails extra processing overhead and extra disk accesses. There are many variants of the second strategy, which process joins in different orders.

The cost of each of the strategies depends on the sizes of the intermediate results, the network transmission costs, and the costs of processing at each node. The query optimizer needs to choose the best strategy, based on cost estimates.

22.9.3.2 Semijoin Strategy

Suppose that we wish to evaluate the expression $r_1 \bowtie r_2$, where r_1 and r_2 are stored at sites S_1 and S_2 , respectively. Let the schemas of r_1 and r_2 be R_1 and R_2 . Suppose that we wish to obtain the result at S_1 . If there are many tuples of r_2 that do not join with any tuple of r_1 , then shipping r_2 to S_1 entails shipping tuples that fail to contribute to the result. We want to remove such tuples before shipping data to S_1 , particularly if network costs are high.

A possible strategy to accomplish all this is:

1. Compute $temp_1 \leftarrow \Pi_{R_1 \cap R_2}(r_1)$ at S_1 .
2. Ship $temp_1$ from S_1 to S_2 .
3. Compute $temp_2 \leftarrow r_2 \bowtie temp_1$ at S_2 .
4. Ship $temp_2$ from S_2 to S_1 .
5. Compute $r_1 \bowtie temp_2$ at S_1 . The resulting relation is the same as $r_1 \bowtie r_2$.

Before considering the efficiency of this strategy, let us verify that the strategy computes the correct answer. In step 3, $temp_2$ has the result of $r_2 \bowtie \Pi_{R_1 \cap R_2}(r_1)$. In step 5, we compute:

$$r_1 \bowtie r_2 \bowtie \Pi_{R_1 \cap R_2}(r_1)$$

Since join is associative and commutative, we can rewrite this expression as:

$$(r_1 \bowtie \Pi_{R_1 \cap R_2}(r_1)) \bowtie r_2$$

Since $r_1 \bowtie \Pi_{(R_1 \cap R_2)}(r_1) = r_1$, the expression is, indeed, equal to $r_1 \bowtie r_2$, the expression we are trying to evaluate.

This strategy is called a **semijoin strategy**, after the semijoin operator of the relational algebra, denoted $\ltimes$, which we saw in Section 16.4.4. The *natural semijoin* of r_1 with r_2 , denoted $r_1 \ltimes r_2$, is defined as:

$$r_1 \ltimes r_2 \stackrel{\text{def}}{=} \Pi_{R_1}(r_1 \bowtie r_2)$$

Thus, $r_1 \ltimes r_2$ selects those tuples of relation r_1 that contributed to $r_1 \bowtie r_2$. In step 3, $temp_2 = r_2 \ltimes r_1$. The semijoin operation is easily extended to theta-joins. The *theta semijoin* of r_1 with r_2 , denoted $r_1 \ltimes_\theta r_2$, is defined as:

$$r_1 \ltimes_\theta r_2 \stackrel{\text{def}}{=} \Pi_{R_1}(r_1 \bowtie_\theta r_2)$$

For joins of several relations, the semijoin strategy can be extended to a series of semijoin steps. It is the job of the query optimizer to choose the best strategy based on cost estimates.

This strategy is particularly advantageous when relatively few tuples of r_2 contribute to the join. This situation is likely to occur if r_1 is the result of a relational-algebra expression involving selection. In such a case, $temp_2$, that is, $r_2 \bowtie r_1$, may have significantly fewer tuples than r_2 . The cost savings of the strategy result from having to ship only $r_2 \bowtie r_1$, rather than all of r_2 , to S_1 .

Some additional cost is incurred in shipping $temp_1$, that is $\Pi_{R_1 \cap R_2}(r_1)$ to S_2 . If a sufficiently small fraction of tuples in r_2 contribute to the join, the overhead of shipping $temp_1$ will be dominated by the savings of shipping only a fraction of the tuples in r_2 . The overhead of sending $temp_1$ tuples from s_1 to s_2 can be reduced as follows. For the purpose of optimization of join processing, the semijoin operation can be implemented in a manner that overapproximates the true semijoin result. That is, the result should contain all the tuples in the actual semijoin result, but it may contain a few extra tuples. The extra tuples will get eliminated later by the join operation.

An efficient overapproximation of the semijoin result can be computed by using a probabilistic data structure called a **Bloom filter**, which uses bitmaps. Bloom filters are described in more detail in Section 24.1. To implement $r_2 \bowtie r_1$, a Bloom filter with a bitmap b of size m , initialized with all bits set to 0 is used. The join attributes of each tuple of r_1 are hashed to a value in the range $0 \dots (m - 1)$, and the corresponding bit of b is set to 1. The bitmap b , which is much smaller than the relation r_1 , can now be sent to the site containing r_2 . There, the same hash function is computed on the join attributes of each tuple of r_2 . If the corresponding bit is set to 1 in b , that r_2 tuple is accepted (added to the *result* relation), and otherwise it is rejected.

Note that it is possible for different join attribute values, say v_1 and v_2 to have the same hash value; even if r_1 has a tuple with value v_1 , but does not have any tuple with value v_2 , the result of the above procedure may include r_2 tuples whose join attribute value is v_2 . Such a situation is referred to as a **false positive**. However, if v_1 is present in r_1 , the technique will never reject a tuple in r_2 that has join attribute value v_1 , which is important for correctness.

The *result* relation computed above, which is a superset of or equal to $r_2 \bowtie r_1$, is sent to site s_1 . The join $r_1 \bowtie result$ is then computed at site s_1 to get the required join result. False positives may result in extra tuples in *result* that are not present in $r_2 \bowtie r_1$, but such tuples would be eliminated by the join.

To keep the probability of false positives low, the number of bits in the Bloom filter is usually set to a few times the estimated number of distinct join attribute values. Further, it is possible to use k independent hash functions, for some $k > 1$, to identify k bit positions for a given value, and set all of them to 1 when creating the bitmap. When querying it with a given value, the same k hash functions are used to identify k bit locations, and the value is determined to be absent if even one of the k bits has a 0 value. For example, if the bitmap has $10n$ bits, where n is the number of distinct join

attribute values, and $k = 7$ hash functions are used, the false positive rate would be about 1%.

22.9.3.3 Distributed Query Optimization

Several extensions need to be made to existing query optimization techniques in order to optimize distributed query plans.

The first extension is to record the location of data as a *physical property* of the data; recall that optimizers already deal with other physical properties such as the sort order of results. Just as the sort operation is used to create different sort orders, an exchange operation is used to transfer data between different sites.

The second extension is to track where an operator is executed; optimizers already consider different algorithms, such as hash join or merge join, for a given logical operator, in this case, the join operator. The optimizer is extended to additionally consider alternative sites for execution of each algorithm. Note that to execute an operator at a given site, its inputs must satisfy the physical property of being located at that site.

The third extension is to consider semijoin operations to reduce data transfer costs. Semijoin operations can be introduced as logical transformation rules; however, if done naively, the search space increases greatly, making this approach infeasible. Optimization cost can be reduced by restricting, as a heuristic, semijoins to be applied only on database tables, and never on intermediate join results.

A fourth extension is to use schema information to restrict the set of nodes at which a query needs to be executed. Recall from Section 22.9.2 that the local-as-view approach can be used to specify that a relation is partitioned in a particular way. In the example we saw there, site s_3 contains all student tuples with *dept_name* being Comp. Sci., while s_4 contains all the other student tuples. Suppose a query has a selection “*dept_name*='Comp. Sci.'” on *student*; then, the optimizer should recognize that there is no need to involve site s_4 when executing this query.

As another example, if the student data at site s_5 is a replica of the data at site s_3 , then the optimizer can choose to execute the query at either of the sites, depending on which is cheaper; there is no need to execute the query at both sites.

22.9.4 Distributed Directory Systems

A directory is a listing of information about some class of objects such as persons. Directories can be used to find information about a specific object, or in the reverse direction to find objects that meet a certain requirement. Several **directory access protocols** have been developed to provide a standardized way of accessing data in a directory.

A very widely used distributed directory system is the internet **Domain Name Service (DNS)** system, which provides a standardized way to map domain names (such as *db-book.com* or *www.cs.yale.edu*, to the IP addresses of the machines. (Although users see only the domain names, the underlying network routes messages based on IP addresses, and hence a way to convert domain names to IP addresses is critical for

the functioning of the internet.) The **Lightweight Directory Access Protocol (LDAP)** is another very widely used protocol designed for storing organizational data.

Data stored in directories can be represented in the relational model, stored in a relational database, and accessed through standard protocols such as JDBC or ODBC. The question then is, why come up with a specialized protocol for accessing directory information? There are several reasons.

- First, directory access protocols are simplified protocols that cater to a limited type of access to data. They evolved in parallel with the database access protocol standards.
- Second, directory systems were designed to support a hierarchical naming system for objects, similar to file system directory names. Such a naming system is important in many applications. For example, all computers whose names end in `yale.edu` belong to Yale, while those whose names end in `iitb.ac.in` belong to IIT Bombay. Within the `yale.edu` domain, there are subdomains such as `cs.yale.edu`, which corresponds to the CS department in Yale, and `math.yale.edu` which corresponds to the Math department at Yale.
- Third, and most important from a distributed systems perspective, the data in a distributed directory system are stored and controlled in a distributed, hierarchical, manner.

For example, a DNS server at Yale would store information about names of computers at Yale, along with associated information such as their IP addresses. Similarly, DNS servers at Lehigh and IIT Bombay would store information about computers in their respective domains. The DNS servers store information in a hierarchical fashion; for example, the information provided by the Yale DNS server may be stored in a distributed fashion across subdomains at Yale, such as the CS and Math DNS servers.

Distributed directory systems automatically forward queries submitted at a site to the site where the required information is actually stored, to give a unified view of data to users and applications.

Further, distributed directory implementations typically support replication to ensure the availability of data even if some nodes have failed.

Another example of usage of directory systems is for organization data. Such systems store information about employees, such as the employee identifier, name, email, organization unit (such as department), room number, phone number, and (encrypted) password of each employee. The schema of such organizational data are standardized as part of the Lightweight Directory Access Protocol (LDAP). Directory systems based on the LDAP protocol are widely used to authenticate users, using the encrypted passwords stored for each user. (More information about the LDAP data representation may be found in Section 25.5.)

CHAPTER 24



Advanced Indexing Techniques

We studied the concept of indexing, as well as a number of different index structures in Chapter 14. While some index structures, such as B⁺-trees, were covered in detail, others such as hashing, write-optimized indices, bitmap indices, and spatial indices were only briefly outlined in Chapter 14. In this chapter we provide further details of these index structures. We provide detailed coverage of the LSM tree and its variants. We then provide a detailed description of bitmap indices. Next, we provide more detailed coverage of spatial indexing, covering quad trees and R-trees in more detail. Finally, we cover hashing, with detailed coverage of dynamic hashing techniques.

24.1 Bloom Filter

A **Bloom filter** is a probabilistic data structure that can check for membership of a value in a set using very little space, but at a small risk of overestimating the set of elements that are in the set. A Bloom filter is basically a bitmap. If the set has n values, the associated bitmap has a few times n (typically $10n$) bits; the Bloom filter also has associated with it several hash functions. We assume initially that there is only one hash function $h()$.

The bits in the bitmap are all initially set to 0; subsequently, each value in the set is read, and the hash function $h(v)$ is computed on the element v , with the range of the function being 1 to $10n$. The bit at position $h(v)$ is then set to 1. This is repeated for every element v . To check if a particular value v is present in the set, the hash function $h(v)$ is computed. If bit $h(v)$ in the Bloom filter is equal to 0, we can infer that v cannot possibly be in the set. However, if bit $h(v)$ is equal to 1, v may be present in the set. Note that with some probability, the bit $h(v)$ may be 1 even if v is not present, if some other value v' , present in the set has $h(v') = h(v)$. Thus, a lookup for v results in a *false positive*.

To reduce the chance of false positives, Bloom filters use k independent hash functions $h_i()$, $i = 1..k$, for some $k > 1$; for each value v in the set, bits corresponding to $h_i(v)$, $i = 1..k$ are all set to 1 in the bitmap. When querying the Bloom filter with a

given value v the same k hash functions are used to identify k bit locations; the value v is determined to be absent if even one of these bits has a 0 value. Otherwise the value is judged to be potentially present. For example, if the bitmap has $10n$ bits, where n is the number of values in the set, and $k = 7$ hash functions are used, the false positive rate would be about 1%.

24.2 Log-Structured Merge Tree and Variants

As we saw in Section 14.8, B^+ -tree indices are not efficient for workloads with a very high number of writes, and alternative index structures have been proposed to handle such workloads. We saw a brief description of two such index structures, the *log-structured merge tree* or LSM tree and its variants, in Section 14.8.1, and the *buffer tree*, in Section 14.8.2. In this section we provide further details of the LSM tree and its variants. To help with the discussions, we repeat some of the basic material we presented in Section 14.8.1.

The key idea of the **log-structured merge tree (LSM tree)** is to replace random I/O operations during tree inserts, updates, and deletes with a smaller number of sequential I/O operations. Our initial description focuses on index inserts and lookups; we describe how to handle updates and deletes later in the section.

An LSM tree consists of several B^+ -trees, starting with an in-memory tree, called L_0 , and on-disk trees $L_1, L_2, \dots, L_k$ for some k , where k is called the level. Figure 24.1 depicts the structure of an LSM tree for $k = 3$.

An index lookup is performed by using separate lookup operations on each of the trees $L_0, \dots, L_k$, and merging the results of the lookups. (We assume here that there are no updates or deletes; we will discuss how to perform lookups in the presence of updates/deletes later.)

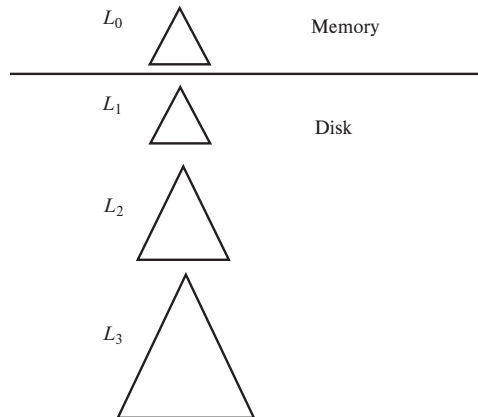


Figure 24.1 Log-structured merge tree with three levels.

24.2.1 Insertion into LSM Trees

When a record is first inserted into an LSM tree, it is inserted into the in-memory B⁺-tree structure L_0 . A fairly large amount of memory space is allocated for this tree. As the tree grows to fill the memory allocated to it, we need to move data from the in-memory structure to a B⁺-tree on disk.

If tree L_1 is empty, the entire in-memory tree L_0 is written to disk to create the initial tree L_1 . However, if L_1 is not empty, the leaf level of L_0 is scanned in increasing key order, and entries are merged with the leaf level entries of L_1 (also scanned in increasing key order). The merged entries are used to create a new B⁺-tree using the bottom-up build process. The new tree with the merged entries then replaces the old L_1 . In either case, after entries of L_0 have been moved to L_1 , all entries in L_0 are deleted. Inserts can then be made to the now empty L_0 .

Note that all entries in the leaf level of the old L_1 tree, including those in leaf nodes that do not have any updates, are copied to the new tree instead of being inserted into the existing L_1 tree node. This gives the following benefits:

- The leaves of the new tree are sequentially located, avoiding random I/O during subsequent merges.
- The leaves are full, avoiding the overhead of partially occupied leaves that can occur with page splits.

There is, however, a cost to using the LSM structure: the entire contents of the tree are copied each time a set of entries from L_0 are copied into L_1 .

If the tree structure is implemented on top of a distributed file system (Section 21.6), copying data to a new tree is often unavoidable, since most distributed file systems do not support updates to an already created block.

To ensure we get a benefit for cases where the index size on disk is much bigger than the in-memory index, the maximum size of L_1 is chosen as k times the target size of L_0 , for some k . Similarly, the maximum size of each L_{i+1} is set to k times the target size of L_i . Once a particular L_i reaches its maximum size, its entries are merged into the next component L_{i+1} . When L_{i+1} reaches its target size, its entries are in turn merged into L_{i+2} , and so on.

Note that if each leaf of L_i has m entries, m/k entries would map to a single leaf node of L_{i+1} . The value of k is chosen to ensure that m/k is some reasonable number, say 10. Let M denote the size of L_0 . Then, the size of a tree at level L_i is $k^i M$. The total number of levels r is thus roughly $\log_k(I/M)$ where I is the total size of the index entries.

Let us now consider the number of I/O operations required with a multiple-level LSM tree. At each L_i , m/k inserts are performed using only one I/O operation. On the other hand, each entry gets inserted once at each level L_i . Thus, the total number of I/O operations for each insert is $(k/m)\log_k(I/M)$. Thus, as long as the number of

levels $r = \log_k(I/M)$ is less than m/k , the overall number of I/O operations per insert is reduced by using an LSM tree as compared to direct insertion into a B^+ -tree.

If, for example, $r = 10$ and $k = 10$, the in-memory index needs to be greater than 1% of the total index size to get a benefit in terms of the number of I/O operations required for inserts. (As before, the benefit of reduced seeks is available even if the L_0 is significantly smaller.)

If the number of levels is greater than m/k , even if there is no benefit in terms of number of I/O operations, there can still be savings since sequential I/O is used instead of random I/O. In Section 24.2.4 we describe a variant of the LSM tree which further reduces the overhead on write operations, at the cost of adding overhead on read operations.

One way to avoid creating large LSM trees with many levels is to *range partition* the relation and create separate LSM trees on each partition. Such partitioning is natural in a parallel environment, as we saw earlier in Section 21.2. In particular, in such environments, a partition can be dynamically repartitioned into smaller pieces whenever it becomes too large, as we saw in Section 21.3.3. With such repartitioning, the size of each LSM tree can be kept small enough to avoid having a large number of levels. There is a price for such partitioning: each partition requires its own L_0 tree in memory. As a result, although it can be used in a centralized setting, the partitioning approach best fits a parallel environment where resources such as processing nodes can be added as the load increases.

24.2.2 Rolling Merges

We assumed for simplicity that when a particular level is full, its entries are entirely merged with the next level. This would result in more I/O load during merges with an unused I/O capacity between merges. To avoid this problem, merging is done on a continuous basis; this is called rolling merge.

With **rolling merge**, a few pages of L_i are merged into corresponding pages of L_{i+1} at a time, and removed from L_i . This is done whenever L_i becomes close to its target size, and it results in L_i shrinking a bit to return to its target size. When L_i grows again, the rolling merge restarts from a point at the leaf level of L_i just after where the earlier rolling merge stopped, so the scan is sequential. When the end of the L_i tree is reached, the scan starts again at the beginning of the tree. Such a merge is called a rolling merge since records are moved from one level to another on a continuous basis.

The number of leaves merged at a time is kept high enough to ensure that the seek time is small compared to the time to transfer data from and to disk.

24.2.3 Handling Deletes and Updates

So far we have only described inserts and lookups. Deletes are handled in an interesting manner. Instead of directly finding an index entry and deleting it, deletion results in insertion of a new **deletion entry** that indicates which index entry is to be deleted. The

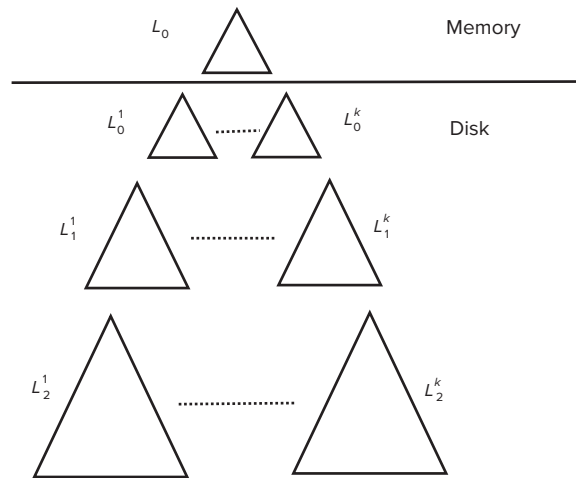


Figure 24.2 stepped-merge index

process of inserting a deletion entry is identical to the process of inserting a normal index entry.

However, lookups have to carry out an extra step. As mentioned earlier, lookups retrieve entries from all the trees and merge them in sorted order of key value. If there is a deletion entry for some entry, both of them would have the same key value. Thus, a lookup would find both the deletion entry and the original entry for that key, which is to be deleted. If a deletion entry is found, the to-be-deleted entry should be filtered out and not returned as part of the lookup result.

When trees are merged, if one of the trees contains an entry, and the other had a matching deletion entry, the entries get matched up during the merge (both would have the same key), and are both discarded.

Updates are handled in a manner similar to deletes, by inserting an update entry. Lookups need to match update entries with the original entries and return the latest value. The update is actually applied during a merge, when one tree has an entry and another has its matching update entry; the update is applied during the merge, and the update entry is discarded.

24.2.4 The Stepped-Merge Index

We now consider a variant of the LSM tree, which has multiple trees at each level instead of one tree per level and performs inserts in a slightly different manner. This structure is shown in Figure 24.2. We call the structure a **stepped-merge index**, following the terminology in an early paper that introduced it. In the developer community, the basic LSM tree, the stepped-merge index, and several other variants are all referred to as LSM trees. We use the terms *stepped-merge index* and *basic LSM tree* to clearly identify which variant we are referring to.

24.2.4.1 Insertion Algorithm

In the stepped-merge index, incoming data are initially stored in memory, in an L_0 tree, in a manner similar to the LSM tree. However, when the tree reaches its maximum size, instead of merging it into an L_1 tree, the in-memory L_0 tree is written to disk. When the in-memory tree again reaches its maximum size, it is again written to disk. Thus, we may have multiple L_0 trees on disk, which we shall refer to as L_0^1, L_0^2 and so forth. Each of the L_0^i trees is a B⁺-tree and can be written to disk using only sequential I/O operations.

If this process is repeated, after a while we would have a large number of trees, each as large as memory, stored on disk. Lookups would then have to pay a high price, since they would have to search through each of the tree structures, incurring separate I/O costs on each search.

To limit the overhead on lookups, once the number of on-disk trees at a level L_i reaches some limit k , all the trees at a level are merged together into one combined new tree structure at the next level L_{i+1} . The leaves of the trees at level L_i are read sequentially, and the keys merged in sorted order, and the level L_{i+1} tree is constructed using standard techniques for bottom-up construction of B⁺-trees. As before, the merge operation avoids random I/O operations, since it reads the individual tree structures sequentially and writes the resultant merged tree also sequentially.

Once a set of trees are merged into a single new tree, future queries can search the merged tree; the original trees can then be deleted (after ensuring any ongoing searches have completed).

The benefit of the stepped-merge index scheme as compared to the basic LSM tree is that index entries are written out only once per level. With the basic LSM tree, each time a tree at level L_i is merged into a tree at level L_{i+1} , the entire contents of the L_{i+1} tree is read and written back to disk. Thus, on average, each record is read and written back $k/2$ times at each level of an LSM tree for a total of $k \log_k(I/M)$ I/O operations. In contrast, with stepped-merge index, each record is written to disk once per layer, and read again when merging into the next layer, for a total of approximately $2 \log_k(I/M)$ I/O operations. Thus, the stepped-merge index incurs significantly less cost for updates.

The total number of bytes written (across all levels) on account of inserting an entry, divided by the size of entry, is called the **write amplification**. To calculate the write amplification of the LSM trees and the stepped-merge index, we can modify the above formulae for I/O operations by ignoring the reads. For a B⁺-tree where each leaf gets on average only one update before it is written back, the write amplification would be the size of the page divided by the size of the index entry.

For a B⁺-tree, if a page has 100 entries, the write amplification would be 100. With $k = 5$, and $I = 100M$, we would have $\log_5(100) = 3$ levels. The write amplification of an LSM tree would then be $5/2 \times 3 = 7.5$. The write amplification of the stepped-merge index would be 3. With $k = 10$, the tree would have $\log_{10}(100) = 2$ levels, leading to a write amplification of 2 for stepped-merge index, and 10 for an LSM tree.

Note that like the basic LSM tree, the stepped-merge index also requires no random I/O operations during insertion, in contrast to a B⁺-tree insertion. Thus, the performance of B⁺-trees would be worse than what the write amplification number above indicates.

Merging can be optimized as follows: While merging k trees at a particular level L_i , into a level L_{i+1} tree, trees at levels $L_j, j < i$ can also be merged in at the same time. Entries in these trees can thus entirely skip one or more levels of the stepped-merge index. Further, if the system has idle capacity, trees at a level L_i can be merged even if there are fewer than k trees at that level. In a situation where there is a long period of time with very few inserts, and the system load is light, trees across all levels could potentially get merged into a single tree at some level r .

24.2.4.2 Lookup Operations Using Bloom Filters

Lookup operations in stepped-merge index have to separately search each of the trees. Thus, compared to the basic LSM scheme, the stepped-merge index increases the burden on lookups, since in the worst case lookups need to access k trees at each level, leading to a total of $k * \log_k(I/M)$ tree lookups, instead of $\log_k(I/M)$ tree lookups in the worst case with a basic LSM tree.

For workloads with a significant fraction of reads, this overhead can be unacceptable. For example, with the stepped-merge index with $I = 100M$ and $k = 5$, a single lookup requires 15 tree traversals, while the LSM tree would require 3 tree traversals. Note that for the common case where each key value occurs in only one tree, only one of the traversals would find a given search key, while all the other traversals would fail to find the key.

To reduce the cost of point lookups (i.e., lookups of a given key value), most systems use a Bloom filter to check if a tree can possibly contain the given key value. One Bloom filter is associated with each tree, and it is built on the set of key values in the tree. To check if a particular tree may contain a search key v , the key v is looked up in the Bloom filter. If the Bloom filter indicates that the key value is absent, it is definitely not present in the tree, and lookup can skip that tree. Otherwise, the key value may be present in the tree, which must be looked up.

A Bloom filter with $10n$ bits, where the tree has n elements, and using 7 hash functions would give a false positive rate of 1 percent. Thus, for a lookup on a key that is present in the index, on average just slightly more than one tree would be accessed. Thus, lookup performance would be only slightly worse than on a regular B⁺-tree.

The Bloom filter check thus works very well for point lookups, allowing a significant fraction of the trees to be skipped, as long as sufficient memory is available to store all the Bloom filters in memory. With I key values in the index, approximately $10I$ bits of memory will be required. To reduce the main memory overhead, some of the Bloom filters may be stored on flash storage.

Note that for range lookups, the Bloom filter optimization cannot be used, since there is no unique hash value. Instead, all trees must be accessed separately.

24.2.5 LSM Trees For Flash Storage

LSM trees were initially designed to reduce the write and seek overheads of hard disks. Flash disks have a relatively low overhead for random I/O operations since they do not require seek, and thus the benefit of avoiding random I/O that LSM tree variants provide is not particularly important with flash disks.

However, recall that flash memory does not allow in-place update, and writing even a single byte to a page requires the whole page to be rewritten to a new physical location; the original location of the page needs to be erased eventually, which is a relatively expensive operation. The reduction in write amplification using LSM tree variants, as compared to traditional B⁺-trees, can provide substantial performance benefits when LSM trees are used with flash storage.

24.3 Bitmap Indices

As we saw in Section 14.9, a bitmap index is a specialized type of index designed for easy querying on multiple keys. Bitmaps work best for attributes that take only a small number of distinct values.

For bitmap indices to be used, records in a relation must be numbered sequentially, starting from, say, 0. Given a number n , it must be easy to retrieve the record numbered n . This is particularly easy to achieve if records are fixed in size and allocated on consecutive blocks of a file. The record number can then be translated easily into a block number and a number that identifies the record within the block.

Recall that *column-oriented storage*, described in Section 13.6, stores attributes in arrays, allowing efficient access of the attribute of the i th record, for any given i . Bitmap indices are thus particularly useful with columnar storage.

We use as a running example a relation *instructor_info*, which has an attribute *gender*, which can take only values **m** (male) or **f** (female), and an attribute *income_level*, where income has been broken up into 5 levels: $L1$: 0-9999, $L2$: 10,000-19,999, $L3$: 20,000-39,999, $L4$: 40,000-74,999, and $L5$: 75,000 – ∞ .

24.3.1 Bitmap Index Structure

As we saw in Section 14.9, a **bitmap** is simply an array of bits. In its simplest form, a **bitmap index** on the attribute A of relation r consists of one bitmap for each value that A can take. Each bitmap has as many bits as the number of records in the relation. The i th bit of the bitmap for value v_j is set to 1 if the record numbered i has the value v_j for attribute A . All other bits of the bitmap are set to 0.

In our example, there is one bitmap for the value **m** and one for **f**. The i th bit of the bitmap for **m** is set to 1 if the *gender* value of the record numbered i is **m**. All other bits of the bitmap for **m** are set to 0. Similarly, the bitmap for **f** has the value 1 for bits corresponding to records with the value **f** for the *gender* attribute; all other bits have

record number				Bitmaps for <i>gender</i>		Bitmaps for <i>income_level</i>	
	<i>ID</i>	<i>gender</i>	<i>income_level</i>	m	f	L1	L2
0	76766	m	L1	10010	01101	10100	01000
1	22222	f	L2			01000	00001
2	12121	f	L1			00001	00010
3	15151	m	L4			00010	00000
4	58583	f	L3			00000	

Figure 24.3 Bitmap indices on relation *instructor_info*.

the value 0. Figure 24.3 shows an example of bitmap indices on a relation *instructor_info*.

We now consider when bitmaps are useful. The simplest way of retrieving all records with value *m* (or value *f*) would be to simply read all records of the relation and select those records with value *m* (or *f*, respectively). The bitmap index doesn't really help to speed up such a selection. While it would allow us to read only those records for a specific gender, it is likely that every disk block for the file would have to be read anyway.

In fact, bitmap indices are useful for selections mainly when there are selections on multiple keys. Suppose we create a bitmap index on attribute *income_level*, which we described earlier, in addition to the bitmap index on *gender*.

Consider now a query that selects women with income in the range \$10,000 to \$19,999. This query can be expressed as

```
select *
from instructor_info
where gender = 'f' and income_level = 'L2';
```

To evaluate this selection, we fetch the bitmaps for *gender* value *f* and the bitmap for *income_level* value *L2* and perform an **intersection** (logical-and) of the two bitmaps. In other words, we compute a new bitmap where bit *i* has value 1 if the *i*th bit of the two bitmaps are both 1 and has a value 0 otherwise. In the example in Figure 24.3, the intersection of the bitmap for *gender* = *f* (01101) and the bitmap for *income_level* = *L2* (01000) gives the bitmap 01000.

Since the first attribute can take two values, and the second can take five values, we would expect only about 1 in 10 records, on an average, to satisfy a combined condition on the two attributes. If there are further conditions, the fraction of records satisfying all the conditions is likely to be quite small. The system can then compute the query result by finding all bits with value 1 in the intersection bitmap and retrieving the cor-

responding records. If the fraction is large, scanning the entire relation would remain the cheaper alternative.

Another important use of bitmaps is to count the number of tuples satisfying a given selection. Such queries are important for data analysis. For instance, if we wish to find out how many women have an income level $L2$, we compute the intersection of the two bitmaps and then count the number of bits that are 1 in the intersection bitmap. We can thus get the desired result from the bitmap index, without even accessing the relation.

Bitmap indices are generally quite small compared to the actual relation size. Records are typically at least tens of bytes to hundreds of bytes long, whereas a single bit represents the record in a bitmap. Thus, the space occupied by a single bitmap is usually less than 1 percent of the space occupied by the relation. For instance, if the record size for a given relation is 100 bytes, then the space occupied by a single bitmap will be $\frac{1}{8}$ of 1 percent of the space occupied by the relation. If an attribute A of the relation can take on only one of eight values, a bitmap index on attribute A would consist of eight bitmaps, which together occupy only 1 percent of the size of the relation.

Deletion of records creates gaps in the sequence of records, since shifting records (or record numbers) to fill gaps would be extremely expensive. To recognize deleted records, we can store an **existence bitmap**, in which bit i is 0 if record i does not exist and 1 otherwise. We shall see the need for existence bitmaps in Section 24.3.2. Insertion of records should not affect the sequence numbering of other records. Therefore, we can do insertion either by appending records to the end of the file or by replacing deleted records.

24.3.2 Efficient Implementation of Bitmap Operations

We can compute the intersection of two bitmaps easily by using a **for** loop: the i th iteration of the loop computes the **and** of the i th bits of the two bitmaps. We can speed up computation of the intersection greatly by using bit-wise **and** instructions supported by most computer instruction sets. A *word* usually consists of 32 or 64 bits, depending on the architecture of the computer. A bit-wise **and** instruction takes two words as input and outputs a word where each bit is the logical **and** of the bits in corresponding positions of the input words. What is important to note is that a single bit-wise **and** instruction can compute the intersection of 32 or 64 bits *at once*.

If a relation had 1 million records, each bitmap would contain 1 million bits, or equivalently 128 kilobytes. Only 31,250 instructions are needed to compute the intersection of two bitmaps for our relation, assuming a 32-bit word length. Thus, computing bitmap intersections is an extremely fast operation.

Just as bitmap intersection is useful for computing the **and** of two conditions, bitmap union is useful for computing the **or** of two conditions. The procedure for bitmap union is exactly the same as for intersection, except we use bit-wise **or** instructions instead of bit-wise **and** instructions.

The complement operation can be used to compute a predicate involving the negation of a condition, such as **not** (*income_level* = *L1*). The complement of a bitmap is generated by complementing every bit of the bitmap (the complement of 1 is 0 and the complement of 0 is 1). It may appear that **not** (*income_level* = *L1*) can be implemented by just computing the complement of the bitmap for income level *L1*. If some records have been deleted, however, just computing the complement of a bitmap is not sufficient. Bits corresponding to such records would be 0 in the original bitmap but would become 1 in the complement, although the records don't exist. A similar problem also arises when the value of an attribute is *null*. For instance, if the value of *income_level* is null, the bit would be 0 in the original bitmap for value *L1* and 1 in the complement bitmap.

To make sure that the bits corresponding to deleted records are set to 0 in the result, the complement bitmap must be intersected with the existence bitmap to turn off the bits for deleted records. Similarly, to handle null values, the complement bitmap must also be intersected with the complement of the bitmap for the value *null*.¹

Counting the number of bits that are 1 in a bitmap can be done quickly by a clever technique. We can maintain an array with 256 entries, where the *i*th entry stores the number of bits that are 1 in the binary representation of *i*. Set the total count initially to 0. We take each byte of the bitmap, use it to index into this array, and add the stored count to the total count. The number of addition operations is $\frac{1}{8}$ of the number of tuples, and thus the counting process is very efficient. A large array (using $2^{16} = 65,536$ entries), indexed by pairs of bytes, would give even higher speedup, but at a higher storage cost.

24.3.3 Bitmaps and B⁺-Trees

Bitmaps can be combined with regular B⁺-tree indices for relations where a few attribute values are extremely common, and other values also occur, but much less frequently. In a B⁺-tree index leaf, for each value we would normally maintain a list of all records with that value for the indexed attribute. Each element of the list would be a record identifier, consisting of at least 32 bits, and usually more. For a value that occurs in many records, we store a bitmap instead of a list of records.

Suppose a particular value v_i occurs in $\frac{1}{16}$ of the records of a relation. Let N be the number of records in the relation, and assume that a record has a 64-bit number identifying it. The bitmap needs only 1 bit per record, or N bits in total. In contrast, the list representation requires 64 bits per record where the value occurs, or $64 * N/16 = 4N$ bits. Thus, a bitmap is preferable for representing the list of records for value v_i . In our example (with a 64-bit record identifier), if fewer than 1 in 64 records have a particular value, the list representation is preferable for identifying records with that

¹Handling predicates such as **is unknown** would cause further complications, which would in general require use of an extra bitmap to track which operation results are unknown.

student, instructor, and so on). We briefly describe the architecture of directory systems.

25.1 Performance Tuning

Tuning the performance of a system involves adjusting various parameters and design choices to improve its performance for a specific application. Various aspects of a database-system design—ranging from high-level aspects such as the schema and transaction design to database parameters such as buffer sizes, down to hardware issues such as number of disks—affect the performance of an application. Each of these aspects can be adjusted so that performance is improved.

25.1.1 Motivation for Tuning

Applications sometimes exhibit poor performance, with queries taking a long time to complete, leading to users being unable to carry out tasks that they need to do. We describe a few real-world examples that we have seen, including their causes and how tuning fixed the problems.

In one of the applications, we found that users were experiencing long delays and time-outs in the web applications. On monitoring the database, we found that the CPU usage was very high, with negligible disk and network usage. Further analysis of queries running on the database showed that a simple lookup query on a large relation was using a full relation scan, which was quite expensive. Adding an index to the attribute used in the lookup drastically reduced the execution time of the query and a key performance problem vanished immediately.

In a second application, we found that a query had very poor performance. Examining the query, we found that the programmer had written an unnecessarily complicated query, with several nested subqueries, and the optimizer produced a bad plan for the query, as we realized after observing the query plan. To fix the problem, we rewrote the query using joins instead of nested subqueries, that is, we decorrelated the query; this change greatly reduced the execution time.

In a third application, we found that the application fetched a large number of rows from a query, and issued another database query for each row that it fetched. This resulted in a large number of separate queries being sent to the database, resulting in poor performance. It is possible to replace such a large number of queries with a single query that fetches all required data, as we see later in this section. Such a change improved the performance of the application by an order of magnitude.

In a fourth application, we found that while the application performed fine under light load during testing, it completely stopped working when subjected to heavy load when it was used by actual users. In this case, we found that in some of the interfaces, programmers had forgotten to close JDBC connections. Databases typically support only a limited number of JDBC connections, and once that limit was reached, the application was unable to connect to the database, and thus it stopped working.

Ensuring that connections were closed fixed this problem. While this was technically a bug fix, not a tuning action, we thought it is a good idea to highlight this problem since we have found many applications have this problem. Connection pooling, which keeps database connections open for use by subsequent transactions, is a related application tuning optimization, since it avoids the cost of repeated opening and closing of database connections.

It is also worth pointing out that in several cases above the performance problems did not show up during testing, either because the test database was much smaller than the actual database size or because the testing was done with a much lighter load (number of concurrent users) than the load on the live system. It is important that performance testing be done on realistic database sizes, with realistic load, so problems show up during testing, rather than on a live system.

25.1.2 Location of Bottlenecks

The performance of most systems (at least before they are tuned) is usually limited primarily by the performance of one or a few components, called **bottlenecks**. For instance, a program may spend 80 percent of its time in a small loop deep in the code, and the remaining 20 percent of the time on the rest of the code; the small loop then is a bottleneck. Improving the performance of a component that is not a bottleneck does little to improve the overall speed of the system; in the example, improving the speed of the rest of the code cannot lead to more than a 20 percent improvement overall, whereas improving the speed of the bottleneck loop could result in an improvement of nearly 80 percent overall, in the best case.

Hence, when tuning a system, we must first try to discover what the bottlenecks are and then eliminate them by improving the performance of system components causing the bottlenecks. When one bottleneck is removed, it may turn out that another component becomes the bottleneck. In a well-balanced system, no single component is the bottleneck. If the system contains bottlenecks, components that are not part of the bottleneck are underutilized, and could perhaps have been replaced by cheaper components with lower performance.

For simple programs, the time spent in each region of the code determines the overall execution time. However, database systems are much more complex, and query execution involves not only CPU time, but also disk I/O and network communication. A first step in diagnosing problems is to use monitoring tools provided by operating systems to find the usage level of the CPU, disks, and network links.

It is also important to monitor the database itself, to find out what is happening in the database system. For example, most databases provide ways to find out which queries (or query templates, where the same query is executed repeatedly with different constants) are taking up the maximum resources, such as CPU, disk I/O, or network capacity. In addition to hardware resource bottlenecks, poor performance in a database system may potentially be due to contention on locks, where transactions wait in lock

Note 25.1 DATABASE PERFORMANCE MONITORING TOOLS

Most database systems provide view relations that can be queried to monitor database system performance. For example, PostgreSQL provides view relations `pg_stat_statements` and `pgpgrowlocks` to monitor resource usage of SQL statements and lock contention respectively. MySQL supports a command `show processinfo` that can be used to monitor what transactions are currently executing and their resource usage. Microsoft SQL Server provides stored procedures `sp_monitor`, `sp_who`, and `sp_lock` to monitor system resource usage. The Oracle Database SQL Tuning Guide, available online, provides details of similar views in Oracle.

queues for a long time. Again, most databases provide mechanisms to monitor lock contention.

Monitoring tools can help detect where the bottleneck lies (such as CPU, I/O, or locks), and to locate the queries that are causing the maximum performance problems. In this chapter, we discuss a number of techniques that can be used to fix performance problems, such as adding required indices or materialized views, rewriting queries, rewriting applications, or adding hardware to improve performance.

To understand the performance of database systems better, it is very useful to model database systems as **queueing systems**. A transaction requests various services from the database system, starting from entry into a server process, disk reads during execution, CPU cycles, and locks for concurrency control. Each of these services has a queue associated with it, and small transactions may spend most of their time waiting in queues—especially in disk I/O queues—instead of executing code. Figure 25.1 illustrates some of the queues in a database system. Note that each lockable item has a separate queue in the concurrency control manager. The database system may have a single queue at the disk manager or may have separate queues for different disks in case the disks are directly controlled by the database. The transaction queue is used by the database system to control the admission of new queries when the number of requests exceeds the number of concurrent query execution tasks that the database allows.

As a result of the numerous queues in the database, bottlenecks in a database system typically show up in the form of long queues for a particular service, or, equivalently, in high utilizations for a particular service. If requests are spaced exactly uniformly, and the time to service a request is less than or equal to the time before the next request arrives, then each request will find the resource idle and can therefore start execution immediately without waiting. Unfortunately, the arrival of requests in a database system is never so uniform and is often random.

If a resource, such as a disk, has a low utilization, then when a request is made, the resource is likely to be idle, in which case the waiting time for the request will be 0. Assuming uniformly randomly distributed arrivals, the length of the queue (and

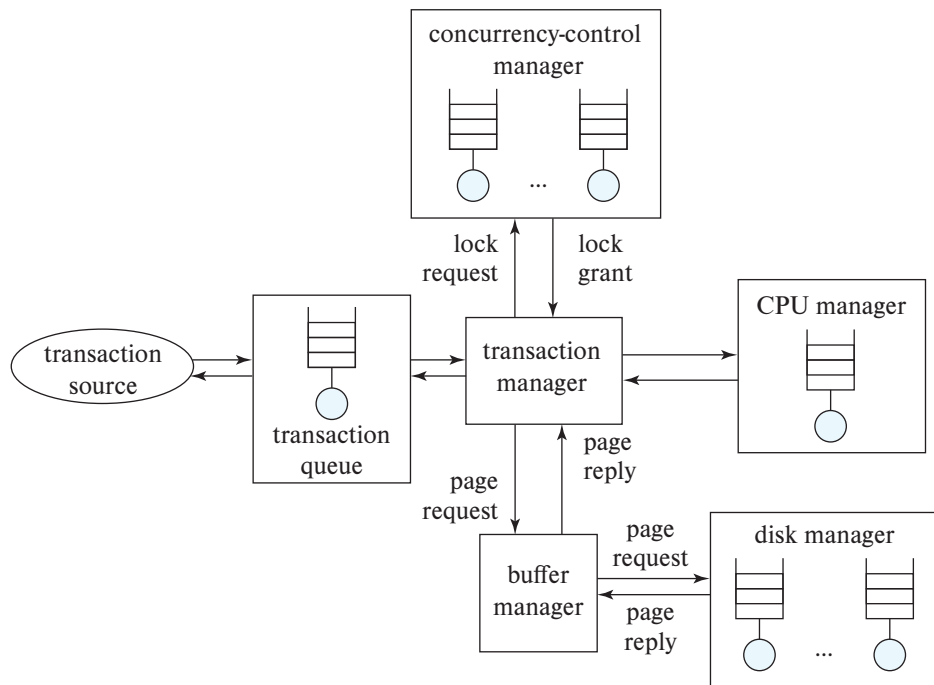


Figure 25.1 Queues in a database system.

correspondingly the waiting time) goes up exponentially with utilization; as utilization approaches 100 percent, the queue length increases sharply, resulting in excessively long waiting times. The utilization of a resource should be kept low enough that queue length is short. As a rule of the thumb, utilizations of around 70 percent are considered to be good, and utilizations above 90 percent are considered excessive, since they will result in significant delays. To learn more about the theory of queueing systems, generally referred to as [queueing theory](#), you can consult the references cited in the bibliographical notes.

25.1.3 Tuning Levels

Tuning is typically done in the context of applications, and can be done at the database system layer, or outside the database system.

Tuning at layers above the database is application dependent, and is not our focus, but we mention a few such techniques. Profiling application code to find code blocks that have a heavy CPU consumption, and rewriting them to reduce CPU load is an option for CPU intensive applications. Application servers often have numerous parameters that can be tuned to improve performance, or to ensure that the application does not run out of memory. Multiple application servers that work in parallel are often

used to handle higher workloads. A *load balancer* is used to route requests to one of the application servers; to ensure session continuity, requests from a particular source are always routed to the same application server. Connection pooling (described in Section 9.7.1) is another widely technique to reduce the overhead of database connection creation. Web application interfaces may be tuned to improve responsiveness, for example by replacing legacy web interfaces by ones based on JavaScript and Ajax (described in Section 9.5.1.3).

Returning to database tuning, database administrators and application developers can tune a database system at three levels.

The highest level of database tuning, which is under the control of application developers, includes the schema and queries. The developer can tune the design of the schema, the indices that are created, and the transactions that are executed to improve performance. Tuning at this level is comparatively system independent.

The second level consists of the database-system parameters, such as buffer size and checkpointing intervals. The exact set of database-system parameters that can be tuned depends on the specific database system. Most database-system manuals provide information on what database-system parameters can be adjusted, and how you should choose values for the parameters. Well-designed database systems perform as much tuning as possible automatically, freeing the user or database administrator from the burden. For instance, in many database systems the buffer size is fixed but tunable. If the system automatically adjusts the buffer size by observing indicators such as page-fault rates, then the database administrator will not have to worry about tuning the buffer size.

The lowest level is at the hardware level. Options for tuning systems at this level include replacing hard disks with solid-state drives (which use flash storage), adding more disks or using a RAID system if disk I/O is a bottleneck, adding more memory if the disk buffer size is a bottleneck, or moving to a system with more processors if CPU usage is a bottleneck.

The three levels of tuning interact with one another; we must consider them together when tuning a system. For example, tuning at a higher level may result in the hardware bottleneck changing from the disk system to the CPU, or vice versa. Tuning of queries and the physical schema is usually the first step to improving performance. Tuning of database system parameters, in case the database system does automate this task, can also be done in parallel. If performance is still poor, tuning of logical schema and tuning of hardware are the next logical steps.

25.1.4 Tuning of Physical Schema

Tuning of the physical schema, such as indices and materialized views, is the least disruptive mode of tuning, since it does not affect application code in any way. We now study different aspects of tuning of the physical schema.

25.1.4.1 Tuning of Indices

We can tune the indices in a database system to improve performance. If queries are the bottleneck, we can often speed them up by creating appropriate indices on relations. If updates are the bottleneck, there may be too many indices, which have to be updated when the relations are updated. Removing indices may speed up certain updates.

The choice of the type of index also is important. Some database systems support different kinds of indices, such as hash indices, B⁺-tree indices, and write-optimized indices such as LSM trees (Section 24.2). If range queries are common, B⁺-tree indices are preferable to hash indices. If the system has a very high write load, but a relatively low read load, write-optimized LSM tree indices may be preferable to B⁺-tree indices.

Whether to make an index a clustered index is another tunable parameter. Only one index on a relation can be made clustered, by storing the relation sorted on the index attributes. Generally, the index that benefits the greatest number of queries and updates should be made clustered.

To help identify what indices to create, and which index (if any) on each relation should be clustered, most commercial database systems provide *tuning wizards*; these are described in more detail in Section 25.1.4.4. These tools use the past history of queries and updates (called the *workload*) to estimate the effects of various indices on the execution time of the queries and updates in the workload. Recommendations on what indices to create are based on these estimates.

25.1.4.2 Using Materialized Views

Maintaining materialized views can greatly speed up certain types of queries, in particular aggregate queries. Recall the example from Section 16.5 where the total salary for each department (obtained by summing the salary of each instructor in the department) is required frequently. As we saw in that section, creating a materialized view storing the total salary for each department can greatly speed up such queries.

Materialized views should be used with care, however, since there is not only space overhead for storing them but, more important, there is also time overhead for maintaining materialized views. In the case of **immediate view maintenance**, if the updates of a transaction affect the materialized view, the materialized view must be updated as part of the same transaction. The transaction may therefore run slower. In the case of **deferred view maintenance**, the materialized view is updated later; until it is updated, the materialized view may be inconsistent with the database relations. For instance, the materialized view may be brought up to date when a query uses the view, or periodically. Using deferred maintenance reduces the burden on update transactions.

The database administrator is responsible for the selection of materialized views and for view-maintenance policies. The database administrator can make the selection manually by examining the types of queries in the workload and finding out which queries need to run faster and which updates/queries may be executed more slowly. From the examination, the database administrator may choose an appropriate set of

materialized views. For instance, the administrator may find that a certain aggregate is used frequently, and choose to materialize it, or may find that a particular join is computed frequently, and choose to materialize it.

However, manual choice is tedious for even moderately large sets of query types, and making a good choice may be difficult, since it requires understanding the costs of different alternatives; only the query optimizer can estimate the costs with reasonable accuracy without actually executing the query. Thus, a good set of views may be found only by trial and error—that is, by materializing one or more views, running the workload, and measuring the time taken to run the queries in the workload. The administrator repeats the process until a set of views is found that gives acceptable performance.

A better alternative is to provide support for selecting materialized views within the database system itself, integrated with the query optimizer. This approach is described in more detail in Section 25.1.4.4.

25.1.4.3 Horizontal Partitioning of Relation Schema

Horizontal partitioning of relations is widely used for parallel and distributed storage and query processing. However, it can also be used in a centralized system to improve queries and updates by breaking up the tuples of a relation into partitions.

For example, suppose that a database stores a large relation that has a *date* attribute, and most operations work on data inserted within the past few months. Suppose now that the relation is partitioned on the *date* attribute, with one partition for each (*year*, *month*) combination. Then, queries that contain a selection on *date*, such as *date*='2018-06-01', need access only partitions that could possibly contain such tuples, skipping all other partitions.

More importantly, indices could be created independently on each partition. Suppose an index is created on an attribute *ID*, with a separate index on each partition. A query that specifies selection on *ID*, along with a date or a date range, need look up the index on only those partitions that match the specified date or date range. Since each partition is smaller than the whole relation, the indices too are smaller, speeding up index lookup. Index insertion is also much faster, since the index size is much smaller than an index on the entire relation. And most importantly, even as the total data size grows, the partition size never grows beyond some limit, ensuring that the performance of such queries does not degrade with time.

There is a cost to such partitioning: queries that do not contain a selection on the partitioning attribute need to individually access each of the partitions, potentially slowing down such queries significantly. If such queries are rare, the benefits of partitioning outweigh the costs, making them an attractive technique for optimization.

Even if the database does not support partitioning internally, it is possible to replace a relation r by multiple physical relations $r_1, r_2, \dots, r_n$, and the original relation r is defined by the view $r = r_1 \cup r_2 \cup \dots \cup r_n$. Suppose that the database optimizer knows

the predicate defining each r_i (in our example, the date range corresponding to each r_i). Then the optimizer can replace a query on r that includes a selection on the partitioning attribute (*date*, in our example), with a query on the only relevant r_i s. Indices would have to be created separately on each of the r_i s.

25.1.4.4 Automated Tuning of Physical Design

Most commercial database systems today provide tools to help the database administrator with index and materialized view selection and other tasks related to physical database design such as how to partition data in a parallel database system.

These tools examine the **workload** (the history of queries and updates) and suggest indices and views to be materialized. The database administrator may specify the importance of speeding up different queries, which the tool takes into account when selecting views to materialize. Often tuning must be done before the application is fully developed, and the actual database contents may be small on the development database but are expected to be much larger on a production database. Thus, some tuning tools also allow the database administrator to specify information about the expected size of the database and related statistics.

Microsoft's Database Tuning Assistant, for example, allows the user to ask "what if" questions, whereby the user can pick a view, and the optimizer then estimates the effect of materializing the view on the total cost of the workload and on the individual costs of different types of queries and updates in the workload.

The automatic selection of indices and materialized views is usually implemented by enumerating different alternatives and using the query optimizer to estimate the costs and benefits of selecting each alternative by using the workload. Since the number of design alternatives and the potential workload may be extremely large, the selection techniques must be designed carefully.

The first step is to generate a workload. This is usually done by recording all the queries and updates that are executed during some time period. Next, the selection tools perform **workload compression**, that is, create a representation of the workload using a small number of updates and queries. For example, updates of the same form can be represented by a single update with a weight corresponding to how many times the update occurred. Queries of the same form can be similarly replaced by a representative with appropriate weight. After this, queries that are very infrequent and do not have a high cost may be discarded from consideration. The most expensive queries may be chosen to be addressed first. Such workload compression is essential for large workloads.

With the help of the optimizer, the tool would come up with a set of indices and materialized views that could help the queries and updates in the compressed workload. Different combinations of these indices and materialized views can be tried out to find the best combination. However, an exhaustive approach would be totally impractical, since the number of potential indices and materialized views is already large, and each

Note 25.2 TUNING TOOLS

Tuning tools, such as the Database Engine Tuning Advisor provided by SQL Server and the SQL Tuning Advisor of Oracle, provide recommendations such as what indices or materialized views to add, or how to partition a relation, to improve performance. These recommendations can then be accepted and implemented by a database administrator.

Auto Tuning in Microsoft Azure SQL can automatically create and drop indices to improve query performance. A risk with automatically changing the physical schema is that some queries may perform poorly. For example, an optimizer may choose a plan using a newly created index, assuming, based on wrong estimates of cost, that the new plan is cheaper than the plan used before the index was created. In reality, the query may run slower using the new plan, which may affect users. The “force last good plan” feature can monitor query performance after any change such as addition of an index, and if performance is worse, it can force the database to use the old plan before the change (as long as it is still valid).

Oracle also provides auto tuning support, for example recommending if an index should be added, or monitoring the use of a query to decide if it should be optimized for fetching only a few rows or for fetching all rows (the best plan may be very different if only the first few rows are fetched or if all rows are fetched).

subset of these is a potential design alternative, leading to an exponential number of alternatives. Heuristics are used to reduce the space of alternatives, that is, to reduce the number of combinations considered.

Greedy heuristics for index and materialized view selection operate as follows: They estimate the benefits of materializing different indices or views (using the optimizer’s cost estimation functionality as a subroutine). They then choose the index or view that gives either the maximum benefit or the maximum benefit per unit space (i.e., benefit divided by the space required to store the index or view). The cost of maintaining the index or view must be taken into account when computing the benefit. Once the heuristic has selected an index or view, the benefits of other indices or views may have changed, so the heuristic recomputes these and chooses the next best index or view for materialization. The process continues until either the available disk space for storing indices or materialized views is exhausted or the cost of maintaining the remaining candidates is more than the benefit to queries that could use the indices or views.

Real-world index and materialized-view selection tools usually incorporate some elements of greedy selection but use other techniques to get better results. They also support other aspects of physical database design, such as deciding how to partition a relation in a parallel database, or what physical storage mechanism to use for a relation.

25.1.5 Tuning of Queries

The performance of an application can often be significantly improved by rewriting queries or by changing how the application issues queries to the database.

25.1.5.1 Tuning of Query Plans

In the past, optimizers on many database systems were not particularly good, so how a query was written would have a big influence on how it was executed, and therefore on the performance. Today's advanced optimizers can transform even badly written queries and execute them efficiently, so the need for tuning individual queries is less important than it used to be. However, sometimes query optimizers choose bad plans for one of several reasons, which we describe next.

Before checking if something needs to be tuned in the plan for a query, it is useful to find out what plan is being used for the query. Most databases support an `explain` command, which allows you to see what plan is being used for a query. The `explain` command also shows the statistics that the optimizer used or computed for different parts of the query plan, and estimates of the costs of each part of a query plan. Variants of the `explain` command also execute the query and get actual tuple counts and execution time for different parts of the query plan.

Incorrect statistics are often the reason for the choice of a bad plan. For example, if the optimizer thinks that the relations involved in a join have very few tuples, it may choose nested loops join, which would be very inefficient if the relations actually have a large number of tuples.

Ideally, database statistics should be updated whenever relations are updated. However, doing so adds unacceptable overhead to update queries. Instead, databases either periodically update statistics or leave it to the system administrator to issue a command to update statistics. Some databases, such as PostgreSQL and MySQL support a command called `analyze`,¹ which can be used to recompute statistics. For example, `analyze instructor` would recompute statistics for the `instructor` relation, while `analyze` with no arguments would recompute statistics for all relations in PostgreSQL. It is highly recommended to run this command after loading data into the database, or after making a significant number of inserts or deletes on a relation.

Some databases such as Oracle and Microsoft SQL Server keep track of inserts and deletes to relations, and they update statistics whenever the relation size changes by a significant fraction, making execution of the `analyze` command unnecessary.

Another reason for poor performance of queries is the lack of required indices. As we saw earlier, the choice of indices can be done as part of the tuning of the physical schema, but examining a query helps us understand what indices may be useful to speed up that query.

Indices are particularly important for queries that fetch only a few rows from a large relation, based on a predicate. For example, a query that finds students in a department

¹The command is called `analyze table` in the case of MySQL.

may benefit from an index on the *student* relation on the attribute *dept_name*. Indices on join attributes are often very useful. For example, if the above query also included a join of *student* with *takes* on the attribute *takes.ID*, an index on *takes.ID* could be useful.

Note that databases typically create indices on primary-key attributes, which can be used for selections as well as joins. For example, in our university schema, the primary-key index on *takes* has *ID* as its first attribute and may thus be useful for the above join.

Complex queries containing *nested subqueries* are not optimized very well by many optimizers. We saw techniques for nested subquery decorrelation in Section 16.4.4. If a subquery is not decorrelated, it gets executed repeatedly, potentially resulting in a great deal of random I/O. In contrast, decorrelation allows efficient set-oriented operations such as joins to be used, minimizing random I/O. Most database query optimizers incorporate some forms of decorrelation, but some can handle only very simple nested subqueries. The execution plan chosen by the optimizer can be found as described in Chapter 16. If the optimizer has not succeeded in decorrelating a nested subquery, the query can be decorrelated by rewriting it manually.

25.1.5.2 Improving Set Orientation

When SQL queries are executed from an application program, it is often the case that a query is executed frequently, but with different values for a parameter. Each call has an overhead of communication with the server, in addition to processing overheads at the server.

For example, consider a program that steps through each department, invoking an embedded SQL query to find the total salary of all instructors in the department:

```
select sum(salary)
from instructor
where dept_name= ?
```

If the *instructor* relation does not have a clustered index on *dept_name*, each such query will result in a scan of the relation. Even if there is such an index, a random I/O operation will be required for each *dept_name* value.

Instead, we can use a single SQL query to find total salary expenses of each department:

```
select dept_name, sum(salary)
from instructor
group by dept_name;
```

This query can be evaluated with a single scan of the *instructor* relation, avoiding random I/O for each department. The results can be fetched to the client side using a single round of communication, and the client program can then step through the results to find the aggregate for each department. Combining multiple SQL queries into a single

```
PreparedStatement pstmt = conn.prepareStatement(
    "insert into instructor values(?,?,?,?)");
pstmt.setString(1, "88877");
pstmt.setString(2, "Perry");
pstmt.setInt(3, "Finance");
pstmt.setInt(4, 125000);
pstmt.addBatch( );
pstmt.setString(1, "88878");
pstmt.setString(2, "Thierry");
pstmt.setInt(3, "Physics");
pstmt.setInt(4, 100000);
pstmt.addBatch( );
pstmt.executeBatch( );
```

Figure 25.2 Batch update in JDBC.

SQL query as above can reduce execution costs greatly in many cases—for example, if the *instructor* relation is very large and has a large number of departments.

The JDBC API also provides a feature called **batch update** that allows a number of inserts to be performed using a single communication with the database. Figure 25.2 illustrates the use of this feature. The code shown in the figure requires only one round of communication with the database, when the `executeBatch()` method is executed, in contrast to similar code without the batch update feature that we saw in Figure 5.2. In the absence of batch update, as many rounds of communication with the database are required as there are instructors to be inserted. The batch update feature also enables the database to process a batch of inserts at once, which can potentially be done much more efficiently than a series of single record inserts.

Another technique used widely in client-server systems to reduce the cost of communication and SQL compilation is to use stored procedures, where queries are stored at the server in the form of procedures, which may be precompiled. Clients can invoke these stored procedures rather than communicate a series of queries.

25.1.5.3 Tuning of Bulk Loads and Updates

When loading a large volume of data into a database (called a **bulk load** operation), performance is usually very poor if the inserts are carried out as separate SQL insert statements. One reason is the overhead of parsing each SQL query; a more important reason is that performing integrity constraint checks and index updates separately for each inserted tuple results in a large number of random I/O operations. If the inserts were done as a large batch, integrity-constraint checking and index update can be done

in a much more set-oriented fashion, reducing overheads greatly; performance improvements of an order of magnitude or more are not uncommon.

To support bulk load operations, most database systems provide a **bulk import** utility and a corresponding **bulk export** utility. The bulk-import utility reads data from a file and performs integrity constraint checking as well as index maintenance in a very efficient manner. Common input and output file formats supported by such bulk import/export utilities include text files with characters such as commas or tabs separating attribute values, with each record in a line of its own (such file formats are referred to as *comma-separated values* or *tab-separated values* formats). Database-specific binary formats as well as XML formats are also supported by bulk import/export utilities. The names of the bulk import/export utilities differ by database. In PostgreSQL, the utilities are called `pg_dump` and `pg_restore` (PostgreSQL also provides an SQL command `copy`, which provides similar functionality). The bulk import/export utility in Oracle is called **SQL*Loader**, the utility in DB2 is called `load`, and the utility in SQL Server is called `bcp` (SQL Server also provides an SQL command called **bulk insert**).

We now consider the case of tuning of bulk updates. Suppose we have a relation *funds_received*(*dept_name*, *amount*) that stores funds received (say, by electronic funds transfer) for each of a set of departments. Suppose now that we want to add the amounts to the balances of the corresponding department budgets. In order to use the SQL update statement to carry out this task, we have to perform a look up on the *funds_received* relation for each tuple in the *department* relation. We can use subqueries in the update clause to carry out this task, as follows: We assume for simplicity that the relation *funds_received* contains at most one tuple for each department.

```
update department set budget = budget +
    (select amount
     from funds_received
     where funds_received.dept_name = department.dept_name)
where exists(
    select *
    from funds_received
    where funds_received.dept_name = department.dept_name);
```

Note that the condition in the **where** clause of the update ensures that only accounts with corresponding tuples in *funds_received* are updated, while the subquery within the **set** clause computes the amount to be added to each such department.

There are many applications that require updates such as that illustrated above. Typically, there is a table, which we shall call the **master table**, and updates to the master table are received as a batch. Now the master table has to be correspondingly updated. SQL:2003 introduced a special construct, called the **merge** construct, to simplify the task of performing such merging of information. For example, the preceding update can be expressed using **merge** as follows:

```

merge into department as A
using      (select *
           from funds_received) as F
on (A.dept_name = F.dept_name)
when matched then
    update set budget = budget + F.amount;

```

When a record from the subquery in the **using** clause matches a record in the *department* relation, the **when matched** clause is executed, which can execute an update on the relation; in this case, the matching record in the *department* relation is updated as shown.

The **merge** statement can also have a **when not matched then** clause, which permits insertion of new records into the relation. In the preceding example, when there is no matching department for a *funds_received* tuple, the insertion action could create a new department record (with a null *building*) using the following clause:

```

when not matched then
    insert values (F.dept_name, null, F.budget)

```

Although not very meaningful in this example,² the **when not matched then** clause can be quite useful in other cases. For example, suppose the local relation is a copy of a master relation, and we receive updated as well as newly inserted records from the master relation. The **merge** statement can update matched records (these would be updated old records) and insert records that are not matched (these would be new records).

Not all SQL implementations support the **merge** statement currently; see the respective system manuals for further details.

25.1.6 Tuning of the Logical Schema

Performance of queries can sometimes be improved by tuning of the logical schema. For example, within the constraints of the chosen normal form, it is possible to partition relations vertically. Consider the *course* relation, with the schema:

course (*course_id*, *title*, *dept_name*, *credits*)

for which *course_id* is a key. Within the constraints of the normal forms (BCNF and 3NF), we can partition the *course* relation into two relations:

course_credit (*course_id*, *credits*)
course_title_dept (*course_id*, *title*, *dept_name*)

²A better action here would have been to insert these records into an error relation, but that cannot be done with the **merge** statement.

The two representations are logically equivalent, since *course_id* is a key, but they have different performance characteristics.

If most accesses to course information look at only the *course_id* and *credits*, then they can be run against the *course_credit* relation, and access is likely to be somewhat faster, since the *title* and *dept_name* attributes are not fetched. For the same reason, more tuples of *course_credit* will fit in the buffer than corresponding tuples of *course*, again leading to faster performance. This effect would be particularly marked if the *title* and *dept_name* attributes were large. Hence, a schema consisting of *course_credit* and *course_title_dept* would be preferable to a schema consisting of the *course* relation in this case.

On the other hand, if most accesses to course information require both *dept_name* and *credits*, using the *course* relation would be preferable, since the cost of the join of *course_credit* and *course_title_dept* would be avoided. Also, the storage overhead would be lower, since there would be only one relation, and the attribute *course_id* would not be replicated.

The **column store** approach to storing data are based on vertical partitioning but takes it to the limit by storing each attribute (column) of the relation in a separate file, as we saw in Section 13.6. Note that in a column store it is not necessary to repeat the primary-key attribute since the i^{th} row can be reconstructed by taking the i^{th} entry for each desired column. Column stores have been shown to perform well for several data-warehouse applications by reducing I/O, improving cache performance, enabling greater gains from data compression, and allowing effective use of CPU vector-processing capabilities.

Another trick to improve performance is to store a *denormalized relation*, such as a join of *instructor* and *department*, where the information about *dept_name*, *building*, and *budget* is repeated for every instructor. More effort has to be expended to make sure the relation is consistent whenever an update is carried out. However, a query that fetches the names of the instructors and the associated buildings will be speeded up, since the join of *instructor* and *department* will have been precomputed. If such a query is executed frequently, and has to be performed as efficiently as possible, the denormalized relation could be beneficial.

Materialized views can provide the benefits that denormalized relations provide, at the cost of some extra storage. A major advantage to materialized views over denormalized relations is that maintaining consistency of redundant data becomes the job of the database system, not the programmer. Thus, materialized views are preferable, whenever they are supported by the database system.

Another approach to speed up the computation of the join without materializing it is to cluster records that would match in the join on the same disk page. We saw such clustered file organizations in Section 13.3.3.

25.1.7 Tuning of Concurrent Transactions

Concurrent execution of different types of transactions can sometimes lead to poor performance because of contention on locks. We first consider the case of read-write

contention, which is more common, and then consider the case of write-write contention.

As an example of **read-write contention**, consider the following situation on a banking database. During the day, numerous small update transactions are executed almost continuously. Suppose that a large query that computes statistics on branches is run at the same time. If the query performs a scan on a relation, it may block out all updates on the relation while it runs, and that can have a disastrous effect on the performance of the system.

Several database systems—Oracle, PostgreSQL, and Microsoft SQL Server, for example— support snapshot isolation, whereby queries are executed on a snapshot of the data, and updates can go on concurrently. (Snapshot isolation is described in detail in Section 18.8.) Snapshot isolation should be used, if available, for large queries, to avoid lock contention in the above situation. In SQL Server, executing the statement

set transaction isolation level snapshot

at the beginning of a transaction results in snapshot isolation being used for that transaction. In Oracle and PostgreSQL, using the keyword **serializable** in place of the keyword **snapshot** in the above command has the same effect, since these systems actually use snapshot isolation (serializable snapshot isolation, in the case of PostgreSQL version 9.1 onwards) when the isolation level is set to serializable.

If snapshot isolation is not available, an alternative option is to execute large queries at times when updates are few or nonexistent. However, for databases supporting web sites, there may be no such quiet period for updates.

Another alternative is to use weaker levels of consistency, such as the **read committed** isolation level, whereby evaluation of the query has a minimal impact on concurrent updates, but the query results are not guaranteed to be consistent. The application semantics determine whether approximate (inconsistent) answers are acceptable.

We now consider the case of **write-write contention**. Data items that are updated very frequently can result in poor performance with locking, with many transactions waiting for locks on those data items. Such data items are called **update hot spots**. Update hot spots can cause problems even with snapshot isolation, causing frequent transaction aborts due to write-validation failures. A commonly occurring situation that results in an update hot spot is as follows: transactions need to assign unique identifiers to data items being inserted into the database, and to do so they read and increment a sequence counter stored in a tuple in the database. If inserts are frequent, and the sequence counter is locked in a two-phase manner, the tuple containing the sequence counter becomes a hot spot.

One way to improve concurrency is to release the lock on the sequence counter immediately after it is read and incremented; however, after doing so, even if the transaction aborts, the update to the sequence counter should not be rolled back. To understand why, suppose T_1 increments the sequence counter, and then T_2 increments the sequence counter before T_1 commits; if T_1 then aborts, rolling back its update, either

by restoring the counter to the original value or by decrementing the counter, will result in the sequence value used by T_2 getting reused by a subsequent transaction.

Most databases provide a special construct for creating **sequence counters** that implement early, non-two-phase lock release, coupled with special-case treatment of undo logging so that updates to the counter are not rolled back if the transaction aborts. The SQL standard allows a sequence counter to be created using the command:

```
create sequence counter1;
```

In the above command, *counter1* is the name of the sequence; multiple sequences can be created with different names. The syntax to get a value from the sequence is not standardized; in Oracle, *counter1.nextval* would return the next value of the sequence, after incrementing it, while the function call *nextval ('counter1')* would have the same effect in PostgreSQL, and DB2 uses the syntax **nextval for** *counter1*.

The SQL standard provides an alternative to using an explicit sequence counter, which is useful when the goal is to give unique identifiers to tuples inserted into a relation. To do so, the keyword **identity** can be added to the declaration of an integer attribute of a relation (usually this attribute would also be declared as the primary key). If the value for that attribute is left unspecified in an insert statement for that relation, a unique new value is created automatically for each newly inserted tuple. A non-two-phase locked sequence counter is used internally to implement the **identity** declaration, with the counter incremented each time a tuple is inserted. Several databases, including DB2 and SQL Server support the **identity** declaration, although the syntax varies. PostgreSQL supports a data type called **serial**, which provides the same effect; the PostgreSQL type **serial** is implemented by transparently creating a non-two-phase locked sequence.

It is worth noting that since the acquisition of a sequence number by a transaction cannot be rolled back if the transaction aborts (for reasons discussed earlier), transaction aborts may result in *gaps in the sequence numbers* in tuples inserted in the database. For example, there may be tuples with identifier value 1001 and 1003, but no tuple with value 1002, if the transaction that acquired the sequence number 1002 did not commit. Such gaps are not acceptable in some applications; for example, some financial applications require that there be no gaps in bill or receipt numbers. Database provided sequences and automatically incremented attributes should not be used for such applications, since they can result in gaps. A sequence counter stored in normal tuples, which is locked in a two-phase manner, would not be susceptible to such gaps since a transaction abort would restore the sequence counter value, and the next transaction would get the same sequence number, avoiding a gap.

Long update transactions can cause performance problems with system logs and can increase the time taken to recover from system crashes. If a transaction performs many updates, the system log may become full even before the transaction completes, in which case the transaction will have to be rolled back. If an update transaction runs for a long time (even with few updates), it may block deletion of old parts of the log,

if the logging system is not well designed. Again, this blocking could lead to the log getting filled up.

To avoid such problems, many database systems impose strict limits on the number of updates that a single transaction can carry out. Even if the system does not impose such limits, it is often helpful to break up a large update transaction into a set of smaller update transactions where possible. For example, a transaction that gives a raise to every employee in a large corporation could be split up into a series of small transactions, each of which updates a small range of employee-ids. Such transactions are called **minibatch transactions**. However, minibatch transactions must be used with care. First, if there are concurrent updates on the set of employees, the result of the set of smaller transactions may not be equivalent to that of the single large transaction. Second, if there is a failure, the salaries of some of the employees would have been increased by committed transactions, but salaries of other employees would not. To avoid this problem, as soon as the system recovers from failure, we must execute the transactions remaining in the batch.

Long transactions, whether read-only or update, can also result in the lock table becoming full. If a single query scans a large relation, the query optimizer would ensure that a relation lock is obtained instead of acquiring a large number of tuple locks. However, if a transaction executes a large number of small queries or updates, it may acquire a large number of locks, resulting in the lock table becoming full.

To avoid this problem, some databases provide for automatic **lock escalation**; with this technique, if a transaction has acquired a large number of tuple locks, tuple locks are upgraded to page locks, or even full relation locks. Recall that with multiple-granularity locking (Section 18.3), once a coarser-level lock is obtained, there is no need to record finer-level locks, so tuple lock entries can be removed from the lock table, freeing up space. On databases that do not support lock escalation, it is possible for the transaction to explicitly acquire a relation lock, thereby avoiding the acquisition of tuple locks.

25.1.8 Tuning of Hardware

Hardware bottlenecks could include memory, I/O, CPU and network capacity. We focus on memory and I/O tuning in this section. The availability of processors with a large number of CPU cores, and support for multiple CPUs on a single machine allows system designers to choose the CPU model and number of CPUs to meet the CPU requirements of the application at an acceptable cost. How to tune or choose between CPU and network interconnect options is a topic outside the domain of database tuning.

Even in a well-designed transaction processing system, each transaction usually has to do at least a few I/O operations, if the data required by the transaction are on disk. An important factor in tuning a transaction processing system is to make sure that the disk subsystem can handle the rate at which I/O operations are required. For instance, consider a hard disk that supports an access time of about 10 milliseconds, and average transfer rate of 25 to 100 megabytes per second (a fairly typical disk today). Such a disk

would support a little under 100 random-access I/O operations of 4 kilobytes each per second. If each transaction requires just two I/O operations, a single disk would support at most 50 transactions per second.

An obvious way to improve performance is to replace a hard disk with a solid-state drive (SSD), since a single SSD can support tens of thousands of random I/O operations per second. A drawback of using SSDs is that they cost a lot more than hard disks for a given storage capacity. Another way to support more transactions per second is to increase the number of disks. If the system needs to support n transactions per second, each performing two I/O operations, data must be striped (or otherwise partitioned) across at least $n/50$ hard disks (ignoring skew), or $n/5000$ SSDs, if the SSD supports 10,000 random I/O operations per second.

Notice here that the limiting factor is not the capacity of the disk, but the speed at which random data can be accessed (limited in a hard disk by the speed at which the disk arm can move). The number of I/O operations per transaction can be reduced by storing more data in memory. If all data are in memory, there will be no disk I/O except for writes. Keeping frequently used data in memory reduces the number of disk I/Os and is worth the extra cost of memory. Keeping very infrequently used data in memory would be a waste, since memory is much more expensive than disk.

The question is, for a given amount of money available for spending on disks or memory, what is the best way to spend the money to achieve the maximum number of transactions per second? A reduction of one I/O per second saves:

$$(price\ per\ disk\ drive)/(access\ per\ second\ per\ disk)$$

Thus, if a particular page is accessed once in m seconds, the saving due to keeping it in memory is $\frac{1}{m}$ times the above value. Storing a page in memory costs:

$$(price\ per\ megabyte\ of\ memory)/(pages\ per\ megabyte\ of\ memory)$$

Thus, the break-even point is:

$$\frac{1}{m} * \frac{price\ per\ disk\ drive}{access\ per\ second\ per\ disk} = \frac{price\ per\ megabyte\ of\ memory}{pages\ per\ megabyte\ of\ memory}$$

We can rearrange the equation and substitute current values for each of the above parameters to get a value for m ; if a page is accessed more frequently than once in m seconds, it is worth buying enough memory to store it.

As of 2018, hard-disk technology and memory and disk prices (which we assume to be about \$50 for a 1-terabyte disk and \$80 for 16-gigabytes of memory) give a value of m around 4 hours for 4-kilobytes pages that are randomly accessed; that is, if a page on hard disk is accessed at least once in 4 hours, it makes sense to purchase enough memory to cache it in memory. Note that if we use larger pages, the time decreases; for example, a page size of 16-kilobytes will lead to a value of m of 1 hour instead of 4 hours.

With disk and memory cost and speeds as of the 1980/1990s, the corresponding value was 5 minutes with 4-kilobytes pages. Thus, a widely used rule of thumb, called the **five minute rule**, which said that data should be cached in memory if it is accessed more frequently than once in 5 minutes.

With SSD technology and prices as of 2018 (which we assume to be around \$500 for a 800 gigabytes SSD, which supports 67,000 random reads and 20,000 random writes per second), if we make the same comparison between keeping a page in memory versus fetching it from SSD, the time comes to around 7 minutes with 4-kilobyte pages. That is, if a page on SSD is accessed more frequently than once in 7 minutes, it is worth purchasing enough memory to cache it in memory.

For data that are sequentially accessed, significantly more pages can be read per second. Assuming 1 megabyte of data are read at a time, the breakeven point for hard disk currently is about 2.5 minutes. Thus, sequentially accessed data on hard disk should be cached in memory if they are used at least once in 2.5 minutes. For SSDs, the breakeven point is much smaller, at 1.6 seconds. In other words, there is little benefit in caching sequentially accessed data in memory unless it is very frequently accessed.

The above rules of thumb take only the number of I/O operations per second into account and do not consider factors such as response time. Some applications need to keep even infrequently used data in memory to support response times that are less than or comparable to disk-access time.

Since SSD storage is more expensive than disk, one way to get faster random I/O for frequently used data, while paying less for storing less frequently used data, is to use the **flash-as-buffer** approach. In this approach, flash storage is used as a persistent buffer, with each block having a permanent location on disk, but stored in flash instead of being written to disk as long as it is frequently used. When flash storage is full, a block that is not frequently used is evicted and flushed back to disk if it was updated after being read from disk. Disk subsystems that provide hard disks along with SSDs that act as buffers are commercially available. A rule of thumb for deciding how much SSD storage to purchase is that a 4-kilobyte page should be kept on SSD, instead of hard disk, if it is accessed more frequently than once in a day (the computation is similar to the case of caching in main memory versus fetching from disk/SSD). Note that in such a setup, the database system cannot control what data reside in which part of the storage.

If the storage system allows direct access to SSDs as well as hard disks, the database administrator can control the mapping of relations or indices to disks and allocate frequently used relations/indices to flash storage. The tablespace feature, supported by most database systems, can be used to control the mapping by creating a tablespace on flash storage and assigning desired relations and indices to that tablespace. Controlling the mapping at a finer level of granularity than a relation, however, requires changes to the database-system code.

Another aspect of tuning is whether to use RAID 1 or RAID 5. The answer depends on how frequently the data are updated, since RAID 5 is much slower than RAID 1 on

random writes: RAID 5 requires 2 reads and 2 writes to execute a single random write request. If an application performs r random reads and w random writes per second to support a particular throughput rate, a RAID 5 implementation would require $r + 4w$ I/O operations per second, whereas a RAID 1 implementation would require $r + 2w$ I/O operations per second. We can then calculate the number of disks required to support the required I/O operations per second by dividing the result of the calculation by 100 I/O operations per second (for current-generation disks). For many applications, r and w are large enough that the $(r + w)/100$ disks can easily hold two copies of all the data. For such applications, if RAID 1 is used, the required number of disks is actually less than the required number of disks if RAID 5 is used! Thus, RAID 5 is useful only when the data storage requirements are very large, but the update rates, and particularly random update rates, are small.

25.1.9 Performance Simulation

To test the performance of a database system even before it is installed, we can create a performance-simulation model of the database system. Each service shown in Figure 25.1, such as the CPU, each disk, the buffer, and the concurrency control, is modeled in the simulation. Instead of modeling details of a service, the simulation model may capture only some aspects of each service, such as the **service time**—that is, the time taken to finish processing a request once processing has begun. Thus, the simulation can model a disk access from just the average disk-access time.

Since requests for a service generally have to wait their turn, each service has an associated queue in the simulation model. A transaction consists of a series of requests. The requests are queued up as they arrive and are serviced according to the policy for that service, such as first come, first served. The models for services such as CPU and the disks conceptually operate in parallel, to account for the fact that these subsystems operate in parallel in a real system.

Once the simulation model for transaction processing is built, the system administrator can run a number of experiments on it. The administrator can use experiments with simulated transactions arriving at different rates to find how the system would behave under various load conditions. The administrator could run other experiments that vary the service times for each service to find out how sensitive the performance is to each of them. System parameters, too, can be varied, so that performance tuning can be done on the simulation model.

25.2 Performance Benchmarks

As database servers become more standardized, the differentiating factor among the products of different vendors is those products' performance. **Performance benchmarks** are suites of tasks that are used to quantify the performance of software systems.

25.2.1 Suites of Tasks

Since most software systems, such as databases, are complex, there is a good deal of variation in their implementation by different vendors. As a result, there is a significant amount of variation in their performance on different tasks. One system may be the most efficient on a particular task; another may be the most efficient on a different task. Hence, a single task is usually insufficient to quantify the performance of the system. Instead, the performance of a system is measured by suites of standardized tasks, called *performance benchmarks*.

Combining the performance numbers from multiple tasks must be done with care. Suppose that we have two tasks, T_1 and T_2 , and that we measure the throughput of a system as the number of transactions of each type that run in a given amount of time—say, 1 second. Suppose that system A runs T_1 at 99 transactions per second and T_2 at 1 transaction per second. Similarly, let system B run both T_1 and T_2 at 50 transactions per second. Suppose also that a workload has an equal mixture of the two types of transactions.

If we took the average of the two pairs of numbers (i.e., 99 and 1, versus 50 and 50), it might appear that the two systems have equal performance. However, it is *wrong* to take the averages in this fashion—if we ran 50 transactions of each type, system A would take about 50.5 seconds to finish, whereas system B would finish in just 2 seconds!

The example shows that a simple measure of performance is misleading if there is more than one type of transaction. The right way to average out the numbers is to take the **time to completion** for the workload, rather than the average *throughput* for each transaction type. We can then compute system performance accurately in transactions per second for a specified workload. Thus, system A takes 50.5/100, which is 0.505 seconds per transaction, whereas system B takes 0.02 seconds per transaction, on average. In terms of throughput, system A runs at an average of 1.98 transactions per second, whereas system B runs at 50 transactions per second. Assuming that transactions of all the types are equally likely, the correct way to average out the throughputs on different transaction types is to take the **harmonic mean** of the throughputs. The harmonic mean of n throughputs $t_1, t_2, \dots, t_n$ is defined as:

$$\frac{n}{\frac{1}{t_1} + \frac{1}{t_2} + \dots + \frac{1}{t_n}}$$

For our example, the harmonic mean for the throughputs in system A is 1.98. For system B, it is 50. Thus, system B is approximately 25 times faster than system A on a workload consisting of an equal mixture of the two example types of transactions.

25.2.2 Database-Application Classes

Online transaction processing (OLTP) and **decision support**, including **online analytical processing (OLAP)**, are two broad classes of applications handled by database systems. These two classes of tasks have different requirements. High concurrency and clever

techniques to speed up commit processing are required for supporting a high rate of update transactions. On the other hand, good query-evaluation algorithms and query optimization are required for decision support. The architecture of some database systems has been tuned to transaction processing; that of others, such as the Teradata series of parallel database systems, has been tuned to decision support. Other vendors try to strike a balance between the two tasks.

Applications usually have a mixture of transaction-processing and decision-support requirements. Hence, which database system is best for an application depends on what mix of the two requirements the application has.

Suppose that we have throughput numbers for the two classes of applications separately, and the application at hand has a mix of transactions in the two classes. We must be careful even about taking the harmonic mean of the throughput numbers because of **interference** between the transactions. For example, a long-running decision-support transaction may acquire a number of locks, which may prevent all progress of update transactions. The harmonic mean of throughputs should be used only if the transactions do not interfere with one another.

25.2.3 The TPC Benchmarks

The **Transaction Processing Performance Council (TPC)** has defined a series of benchmark standards for database systems.

The TPC benchmarks are defined in great detail. They define the set of relations and the sizes of the tuples. They define the number of tuples in the relations not as a fixed number, but rather as a multiple of the number of claimed transactions per second, to reflect that a larger rate of transaction execution is likely to be correlated with a larger number of accounts. The performance metric is throughput, expressed as **transactions per second (TPS)**. When its performance is measured, the system must provide a response time within certain bounds, so that a high throughput cannot be obtained at the cost of very long response times. Further, for business applications, cost is of great importance. Hence, the TPC benchmark also measures performance in terms of **price per TPS**. A large system may have a high number of transactions per second, but it may be expensive (i.e., have a high price per TPS). Moreover, a company cannot claim TPC benchmark numbers for its systems *without* an external audit that ensures that the system faithfully follows the definition of the benchmark, including full support for the ACID properties of transactions.

The first in the series was the **TPC-A benchmark**, which was defined in 1989. This benchmark simulates a typical bank application by a single type of transaction that models cash withdrawal and deposit at a bank teller. The transaction updates several relations—such as the bank balance, the teller’s balance, and the customer’s balance—and adds a record to an audit-trail relation. The benchmark also incorporates communication with terminals, to model the end-to-end performance of the system realistically. The **TPC-B benchmark** was designed to test the core performance of the database system, along with the operating system on which the system runs. It removes the parts

of the TPC-A benchmark that deal with users, communication, and terminals, to focus on the backend database server. Neither TPC-A nor TPC-B is in use today.

The **TPC-C benchmark** was designed to model a more complex system than the TPC-A benchmark. The TPC-C benchmark concentrates on the main activities in an order-entry environment, such as entering and delivering orders, recording payments, checking status of orders, and monitoring levels of stock. The TPC-C benchmark is still widely used for benchmarking online transaction processing (OLTP) systems.

The more recent **TPC-E benchmark** is also aimed at OLTP systems but is based on a model of a brokerage firm, with customers who interact with the firm and generate transactions. The firm in turn interacts with financial markets to execute transactions.

The **TPC-D benchmark** was designed to test the performance of database systems on decision-support queries. Decision-support systems are becoming increasingly important today. The TPC-A, TPC-B, and TPC-C benchmarks measure performance on transaction-processing workloads and should not be used as a measure of performance on decision-support queries. The D in TPC-D stands for **decision support**. The TPC-D benchmark schema models a sales/distribution application, with parts, suppliers, customers, and orders, along with some auxiliary information. The sizes of the relations are defined as a ratio, and database size is the total size of all the relations, expressed in gigabytes. TPC-D at scale factor 1 represents the TPC-D benchmark on a 1-gigabyte database, while scale factor 10 represents a 10-gigabyte database. The benchmark workload consists of a set of 17 SQL queries modeling common tasks executed on decision-support systems. Some of the queries make use of complex SQL features, such as aggregation and nested queries.

The benchmark's users soon realized that the various TPC-D queries could be significantly speeded up by using materialized views and other redundant information. There are applications, such as periodic reporting tasks, where the queries are known in advance, and materialized views can be selected carefully to speed up the queries. It is necessary, however, to account for the overhead of maintaining materialized views.

The **TPC-H benchmark** (where "H" represents ad hoc) is a refinement of the TPC-D benchmark. The schema is the same, but there are 22 queries, of which 16 are from TPC-D. In addition, there are two updates, a set of inserts, and a set of deletes. TPC-H prohibits materialized views and other redundant information and permits indices only on primary and foreign keys. This benchmark models ad hoc querying where the queries are not known beforehand, so it is not possible to create appropriate materialized views ahead of time. A variant, TPC-R (where R stands for "reporting"), which is no longer in use, allowed the use of materialized views and other redundant information.

The **TPC-DS benchmark** is a follow-up to the TPC-H benchmark and models the decision-support functions of a retail product supplier, including information about customers, orders, and products, and with multiple sales channels such as retail stores and online sales. It has two subparts of the schema, corresponding to ad hoc querying and reporting, similar to TPC-H and TPC-R. There is a query workload, as well as a data maintenance workload.